
硕士学位论文

# 基于云边协同的工程机械故障诊断系统研究

# Research on Fault Diagnosis System for Construction Machinery Based on Cloud Edge Collaboration

作　者：伊斯罗贡
导　师：张博 副教授

中国矿业大学

二〇二五年六月

# 学位论文使用授权声明

本人完全了解中国矿业大学有关保留、使用学位论文的规定，同意本人所撰写的学位论文的使用授权按照学校的管理规定处理：

作为申请学位的条件之一，学位论文著作权拥有者须授权所在学校拥有学位论文的部分使用权，即：①学校档案馆和图书馆有权保留学位论文的纸质版和电子版，可以使用影印、缩印或扫描等复制手段保存和汇编学位论文；②为教学和科研目的，学校档案馆和图书馆可以将公开的学位论文作为资料在档案馆、图书馆等场所或在校园网上供校内师生阅读、浏览。另外，根据有关法规，同意中国国家图书馆保存研究生学位论文。

（保密的学位论文在解密后适用本授权书）。

作者签名：　　导师签名：

年　月　日　　年　月　日

中图分类号 TP311 学校代码 10290

UDC 004 密 级 公开

# 中国矿业大学

## 硕士学位论文

# 基于云边协同的工程机械故障诊断系统研究

# Research on Fault Diagnosis System for Construction Machinery Based on Cloud Edge Collaboration

作 者 伊斯罗贡 导 师 张博

申请学位 工学硕士学位 培养单位 计算机学院

学科专业 计算机科学与技术 研究方向 智能故障诊断

答辩委员会主席 牛强 评 阅 人 盲评

二〇二五年六月

# 致谢

首先，我要特别感谢我的导师——张博副教授。张老师严于律己、宽以待人的崇高风范深深感染和激励着我，也教会了我许多为人处世的道理。张老师治学严谨、学识渊博，在我整个研究过程中始终给予耐心指导，针对我的科研能力与研究现状，为我指明方向、合理设计课题，帮助我从整体上把握论文的结构与逻辑框架。从选题、开题到论文撰写的每一个环节，张老师都倾注了大量心血。由于自身能力有限，难免在学习过程中出现许多不足和问题，但张老师始终以宽容、耐心的态度包容我、鼓励我、引导我，不仅在学业上给予我严谨指导，更在做人方面给予我深刻启发。张老师的教诲与帮助将成为我人生道路上弥足珍贵的财富。对于张老师的感激之情难以言表，唯愿在今后的学习与工作中，始终以严谨的态度、勤奋的精神和不断的努力，来回报这份沉甸甸的师恩。

同时，我还要感谢国际学院和计算机学院的诸位老师在学习和生活中给予我的帮助与教导。感谢学院领导和老师们在我学习期间的关心、支持与培养，为我创造了良好的学术环境和成长平台。无论是在课程学习还是科研探索过程中，学院给予的鼓励与指导都让我受益匪浅。我将永远铭记这份理解、支持与尊重，并在今后的学习与工作中，以更加积极、踏实的态度回馈学校的栽培，不负众望。

感谢在学习和生活中给予我极大关怀与帮助的同学和朋友。你们的鼓励让我在困境中重拾信心，你们的建议为我带来思路上的启发，你们的帮助让我感受到温暖，你们的友谊更是我人生中宝贵的财富。我将这份真挚的情谊永记于心，倍加珍惜。

# 摘 要

随着智能制造的迅猛发展，故障诊断在保障工程机械运行安全与效率方面变得日益重要。本文提出了一种基于云边协同计算的新型故障诊断系统，针对边缘计算中存在的数据延迟、网络依赖性强以及计算能力有限等挑战，提供了有效解决方案。该系统融合了深度学习技术，采用包括设备层、边缘层和云端层的分层架构，实现了高效且可扩展的故障检测能力。本研究首先分析了工程机械的故障诊断需求，重点关注滚动轴承的故障识别。随后，系统性地探讨了故障检测与预测的理论基础，并评估了传统的诊断方法，包括基于知识、基于模型和数据驱动的诊断方法。在深度学习优势的加持下，本文构建了一个云边协同的故障诊断模型，并将其划分为云端和边缘端。边缘层主要执行实时信号采集、数据预处理及轻量化模型推理，而云端层负责复杂模型推理、训练及集中式管理。为优化边缘与云之间的数据流动，系统引入了基于预测置信度阈值的决策机制，以确保诊断结果的可靠性并降低不必要的带宽消耗。本系统通过嵌入式设备实现部署，并在多种故障场景下开展了大量实验验证。实验结果表明，该系统具有高准确率、低延迟和良好的适应性，适用于工程机械实际运行环境。本研究为智能预测性维护提供了一个可扩展、高效且协同的故障诊断框架，为对实时性和鲁棒性要求较高的工业应用场景提供了实用价值。

本论文有图 68 幅，表 19 个，参考文献 131 篇。

**关键词：**云边协同；工程机械；深度学习；智能系统；实时故障诊断

# Abstract

With the rapid advancement of intelligent manufacturing, fault diagnosis has become increasingly critical in ensuring the operational safety and efficiency of construction machinery. This thesis presents a novel fault diagnosis system based on cloud-edge collaborative computing, addressing challenges such as data latency, network dependency, and limited computational capacity at the edge. The system integrates deep learning techniques with a layered architecture consisting of equipment, edge, and cloud planes to enable efficient and scalable fault detection. The research begins by analyzing the fault diagnosis requirements of construction machinery, with a particular focus on rolling bearings. It then explores the theoretical foundations of fault detection and prognosis, and evaluates traditional diagnostic methodologies, including knowledge-based, model-based, and data-driven approaches. Leveraging the strengths of deep learning, the study develops a cloud-edge collaborative model that is partitioned for edge and cloud. The edge layer performs real-time signal acquisition, data preprocessing, and lightweight model inference, while the cloud layer handles complex model inference, model training, and centralized management. A decision-making mechanism based on prediction confidence thresholds is introduced to optimize the data flow between edge and cloud, ensuring reliability and reducing unnecessary bandwidth usage. The system is implemented using embedded devices and evaluated through extensive experiments under different fault scenarios. Results demonstrate high accuracy, low latency, and improved adaptability of the proposed system in real-world construction machinery environments. This study contributes to the advancement of intelligent predictive maintenance by providing a scalable, efficient, and collaborative framework for fault diagnosis, offering practical value for industrial applications where real-time performance and robustness are critical.

This thesis has 68 images, 19 tables, and 131 references.

**Keywords:** cloud-edge collaboration; construction machinery; deep learning; intelligent systems; real-time fault diagnosis

# 详细中文摘要

## 1 绪论

随着工业 4.0 与智能制造的快速发展，工程机械设备的自动化程度与运行复杂性不断提升，对其可靠性与故障响应能力提出了更高要求。传统的故障诊断方法如基于专家知识或机理模型的方法在面对非线性、高维度、动态变化的工业数据时存在明显不足。近年来，深度学习等数据驱动方法在故障诊断领域展现出强大的特征提取与识别能力，但其对计算资源和实时性的高要求也带来了挑战。边缘计算通过在设备侧进行初步处理，能够显著降低延迟，但计算能力有限。为此，融合云计算与边缘计算优势的云边协同架构成为解决现有问题的有效手段，具有重要的理论研究意义和工程应用价值。

当前，智能故障诊断技术主要涵盖信号采集、特征提取和故障识别三个关键环节。国内外在多传感器信号采集方面已有广泛研究，应用传感器类型涵盖振动、声发射、温度和电流等。特征提取方法方面，时域、频域与时频域分析方法各有优势，适用于不同的信号特性。故障识别技术从早期的知识驱动与模型驱动逐步过渡到以深度学习为代表的数据驱动方法，后者因其自动化程度高、适应性强，成为研究主流。同时，云计算与边缘计算在工业物联网中的融合应用日益广泛，为智能诊断系统提供了强有力的支撑。

本论文围绕工程机械故障诊断的现实需求，提出并构建了一种基于云边协同架构的智能诊断系统。研究内容涵盖典型故障场景分析、现有方法的对比评述，以及三级架构系统的设计与实现，包括设备层、边缘层和云端层。在算法设计方面，本文开发了适用于云边协同的深度学习模型，并针对数据传输效率提出了置信度阈值决策机制，优化模型部署与响应策略。此外，论文还完成了系统的构建与多工况下的实验验证，从诊断准确率、实时性、通信开销及系统鲁棒性等方面对系统性能进行了全面评估。

## 2 相关理论基础

第二章详细介绍了工程机械故障诊断领域的相关理论基础，涵盖了故障诊断的关键概念和数学表述，并系统回顾了传统与现代的故障诊断方法。首先，明确了故障诊断中常用的术语定义，如故障、故障模式、故障原因、故障机制、故障特征、故障检测、故障隔离、故障识别以及故障诊断等概念，并通过数学模型进行了理论表述。特别指出，区分正常变化与故障的必要性，强调了扰动与已建立运行模式显著变化之间的区别。其次，介绍了故障诊断系统的构成，包括信号采集、特征提取与分析、故障分类与诊断决策等子系统，并指出特征提取是整个故障诊断过程中的关键步骤。

在故障诊断方法部分，详细评述了三类主要的故障诊断方法：知识驱动方法、模型驱动方法和数据驱动方法。知识驱动方法依赖于专家经验，易于理解和实现，但受限于专家知识的局限性；模型驱动方法通过精确的物理模型描述系统，但对系统的建模要求较高，且难以适应复杂系统；数据驱动方法则通过数据挖掘和机器学习技术，不依赖精确的系统模型，具有较强的适应性和灵活性，因此成为当前故障诊断研究的主流趋势。

在深度学习部分，介绍了深度学习的基本原理和常见模型结构。包括卷积神经网络（CNN）、循环神经网络（RNN）、长短期记忆网络（LSTM）、深度置信网络（DBN）、图神经网络（GNN）、Transformer 模型、生成对抗网络（GAN）和自编码器（AE）。其中，CNN 主要擅长捕捉空间特征和局部模式，RNN 和 LSTM 适用于时序数据分析，能够有效捕捉长期依赖关系，DBN 适合无监督特征学习和概率建模，GNN 则能够捕捉传感器之间的复杂关系，Transformer 模型特别擅长处理长序列数据，GAN 在数据增强和稀有故障模式生成中具有重要应用，而 AE 则广泛应用于降维和特征提取。这些模型的优劣和适用范围在本节中得到了详细讨论，并强调了在实际应用中根据数据特征和任务需求来选择合适的模型。

在现代深度学习技术在故障诊断中的应用部分，进一步探讨了各类深度学习技术在实际故障诊断中的具体应用。包括 CNN 在机械设备故障特征自动提取中的广泛应用，LSTM 和 RNN 在时序故障数据的预测和异常检测中的优势，DBN 在复杂数据结构分析中的应用，GNN 在捕捉设备传感器网络结构关系中的潜力，Transformer 模型在长序列数据分析和故障预测中的突出表现，GAN 在数据增强和稀有故障模式扩展中的有效应用，以及 AE 在高维数据降维和鲁棒特征提取中的显著优势。这些技术不仅显著提高了故障诊断的精度，还增强了模型的鲁棒性。特别是技术融合（例如 CNN-GNN 融合模型）被认为是未来研究的重要趋势。

最后，总结了相关理论基础的重要性与实际意义，强调了故障诊断方法选择时需要综合考虑实际需求、数据资源、计算资源以及运行条件等因素。并指出，数据驱动方法，尤其是深度学习技术，在处理复杂故障诊断任务时展现出了巨大的潜力，为后续章节中的云边协同诊断系统研究奠定了理论基础

# 3 云边缘协同故障诊断框架

第三章主要提出并详细阐述了基于云边协同计算的工程机械故障诊断框架，重点介绍了云计算与边缘计算的基本概念、云边协同的概念，以及具体框架的结构和各部分功能。通过对云计算与边缘计算的特点与优缺点的比较，本章为理解云边协同在故障诊断中的应用提供了清晰的理论基础。具体内容如下：

首先，定义了云计算和边缘计算的基本概念，并比较了两者在实际应用中的异同。云计算是通过互联网提供计算服务的一种模式，能够提供大规模的虚拟资

源，具有较高的弹性和可用性，但由于其依赖网络，存在一定的延迟和带宽问题，难以应对实时性要求较高的场景。而边缘计算则通过将计算和存储资源下沉至设备端，解决了云计算在实时性、带宽需求和数据安全方面的不足，具有更强的实时处理能力和数据隐私保护能力，适用于实时性要求高的工业应用。重要的是，本节明确指出，云计算和边缘计算并非对立关系，而是互为补充，通过两者的协同工作，能够更好地满足工业场景中多样化的需求。

然后，介绍了云边协同的概念，阐述了云计算和边缘计算如何通过融合各自的优势，达到资源协同、数据协同与服务协同的目标。资源协同指边缘节点与云端共同协作，提供计算、存储与网络资源；数据协同则是边缘节点负责数据的采集、预处理和初步分析，将处理后的数据上传至云端进行进一步处理；服务协同方面，云端训练模型后将模型下发至边缘节点进行推理，并接受云端的生命周期管理。云边协同通过这种方式实现了资源与数据的高效利用，提高了整个系统的处理能力和效率。

另外，提出了云边协同故障诊断框架，该框架分为设备平面、边缘平面和云平面三个核心部分，每一层次的功能和协作方式如下：设备平面负责机械设备和传感器的振动数据采集，尤其是针对滚动轴承的故障诊断；边缘平面作为中间层，负责数据的实时采集与初步处理，包括数据预处理和初步故障推理；云平面则负责模型的训练和深度分析，并通过事件流处理、规则引擎等功能实现对数据的管理与模型的更新。边缘节点通过低功耗计算单元支持多种通信协议，进行初步的故障分析，从而减少了数据上传需求，优化了带宽消耗；云端则处理更复杂的数据分析和模型优化，显著提高了诊断精度和系统的鲁棒性。

最后，总结了本章的主要内容，指出基于云边协同计算的框架能够有效解决单纯依赖云计算系统所面临的实时性和带宽问题。通过设备平面、边缘平面和云平面的协同工作，框架实现了一个高效、实时且灵活的故障诊断系统。边缘节点在本地进行实时数据处理和初步故障分析，减少了不必要的数据传输，而云端的强大计算能力则使得复杂分析和模型优化成为可能。该框架不仅提升了故障诊断的准确性和鲁棒性，还为后续的系统部署和具体模型的实现提供了理论指导和架构基础，展示了云边协同在工业应用中的巨大潜力和价值。

# 4 云边协同故障诊断模型实现

本章聚焦于在云边协同计算环境下构建适用于工程机械故障诊断的深度学习模型，特别针对机械设备中最为关键的部件之一——滚动轴承的故障识别问题，设计并实现了一种结合云端与边缘端计算能力的宽卷积神经网络（WDCNN）模型。本章在理论与实践层面详细阐述了模型的设计思路、结构组成、不同部署策略下的架构差异、决策机制设定方法以及实验过程与结果分析。最终，本章通过

实验验证了模型在精度、实时性、资源占用和鲁棒性方面的综合优势，具备良好的工业应用前景。具体内容如下：

## 4.1 提出的故障诊断模型（Proposed Fault Diagnosis Model）

本研究所提出的深度神经网络模型基于宽卷积神经网络（Wide Deep Convolutional Neural Network, WDCNN）构建，重点解决传统卷积神经网络在噪声环境中特征提取能力不足的问题。通过采用较大的卷积核宽度，增强模型对宽频带信号中关键故障特征的捕捉能力，从而提升模型在复杂工况下的鲁棒性。

该模型主要由两大模块组成：

（1）滤波层（Filter Layer）：包括多个卷积层、批量归一化（Batch Normalization）、ReLU 激活函数以及最大池化（MaxPooling）操作。此部分负责从原始时序振动数据中提取低层次至高层次的特征，并通过归一化与非线性映射提升模型的泛化能力与收敛速度。

（2）分类器（Classifier）：由全连接层（Fully Connected Layer）与 Softmax 输出层构成，用于实现多分类故障识别任务。分类器根据特征图输出对应的概率分布，从而判断设备当前可能存在的具体故障类型。

## 4.2 单传感器模型架构（Single Sensor Model Architecture）

针对工业现场中常见的单一传感器数据来源场景，构建了云边协同下的单传感器模型架构。该模型设计充分考虑了边缘计算资源受限的现实情况，采用分层部署的方式，在保证诊断准确率的同时兼顾计算效率。

（1）边缘端部署：在边缘设备上部署模型的浅层网络结构，减少卷积层数量，显著降低模型推理的计算负载。边缘端能够快速完成对大部分故障信号的初步诊断，满足实时处理的需求。

（2）云端部署：保留完整模型结构，处理边缘端无法高置信度判断的复杂样本。云端利用其强大的计算资源对剩余数据进行深度分析，提升整体系统的诊断准确率。

同时，模型采用了 BranchyNet 结构引入多出口机制，允许网络在浅层、中层或深层进行早期输出，从而在不同计算环境下动态调整处理流程，提高系统的灵活性与响应速度。

## 4.3 多传感器模型架构（Multi-sensor Model Architecture）

为进一步提升故障诊断的精度与信息完整性，提出了支持多模态数据输入的多传感器模型架构。相比单传感器模型，多传感器模型可融合多个来源（如水平、垂直方向的振动数据）的信息，实现对设备状态的全面感知。

模型采用横向扩展结构以接纳多传感器输入，并引入以下四种特征聚合策略进行特征整合：

（1）最大池化（Max Pooling）：强调故障特征的最强响应，具有良好的抗噪声能力。

最小池化（Min Pooling）：保留最小响应，适用于异常检测类任务。

（3）平均池化（Average Pooling）：提取特征的平均信息，稳定性强但敏感度略低。

（4）特征连接（Concatenation）：将各传感器特征直接拼接保留全部信息，适用于高精度诊断需求。

实验结果表明，最大池化法在计算效率与精度之间取得了较好平衡，而特征连接法虽性能最优，但资源消耗大，适合云端处理场景。

## 4.4 云边决策阈值（Cloud-edge Decision Making Threshold Values）

本研究提出了基于预测置信度的动态决策阈值机制，用于智能判定数据应在边缘端处理或传输至云端进一步分析。

（1）对于单传感器模型，边缘端处理 85%以上的数据样本已能实现高置信度的准确预测，仅有 15%左右的数据需上传云端，显著减轻了网络负担。

（2）对于多传感器模型，不同聚合策略下的阈值略有差异。其中，最大池化方法使得边缘端处理数据比例达到 93.63%，是最具实时性与效率的方案。

该机制确保了资源利用最优，同时通过低置信度样本的智能分流，提高了系统整体诊断精度。

## 4.5 实验设计与分析（Experiments）

为验证所提出模型的有效性，选用凯斯西储大学（Case Western Reserve University, CWRU）滚动轴承故障数据集作为实验数据来源。实验涵盖多个故障类别（如内圈故障、外圈故障、滚动体故障等）与不同工况，确保模型适应性与通用性。

实验结果表明：

（1）所有模型在不同故障类别上均取得了较高的分类准确率，单传感器模型准确率可达 99%以上，多传感器模型在特征连接方式下甚至达到了接近 100%的精度。

（2）多传感器架构在故障特征明显但信号质量不佳的情况下，显著优于单传感器模型，具备更强的鲁棒性。

（3）各种特征融合策略中，最大池化法在保持高准确率的同时大幅降低计算成本，最适合部署在边缘设备。

# 5 云边协同系统实现

本章在第三章提出的云边协同故障诊断框架和第四章构建的深度学习模型基础上，实现了一个完整的云边协同工程机械故障诊断系统。通过原型系统的构建与实验验证，证明了该系统在准确性、实时性和可扩展性方面的优势，具备实际工业部署的潜力。

## 5.1 系统设计（System Design）

系统总体架构分为设备平面、边缘平面和云平面三层，实现从数据采集到深度诊断与模型更新的全流程闭环。数据流自设备平面向上传递，诊断结果及模型参数通过 OTA（Over-The-Air）机制自云端向下分发，实现高效的协同计算与动态模型更新。

诊断流程包括边缘端的实时初步诊断、云端的深度分析，以及基于预测置信度的动态决策机制，以此优化系统带宽使用并提升诊断实时性与准确性。

## 5.2 设备平面实现（Equipment-plane Implementation）

设备平面主要承担机械设备状态感知与原始数据采集功能，具体包括：

（1）传感器选型与部署：采用高性能振动传感器（如 CYT9200）监测轴承状态，传感器安装位置经过结构动力学分析优化，以最大程度捕捉故障特征。

（2）采集设备配置：选用 WebDAQ504 振动数据采集设备，结合智能车载终端（TBOX），支持高频数据实时传输与初步预处理。

通过科学的选型与部署方案，确保原始数据的时效性、准确性与可用性，为后续诊断奠定基础。

## 5.3 边缘平面实现（Edge-plane Implementation）

边缘平面负责接收传感器数据，进行实时预处理与轻量级故障识别，其软硬件实现如下：

硬件架构：

（1）WebDAQ504：实现高采样率振动信号采集。

（2）TBOX 智能终端：内置边缘计算模块，支持多种工业通信协议（MQTT、BLE、OPC-UA、Modbus、CAN 等），实现设备联网与数据处理。

软件架构（智能边缘网关）：

（3）Edge Manager：协调各边缘模块的运行。

（4）Data Collector：支持多通道、多协议的数据采集。

（5）Data Preprocessor：执行归一化、滤波、异常值剔除等预处理操作。

（6）Model Inferencer：部署轻量化 CNN 模型，完成本地实时故障判别。

（7）Decision Maker：根据模型输出置信度判断是否需上传至云端。

（8）OTA Updater：接收云端下发模型并实现自动部署。

（9）Temp Storage & Data Distributor：实现数据的缓存与按需上传。

边缘层具备快速响应能力，可在 95%以上的场景中完成实时故障分类，有效减少不必要的数据上传，降低带宽负载。

## 5.4 云平面实现（Cloud-plane Implementation）

云平面作为系统核心，承担大规模计算、复杂分析及模型训练任务：

（1）基础架构：基于虚拟化服务器集群部署，结合容器调度平台（如 Kubernetes）实现资源高效管理。

（2）数据处理平台：集成 Hadoop、Spark、PostgreSQL、Cassandra、Redis 等组件，支持流式与批量数据处理。

（3）微服务架构：

Transmission Microservice Set：处理 MQTT 和 HTTP 数据传输。

Core Microservice Set：提供 API 接口、规则引擎与设备状态监控服务。

（4）模型训练与 OTA 管理：在云端完成深度神经网络模型的训练、评估与部署，通过 OTA 推送至边缘端，形成闭环优化。

该层结构具备强大的可扩展性与数据处理能力，为系统提供全局最优诊断策略与模型更新保障。

## 5.5 原型实验（Prototype Experiments）

构建系统原型后，采用 CWRU 轴承故障数据进行实验验证，测试包括：

（1）单/多传感器诊断能力验证

（2）边缘实时响应效率评估

（3）云端处理与更新流程完整性测试

实验结果表明：

（1）系统可准确识别多种轴承故障状态，诊断精度高。

（2）边缘端在大多数场景中即可完成故障识别，降低了对云端的依赖。

（3）云端处理复杂案例能力强，同时实现动态模型更新，系统整体性能稳定，反应迅速。

# 6 结论

本研究提出了一种基于云边协同架构的工程机械故障诊断系统，有效融合了边缘计算的实时性与云计算的强大算力，实现了高效、可靠的故障检测与诊断流程。系统在边缘层完成实时数据处理与初步诊断，有效降低了通信延迟和带宽压

力；而在云端，则集中处理复杂的模型训练、大规模数据分析与管理任务，从而在保持诊断准确性的同时，提高了系统的可扩展性与鲁棒性。

实验结果表明，该云边协同策略可有效应对工业环境中常见的网络延迟与带宽受限问题，在保障实时性的前提下，实现了高性能的故障诊断能力。系统通过模型的分层部署与 OTA（Over-the-Air）机制，支持模型的远程更新和快速迭代，进一步增强了系统的长期适应能力与维护效率。

此外，研究中引入了深度学习模型（尤其是 CNN），并结合多传感器数据融合策略，显著提升了故障分类的准确性，展示了深度学习方法在复杂工业场景下的应用潜力与实际价值。

本研究的原型系统已在实验环境中展现出良好的性能，但在未来的研究与实际应用中，仍可在以下几个方面进行深入拓展：

边缘设备生态拓展：未来可进一步拓展边缘计算设备的种类与数量，增强系统对不同工业环境的适配能力，实现更广泛的工程机械类型支持。

系统实时性的持续优化：在更动态或更复杂的操作场景中，可结合轻量模型、模型剪枝和边缘加速技术，进一步优化系统响应速度与实时性。

自适应诊断能力：未来可探索引入元学习、迁移学习等方法，使系统具备处理未知故障或少样本故障的能力，提升其智能性与通用性。

跨行业拓展应用：本系统架构与诊断方法具有高度通用性，未来可推广应用至如风电设备、轨道交通、石化装置等其他工业领域，为智能维护提供技术支撑。

# 目　录

# Contents

# 图清单

# 表清单

# 变量注释表

| | |
|---|---|
| $A_{ij}$ | 邻接矩阵关系 |
| $D$ | 度矩阵 |
| $L$ | 归一化后的拉普拉斯矩阵 |
| $z$ | 隐藏变量 |
| $d(A,B)$ | $A,B$两个点之间的欧式距离 |
| $D_{KL}$ | KL 散度 |
| $X$ | 指标特征矩阵 |
| $X_0$ | 参考序列 |
| $y_i(j)$ | 标准化后指标数值 |
| Y | 标准化后矩阵 |
| $\Delta$ | 绝对差值矩阵 |
| $\xi_i(j)$ | 第$i$个样本的第$j$个指标值的灰色关联系数 |
| $\rho$ | 灰色关联度系数计算中的分辨系数 |
| $r_i$ | 第$i$个样本的灰色关联度 |
| $\omega_j$ | 为第$j$个指标值的权重 |
| $\overline{x_j}$ | 第$j$个指标的均值 |
| $S_j$ | 第$j$个指标的标准差 |
| $\rho_{p,q}$ | 斯皮尔曼相关系数 |
| $C_j$ | 信息量 |
| $e_j$ | 第$j$个指标的熵值 |
| $S(e_j)$ | 激活后的信息熵 |
| W(X) | 动态权重矩阵 |
| $W^0$ | 初始的固定权重 |
| $S_j$(X) | 状态变权向量 |
| $A$ | 调权水平 |

# 1 绪论
# 1 Introduction

## 1.1 研究背景及意义（Research Background and Significance）

With the increasing complexity and automation of construction machinery, ensuring equipment reliability and operational safety has become more critical than ever. Faults in key components such as bearings, motors, and gearboxes not only compromise the performance of machinery but may also result in costly downtime, safety hazards, and economic losses. As a result, intelligent fault diagnosis has emerged as an essential aspect of modern equipment maintenance and management. Traditional fault diagnosis methods, including knowledge-based and model-based techniques, often face limitations in dealing with the nonlinear, high-dimensional, and time-varying characteristics of mechanical systems.

In recent years, data-driven approaches, particularly those based on deep learning, have demonstrated strong capabilities in feature extraction, classification, and pattern recognition, enabling more accurate and automated fault diagnosis. At the same time, the advent of the Industrial Internet of Things (IIoT) has led to a surge in sensor deployment across machinery, generating massive volumes of operational data. Cloud computing offers powerful resources for storage and analysis of this data, but its reliance on stable network connectivity and centralized processing introduces latency and bandwidth constraints，especially in scenarios requiring real-time decision-making. Edge computing addresses these limitations by bringing computation closer to the data source, allowing for faster processing and reduced data transmission. However, edge devices often suffer from limited computational power and storage capacity. Therefore, a cloud-edge collaborative approach that combines the strengths of both paradigms presents a promising solution for intelligent fault diagnosis in construction machinery. By integrating advanced sensing, intelligent diagnostics, and distributed computing, this study contributes to the development of intelligent maintenance systems aligned with the goals of Industry 4.0 and smart manufacturing.

## 1.2 中国内外的研究现状（The Current Status of Domestic and International Research）

As shown in Figure 1-1, the current basic approach to intelligent fault diagnosis of machinery involves analyzing the collected signals in the time domain, frequency domain, or time-frequency domain. This analysis is used to extract fault characteristic

information contained in the signals. Subsequently, the extracted fault features are processed using linear or nonlinear classifiers to achieve the identification of mechanical faults.

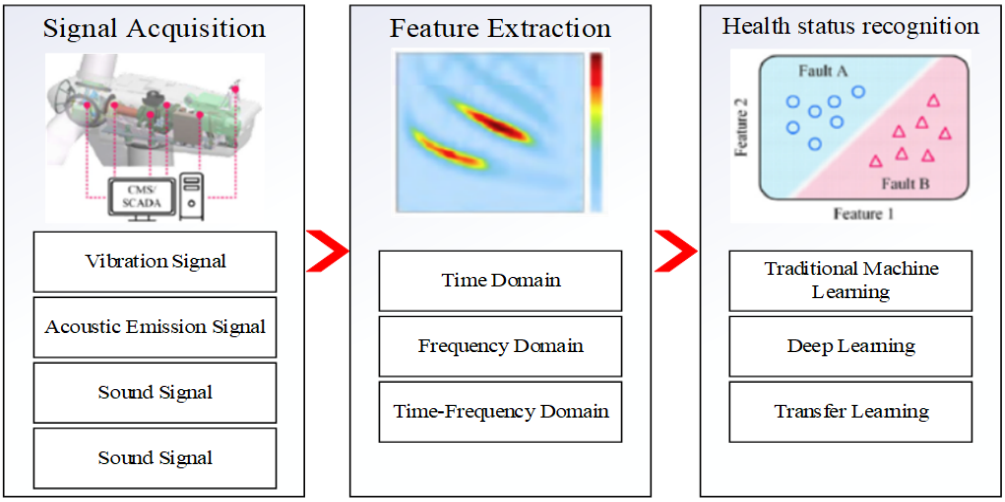



图 1-1 机械智能故障诊断流程图

Figure 1-1 Mechanical Intelligent Fault Diagnosis Diagram

### 1.2.1 Signal Acquisition

Acquiring signals that represent the operational state of mechanical equipment using advanced sensing technologies is a prerequisite for mechanical fault diagnosis. Fault information in mechanical equipment often manifests across multiple physical fields, such as dynamics, acoustics, tribology, and thermodynamics. Therefore, researchers at home and abroad use different sensors to capture signals from these various physical fields. Examples include:

Vibration Signals: Vibration signals are widely used for fault diagnosis in components such as bearings[1,2] and gearboxes[3–5]. They are effective in detecting structural abnormalities and wear in rotating machinery.

Acoustic Emission Signals: These signals are applied to detect early-stage faults and deformations in bearings[6] and gears[7], particularly under low-speed operating conditions and in low-frequency noise environments.

Temperature Signals: Changes in temperature provide valuable information about inefficiencies, overheating, or wear in mechanical components.

Current Signals: Current signals play a crucial role in diagnosing faults in electromechanical systems[8,9]. These signals can be easily collected using current transformers, without the need for invasive installation within the machinery.

The integration of diverse sensors allows for the comprehensive acquisition of data

reflecting the health and operational state of mechanical equipment, forming the foundation for accurate and effective fault diagnosis.

### 1.2.2 Feature Extraction

Feature extraction based on signal processing technology is the primary method for representing mechanical fault information[10,11]. When mechanical equipment experiences a fault, its characteristics typically manifest to varying degrees in the time domain, frequency domain, and time-frequency domain.

Time Domain Analysis: Time domain methods focus on statistical features such as mean, root mean square (RMS), peak value, waveform factor, pulse factor, kurtosis, margin index, and crest factor. These metrics are particularly effective for analyzing stationary signals.

Frequency Domain Analysis: Frequency domain techniques, including spectrum analysis, power spectrum analysis, higher-order spectrum analysis, refined spectrum analysis, cepstrum analysis, envelope spectrum analysis, and holographic spectrum analysis, are also well-suited for stationary signals.

Time-Frequency Domain Analysis: Time-frequency domain methods are used for analyzing non-stationary signals. These techniques represent the changes in signal energy or power across both time and frequency in a two-dimensional space. Typical methods include:

Short-Time Fourier Transform (STFT)[12]: Effective for time-localized spectral analysis.

Wavelet Transform (WT)[13]: Suitable for multi-resolution analysis.

Empirical Mode Decomposition (EMD)[14]: Useful for decomposing complex signals into intrinsic mode functions.

Hilbert-Huang Transform: Provides enhanced capabilities for non-linear and non-stationary signal analysis.

These methods form the basis for extracting fault features that serve as inputs for advanced diagnostic algorithms, enabling precise and reliable fault detection and characterization.

## 1.3 主要研究内容（Main Research Contents）

This study focuses on the design and implementation of a fault diagnosis system for construction machinery based on cloud-edge collaborative computing. The main research contents of this thesis are summarized as follows:

Analysis of Fault Diagnosis Requirements in Construction Machinery. The study begins with an analysis of typical faults occurring in key components of construction machinery, especially rolling bearings. It explores the significance of timely fault detection in enhancing equipment reliability, minimizing downtime, and improving maintenance efficiency.

Review of Existing Fault Diagnosis Methods and Technologies. A comprehensive literature review is conducted on traditional and modern fault diagnosis techniques, including knowledge-based, model-based, and data-driven approaches. Particular attention is given to the emergence of deep learning (DL) techniques in intelligent fault detection and prediction (FDP).

Proposal of a Cloud-Edge Collaborative Fault Diagnosis Framework. A layered framework is designed, integrating sensing equipment, edge computing units, and cloud infrastructure. The framework outlines how data is collected, preprocessed, diagnosed at the edge, and further analyzed or stored in the cloud, addressing latency, bandwidth, and model update challenges.

Development and Deployment of a Deep Learning-Based Diagnosis Model. A deep neural network (DNN) architecture is developed and trained using publicly available datasets such as CWRU. The model is then partitioned into edge and cloud segments to enable real-time inference on edge devices while leveraging cloud resources for training and fine-tuning.

Implementation of Decision-Making Mechanisms at the Edge and Cloud Levels. This study introduces confidence-based decision-making algorithms that operate based on predefined thresholds, determining which data should be transmitted to the cloud. These algorithms aim to optimize bandwidth efficiency by ensuring that only the most relevant data is transmitted. In cases where predictions exhibit a high degree of uncertainty, the algorithms escalate these instances for more thorough analysis, thus enhancing the overall diagnostic capability of the system.

Design and Realization of a Prototype System. A prototype system is built to validate the proposed framework. It includes the integration of sensors, embedded edge computing units (e.g., Raspberry Pi), cloud services, a rule engine, and an OTA model repository. Experiments are conducted to evaluate system performance under different fault scenarios.

Experimental Evaluation and Performance Analysis. Extensive experiments are performed to validate the effectiveness of the proposed system. Performance metrics

include fault diagnosis accuracy, response time, model update efficiency, and system robustness in real-world conditions.

## 1.4 章节安排（Chapter Arrangement）

In this study, we address three primary challenges in the field of fault diagnosis for construction machinery. First, we propose a cloud-edge collaborative framework aimed at optimizing the utilization of both cloud and edge resources, ensuring efficient fault diagnosis. By distributing computational tasks between the cloud and edge, the system enhances scalability while minimizing the reliance on centralized cloud resources. Second, we develop a robust deep learning model capable of performing seamless inference in a cloud-edge collaborative environment. This model improves real-time fault diagnosis performance by reducing the latency associated with edge-cloud data transfer, allowing for faster and more accurate fault detection in dynamic operational settings. Finally, to further enhance efficiency, we introduce a threshold-based decision-making mechanism designed to minimize unnecessary edge-cloud data transfers. Based on the description above, this paper consists of six main chapters, and the research framework is shown in Figure 1-2.

To provide a comprehensive overview of the proposed cloud-edge collaborative fault diagnosis system for construction machinery, this thesis is organized as follows:

**Chapter 1: Introduction**

This chapter introduces the background, significance, and current research status of mechanical fault diagnosis, particularly in construction machinery. It also outlines the main research contents and the overall structure of the thesis.

**Chapter 2: Relevant Theoretical Foundations**

This chapter presents the theoretical basis of fault diagnosis, including formal definitions of fault, fault diagnosis, and prognosis. It also reviews conventional and modern diagnostic methods, especially data-driven and deep learning-based approaches.

**Chapter 3: Cloud-Edge Collaborative Fault Diagnosis Framework**

This chapter proposes a cloud-edge collaborative architecture tailored for construction machinery. It introduces the key components across the equipment, edge, and cloud planes, and explains how these components interact to perform real-time fault diagnosis.


图 1-2 本文研究框架图

Figure 1-2 This thesis studies the frame diagram

**Chapter 4: Cloud-Edge Collaborated Fault Diagnosis Model Implementation**

This chapter details the design, training, and partitioning of the deep learning-based fault diagnosis model. It includes both single-sensor and multi-sensor architectures, and explains how decision-making thresholds are applied to manage cloud-edge collaboration.

**Chapter 5: Cloud-Edge Collaborated System Implementation**

This chapter focuses on the practical deployment of the system. It describes the implementation of software and hardware components across the equipment, edge, and cloud layers, and presents a prototype system with experimental validations.

**Chapter 6: Conclusion and Future Perspectives**

The final chapter summarizes the key findings and innovations of the research. It reflects on the limitations of the current work and suggests directions for future research in intelligent fault diagnosis under cloud-edge collaborative frameworks.

# 2 相关理论基础
# 2 Relevant Theoretical Foundations

Fault diagnosis is a multidisciplinary domain, integrating concepts from applied mathematics, control theory, information theory, and reliability theory. To facilitate a deeper understanding of fault diagnosis systems in construction machinery, this section presents the mathematical formulations of faults and the methodologies employed in fault diagnosis systems. As this study adopts a data-driven fault diagnosis approach leveraging deep learning techniques, a comprehensive review of deep learning methodologies is conducted to systematically analyze their applicability, effectiveness, and advancements in fault diagnosis for construction machinery.

## 2.1 问题的理论表述（Theoretical Formulation of the Problem）

This section presents the generalized definitions of faults and the mathematical formulations associated with fault diagnosis and prognosis (FDP) problems. The definitions and distinctions of essential FDP summarized in Table 2-1.

表 2-1 故障诊断中使用术语的定义

Table 2-1 Definition of the terminology used in fault diagnosis

| Term | Definition |
|---|---|
| Fault | A malfunction that prevents the system from executing its intended function. |
| Fault mode | The observable characteristics of a fault, commonly referred to as the fault type. |
| Fault cause | The primary factors contributing to the occurrence of a fault. |
| Fault mechanism | The progression of physical changes that ultimately lead to fault formation. |
| Fault feature | A parameter or feature that signifies the presence of an abnormal condition caused by a fault. |
| Fault detection | The procedure for identifying whether a fault has occurred. |
| Fault isolation | The process of determining the nature and/or location of a fault. |
| Fault identification or estimation | The process of determining the magnitude/intensity of the fault. |
| Fault diagnosis | The comprehensive process encompassing fault detection, isolation, and estimation. |

### 2.1.1 Formal Definitions of Fault

In construction machinery condition monitoring, the observed variations in sensor data are continuous and dynamic; however, not all deviations indicate failures or faults. To accurately distinguish faults from normal operational variations, the following key considerations must be taken into account:

Random noise-induced variations are not inherently indicative of faults. However, a significant change in the variance of noise is generally considered a potential fault.

Fluctuations within a stable range under a given operational condition do not constitute a malfunction. The acceptable fluctuation range may vary depending on different operating conditions.

Deviations that disrupt the established operational pattern are indicative of a fault and require further diagnostic evaluation.

Figure 2.1 presents a comparative analysis between the normal three-phase current waveform and the waveform under an interturn short-circuit fault within the same operating conditions. Notably, while the current amplitude remains within the expected operating range, a distinct pattern deviation occurs in the blue waveform for Time > 120 (ms), indicating the presence of a fault.

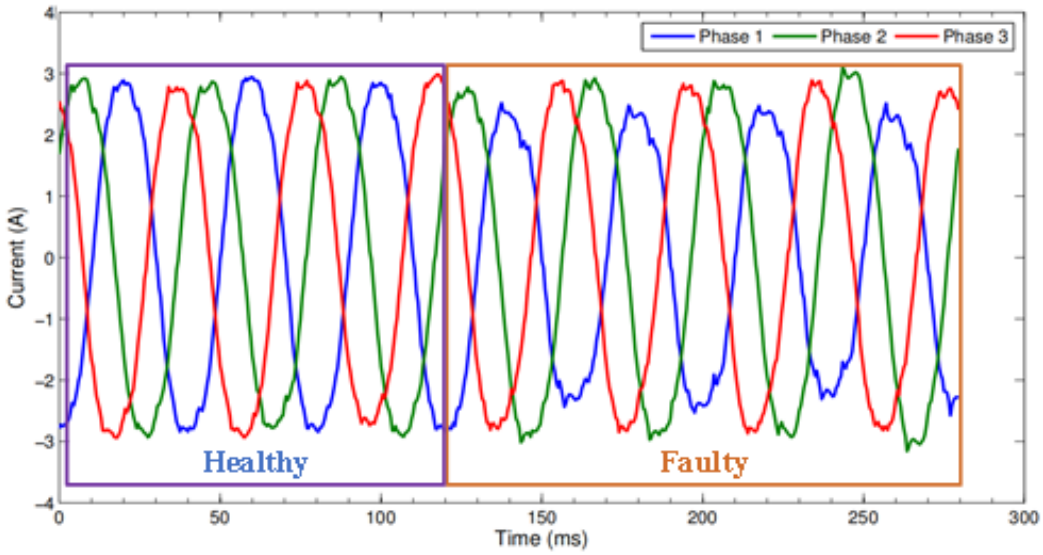



图 2-1 三相电流波形故障示例

Figure 2-1 Illustrative Example of a Fault in the Three-Phase Current Waveform

### 2.1.2 Mathematical Representation of Fault Diagnosis

Consider a system with $N$ physical variables (e.g., vibration, pressure, current, temperature) measured over a specific time interval $T = [t_1, t_2]$ by a set of sensors

positioned at a specific location on a given device. The set of monitored variables can be defined as:

$$M(T) = \{ m_i(T) \mid i = 1,2, \dots, N \} \quad (2-1)$$

where $m_i(T)$ represents the measurement of the $i$-th physical variable over time $T$. Given a current operating condition $p$, the fault indicator function $f_\theta(M(T), p)$ is formulated to assess whether the system state $s$ is healthy or faulty. This function is expressed as:

$$s = f_\theta(M(T), p) \quad (2-2)$$

where $f_\theta$ is the fault diagnosis function with parameter $\theta$. The output range of $f_\theta$ is $\{0,1\}$, where $0$ represents a healthy state, and $1$ indicates a faulty state. When the monitoring variable $M(T)$ and operating condition $p$ are known, the corresponding fault state can be theoretically determined. For a given type of device, its fault indicator function $f$ remains uniquely defined, ensuring a deterministic fault assessment under specific conditions. In this way the fault diagnosis is transformed into the process of determining the parameter $\theta$ of the fault indicator function $f$. While forward modeling using physical models can explicitly solve for $\theta$, these models are often highly complex or even unsolvable in practical applications.

To overcome this challenge, data-driven fault diagnosis methods leverage existing datasets to extract the optimal parameter $\theta$ by learning from data[15]. This transforms the problem into an inverse optimization task, where the goal is to determine a parameter $\theta'$ within the parameter space $\Theta$ such that the output pattern over a large dataset closely matches the actual system behavior. Mathematically, this is formulated as:

$$\arg\min_{\theta' \in \Theta} ||s' - f_{\theta'}(M'(T), p')|| \quad (2-3)$$

where: $s$' represents the actual system state labels in the known sample set, ($M$'($T$), $p$') denotes the monitored data vectors and operating conditions from the dataset, The objective function minimizes the deviation between the model output and the actual system state, transforming fault diagnosis into an optimization problem. Once the system status is classified as faulty, the fault classification process begins. The observed fault pattern is compared against reference fault patterns stored in a fault database, and the smallest deviation is used to identify the most probable fault type.

It is important to note that the original sensor data $M(T)$ used for diagnosis is typically high-dimensional and contains redundant features. Therefore, feature selection, feature extraction, or feature fusion techniques are commonly employed to

reduce data dimensionality, improving diagnostic accuracy and computational efficiency.

### 2.1.3 Mathematical Representation of Fault Prognosis

A significant challenge in fault prognosis is the accurate estimation of the Remaining Useful Life (RUL) of construction machinery, as illustrated in Figure 2-2. This process requires the selection of an appropriate health indicator that effectively reflects the degree of degradation in the machinery's health. Additionally, a threshold value must be established to indicate when the machinery will reach functional failure.

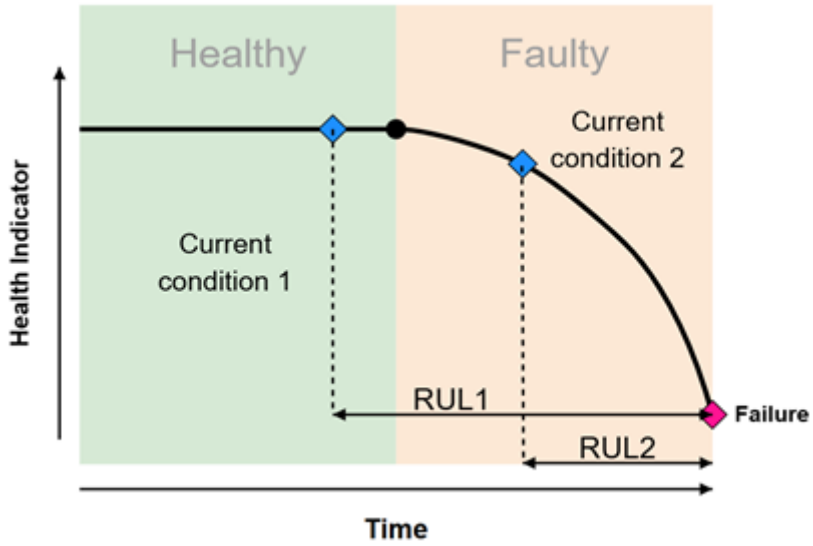



图 2-2 建筑机械生命周期图

Figure 2-2 Diagram construction machinery life cycle

Given a set of $k$ historical data points and their corresponding health feature sequences can be defined as:

$$\{ f_i(n) \mid i = 1,2, \dots, k, n = 1,2, \dots, N \} \qquad (2-4)$$

where N represents the length of the health feature sequence, a dataset $\{T_i(l), f_i(l)\}$ can be constructed. This dataset pairs historical time data with the corresponding health indices. Using the established construction machinery degradation model g, regression fitting can be applied to the dataset $\{T_i(l), f_i(l)\}$ to determine the model parameters of g. Given the current health indicator sequence $f(n)$, the degradation model g is then used to extrapolate and predict the future trajectory of the health features, denoted as $f^{\wedge}$.

The predicted evolution curve $f^{\wedge}$ is compared against the failure threshold. The machinery is predicted to fail when $f^{\wedge}$ exceeds the threshold for the first time at time $T_f$. Assuming that $TN$ represents the time length of the known observation data, the RUL is calculated as:

$$RUL = T_f - T_N \qquad (2-5)$$

The critical aspect of fault prognosis lies in the selection of the degradation model. Factors influencing this choice include the global degradation behavior, short-term degradation characteristics, the amount of available data, and the level of data noise, all of which contribute to the model's accuracy and reliability in predicting the machinery's remaining useful life.

## 2.2 故障诊断系统（Fault Diagnosis System）

A fault diagnosis system is a systematic approach used to detect, isolate, and identify faults in a system, ensuring reliability and performance by analyzing deviations from normal operation. A general fault diagnosis system is illustrated in Figure 2-3.

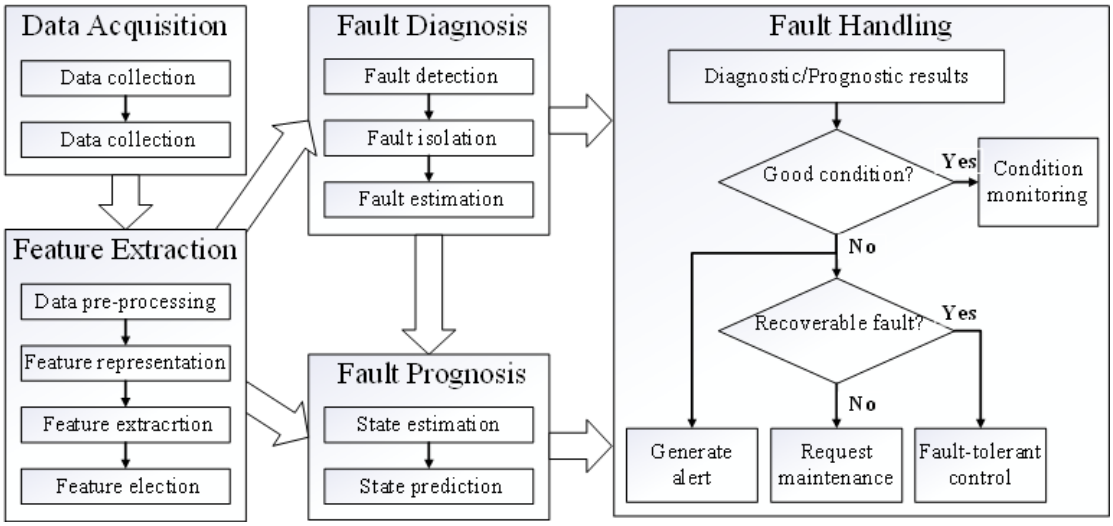



图 2-3 一般故障诊断系统。改编自[16,17]

Figure 2-3 General fault diagnosis system. Adapted from[16,17]

The process begins with data acquisition, which involves collecting and storing data from experimental measurements or high-fidelity simulation models, specifically from construction machinery in this study. This data serves as the foundation for model identification, fault characterization, and algorithm verification. Next, feature extraction is conducted through data preprocessing, feature representation, feature extraction, and feature selection. These extracted features form the basis for subsequent fault diagnosis. The fault diagnosis process consists of three key tasks: fault detection, fault isolation, and fault estimation, each serving a distinct role in identifying and characterizing faults. Finally, the fault handling module analyzes the diagnostic results and determines appropriate responses, such as triggering alarms, implementing fault-tolerant control strategies, isolating faulty components, or even disconnecting power to prevent further damage.

## 2.3 故障诊断方法（Fault Diagnosis Methods）

There is a substantial body of research on diagnostic methodologies. As illustrated in Figure 2-4, these methods can be categorized into three primary approaches: knowledge-based, model-based, and data-driven. This classification is based on the findings presented in Refs[18–22].

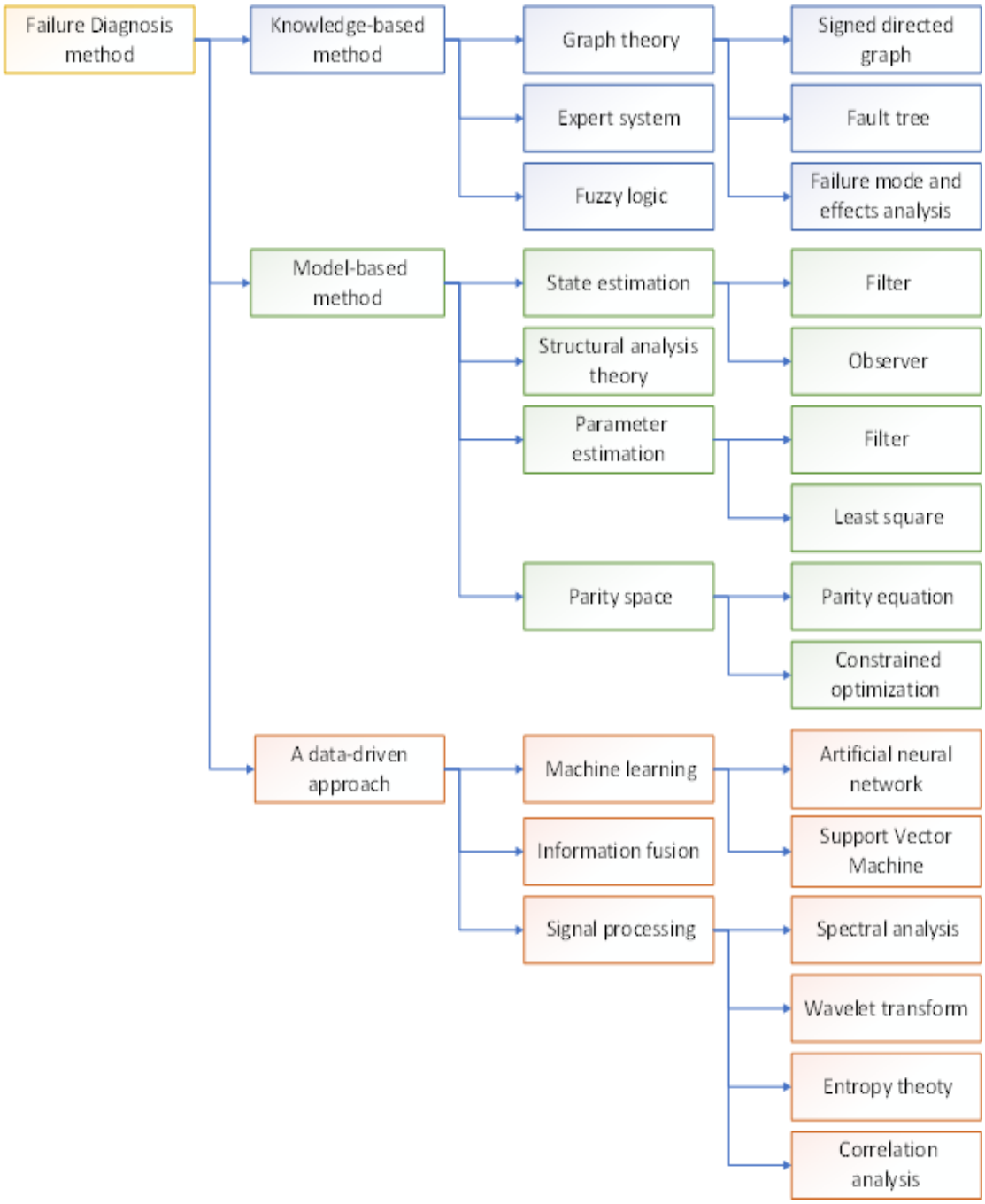



图 2-4 故障诊断方法分类

Figure 2-4 Classification of fault diagnostic methods

### 2.3.1 Knowledge-Based Methods

These diagnostic methodologies leverage domain knowledge and observational data, making them particularly suitable for complex and nonlinear systems without requiring the development of explicit mathematical models. One of their primary advantages lies in their interpretability, as both their operational principles and diagnostic outcomes are relatively straightforward to comprehend. Among the most widely adopted knowledge-based diagnostic approaches are graph theory-based methods, expert systems, and fuzzy logic-based techniques. Specifically, graph theory-based methodologies, including signed directed graphs (SDGs)[23], fault trees[24], and failure mode and effects analysis (FMEA)[25], enable the construction of a fault diagnosis network by mapping the fault propagation relationships among system components. By employing appropriate search algorithms, faults can be systematically identified and localized within the system. Expert systems are specialized computer programs designed to replicate the reasoning and decision-making processes of human experts[26]. These systems utilize a structured knowledge base, derived from historical datasets and expert insights, to establish diagnostic rules and decision-making frameworks. Fuzzy logic-based methods provide an effective means of handling qualitative knowledge and uncertainties in fault diagnosis. As fuzzy logic aligns with the cognitive processes of human reasoning, it facilitates the processing of imprecise or uncertain information. In fault diagnosis, fuzzy logic can be implemented through fuzzy parameters, fuzzy models, or fuzzy thresholds, allowing for robust detection and classification of faults in intricate systems.

Recent advancements have seen hybrid knowledge-based systems emerge, combining fuzzy logic with neural networks or incorporating rule-based reasoning into deep learning frameworks. These hybrid systems seek to balance interpretability with adaptability, potentially overcoming the rigidity of traditional expert systems and the data-dependence of purely data-driven methods. Despite their advantages, knowledge-based methods also have limitations. Their performance is heavily reliant on the completeness and correctness of the embedded knowledge base. In rapidly evolving systems or where expert knowledge is scarce, constructing and maintaining such a knowledge base can be resource-intensive and error-prone. Furthermore, unlike statistical learning models, they may not generalize well to new fault conditions outside their predefined scope. Table 2-2 presents a comparative analysis of various knowledge-based diagnostic methodologies, evaluating their key technologies, advantages, and

limitations.

表 2-2 基于知识的诊断方法比较

Table 2-2 Comparison of knowledge-based diagnostic methods

| Knowledge-based methods | Key Technologies | Advantages | Disadvantages |
|---|---|---|---|
| Graph theory | Diagnostic network; Fault propagation relationship; Search strategy. | Clear causality; Easy to interpret the diagnosis results; Easy to analyze qualitatively. | Needs a thorough understanding of fault mechanism; Not suitable for systems with high complexity. |
| Expert system | Knowledge acquisition and representation; Knowledgebase; Rule base. | No need for the mathematical model; Diagnostic results are easy to understand. | Difficulty in knowledge acquisition and representation; Over-reliance on the representativeness and integrity of knowledge. |
| Fuzzy logic | Fuzzy rules; Membership function. | Suitable for handling qualitative knowledge and reasoning. | No self-learning ability; Difficult to develop effective rules. |

In the context of Construction Machinery, multiple fault types may arise, including mechanical faults, sensor faults, and actuator faults. Graph theory-based approaches offer a well-defined causal structure, making diagnostic results highly interpretable. However, the complexity of fault mechanisms in Construction Machinery poses significant challenges in constructing an accurate diagnostic network. Expert systems provide an alternative that does not rely on physics-based models. Nevertheless, their application in Construction Machinery is hindered by challenges such as knowledge acquisition difficulties and inaccurate knowledge representation. The fault states of Construction Machinery can be identified through various anomalous behaviors, including abnormal vibration patterns, excessive temperature rise, irregular hydraulic pressure fluctuations, and unexpected power losses. These fault characteristics, often represented as fuzzy parameters, can be effectively processed using fuzzy logic-based methods. However, the development of robust and reliable diagnostic rules remains a significant challenge in ensuring the accuracy and effectiveness of fault detection.

### 2.3.2 Model-Based Methods

In model-based fault diagnostics, a residual signal is typically derived by comparing the measurable system output with the output generated by a mathematical model[27]. This residual is then analyzed to determine fault presence, location, and severity[28]. The development of high-fidelity models for Construction Machinery, including mechanical, hydraulic, vibration, and multi-physics models, serves as a fundamental framework for model-based fault diagnosis. By offering a comprehensive representation of the dynamic behavior of construction machinery, these models facilitate not only fault detection but also fault localization and severity estimation. Furthermore, the integration of vibration analysis significantly enhances diagnostic accuracy by identifying anomalies related to structural integrity, component misalignment, and excessive wear, thereby enabling more precise fault detection and predictive maintenance strategies. However, model-based diagnostic methods are inherently susceptible to model uncertainties, external interferences, and measurement noise, which may compromise their reliability in practical applications. These methods can be broadly categorized into four primary approaches, including the state estimation, structural analysis theory, parameter estimation and parity space.

In construction machinery, state estimation methods primarily utilize observers or filters to reconstruct and estimate critical internal states, such as hydraulic pressure, structural vibration levels, thermal conditions, and mechanical stress distributions. By comparing these estimated signals with real-time sensor measurements, residuals containing fault-related information can be extracted, facilitating fault detection and diagnosis[19]. Structural analysis theory leverages the over-determined structure within the system's dynamic equations[29] to perform fault detectability and isolability analysis[30–35]. This approach enables the identification of system redundancies that enhance fault localization capabilities. The fundamental principle behind parameter estimation methods for fault diagnosis is that faults induce changes in physical system processes, which in turn alter the model parameters[36,37]. By monitoring variations in key mechanical, hydraulic, vibration, and multi-physics model parameters, fault detection and isolation (FDI) can be effectively achieved in construction machinery. Additionally, the dynamic model of construction machinery establishes a mathematical relationship between system inputs and outputs. The parity space method evaluates this relationship by analyzing input-output measurements to detect deviations from expected behavior, thereby identifying potential faults[38,39]. These model-based

diagnostic approaches provide a robust framework for fault detection, isolation, and severity assessment, contributing to enhanced reliability and predictive maintenance in construction machinery systems. A comparative analysis of the aforementioned model-based fault diagnosis methods is presented in Table 2-3.

表 2-3 基于模型的诊断方法比较

Table 2-3 Comparison of model-based diagnostic methods

| Knowledge-based methods | Key Technologies | Advantages | Disadvantages |
|---|---|---|---|
| State estimation | Reconstruct system state with filters or observers. | Good real-time performance; No need for a large number of inputs stimulus. | Difficult to determine the fault location and damage degree. |
| Structural analysis theory | Structural analysis of system dynamic equations. | Easy to analyze fault detectability and isolability; Workload of selecting residual generators is reduced. | Strongly dependent on the redundant information of the system model. |
| Parameter estimation | Estimate system parameter or fault parameter. | Conducive to fault isolation. | Requires high precision of modeling and sufficient input excitations. |
| Parity space | Equivalent relationship between input and output variables expressed by the system model. | Simple, fast; Suitable for fault isolation. | Affected by model accuracy and noise. |

Various filters and observers have been extensively employed in the fault diagnosis of Construction Machinery, including the Kalman filter (KF)[40], extended Kalman filter (EKF)[41], unscented Kalman filter (UKF)[42], particle filter (PF)[43], Luenberger observer[44], and adaptive observer[45]. Among these approaches, state estimation methods play a crucial role in real-time state monitoring, offering high

efficiency in detecting faults within Construction Machinery. In contrast, parameter estimation methods, such as filter-based techniques[46] and least-squares estimation[47], can be effectively integrated with other diagnostic methods to precisely locate faults in construction machinery. However, these methods demand high model accuracy and sufficient excitation of system inputs[48], making their practical implementation more challenging. For parity space methods, such as the parity equation approach[49] and the constrained optimization method[50], fault isolation for sensors and actuators can be efficiently achieved by analyzing different hypothesized no-fault subsets of system inputs and outputs. This enables accurate identification of faulty components within Construction Machinery. A key advantage of structural analysis theory is its ability to perform fault detectability and isolability analysis independent of specific Construction Machinery parameter values. This significantly reduces the computational burden associated with residual generation for fault isolation, thereby enhancing diagnostic efficiency and system reliability.

### 2.3.3 Data-Driven Methods

These methods directly analyze and process operational data to detect faults, eliminating the need for an accurate analytical model or expert knowledge. In data-driven fault diagnosis for Construction Machinery, the fault detection process is streamlined by disregarding complex fault mechanisms and system degradation, which are often influenced by uncertain and coupled factors. However, the successful application of data-driven approaches typically requires effective pre-processing of raw data to enhance diagnostic accuracy. Moreover, due to the absence of explicit fault mechanism modeling, interpreting and analyzing detected faults can be challenging. Additionally, several intrinsic limitations are associated with data-driven methods, including the requirement for extensive historical datasets, high computational costs, and significant training complexity[51]. Commonly employed data-driven approaches in fault diagnosis include machine learning, information fusion signal processing. Artificial Neural Network (ANN)-Based Fault Diagnosis involves learning implicit relationships between input and output data during an offline training phase, subsequently forming a nonlinear black-box model for real-time fault detection in the online operation phase. A well-trained ANN effectively differentiates between normal and abnormal states within a Construction Machinery, enabling automated fault identification and classification. Support Vector Machine (SVM)-Based Fault Diagnosis employs a kernel function to transform the input space into a higher-

dimensional feature space, where it identifies the optimal hyperplane for classification. This method treats Construction Machinery fault diagnosis as a sample classification problem, leveraging historical data to train a highly accurate fault classification model. Information Fusion-Based Fault Diagnosis integrates multi-source data to enhance fault detection reliability. By analyzing uncertain and heterogeneous information, this approach improves reasoning and decision-making, leading to more robust fault diagnostics. Signal Processing-Based Fault Diagnosis utilizes advanced signal processing techniques to extract fault feature parameters, including deviation, variance, entropy, and correlation coefficients. These extracted features are then compared against reference values from a normal operating state, enabling early fault detection and anomaly identification.

Given that signal processing-based approaches do not account for the dynamics of Construction Machinery, they are relatively easy to implement and well-suited for fault detection. However, these methods face challenges in fault localization, particularly in cases where multiple faults are coupled within the system. Machine learning algorithms possess the capability to adapt to training data by dynamically adjusting model parameters, enabling them to extract meaningful patterns from training samples. Among these, artificial neural networks (ANNs) offer a highly accurate black-box modeling approach for Construction Machinery fault diagnosis. However, the limited availability of fault data may lead to overfitting, reducing the generalization capability of the ANN model and potentially resulting in false alarms. This study addresses this issue through data augmentation techniques to improve model robustness. Compared to ANN, support vector machines (SVMs) exhibit superior generalization ability, making them particularly suitable for small sample datasets[52], which is often the case in Construction Machinery fault diagnosis. However, the performance of SVM is heavily dependent on the selection of an optimal kernel function, which remains a critical challenge for specific diagnostic applications. Furthermore, while SVMs are effective in handling linear and moderately nonlinear problems, they often struggle with high-dimensional data unless carefully preprocessed. In recent years, ensemble learning techniques such as Random Forests and Gradient Boosting Machines have gained traction for their robustness and ability to mitigate overfitting by combining the predictions of multiple base learners. These methods can improve fault classification accuracy, but they typically require substantial feature engineering to extract domain-relevant characteristics, which may be time-consuming and reliant on expert input.

Table 2-4 presents a comparative analysis of various data-driven diagnostic methods for Construction Machinery.

表 2-4 基于数据驱动的诊断方法比较

Table 2-4 Comparison of data-driven diagnostic methods

| Data-driven methods | Key Technologies | Advantages | Disadvantages |
|---|---|---|---|
| Artificial neural network | Neural network structure;<br>Dynamic adjustment of variable weights. | Self-learning from samples;<br>Strong adaptability;<br>Parallel processing. | Need massive historical data and long training time;<br>Poor generalization ability;<br>Over-fitting problems. |
| Support vector machine | Kernel function selection. | Good generalization ability;<br>Applicable to small sample cases. | Difficult to select the optimal kernel function;<br>Low efficiency for large-scale training sets. |
| Information fusion | Appropriate information fusion algorithms. | More accurate diagnostic result. | Difficult to select effective fusion algorithms. |
| Signal processing | Equivalent relationship between Appropriate signal processing techniques. | Easy to implement;<br>Applicable to both linear and nonlinear systems. | Difficult to detect minor faults and directly locate faults;<br>Not suitable for systems with highly coupled components. |

## 2.4 深度学习（Deep Learning）

Deep Learning (DL) is a subfield of Machine Learning (ML) and Artificial Intelligence (AI) that focuses on training artificial neural networks (ANNs) with multiple layers, commonly referred to as deep neural networks (DNNs). These networks automatically learn hierarchical representations from data, making DL highly effective for complex pattern recognition, feature extraction, and high-dimensional data processing. As illustrated in Figure 2-5, DL exists as a subset of ML , which in turn falls under the broader domain of AI.

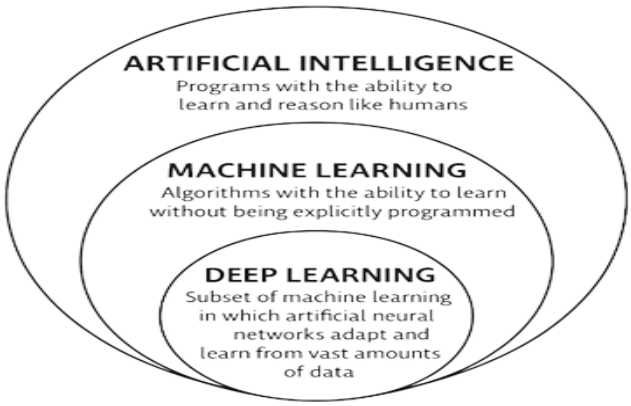



图 2-5 人工智能、机器学习和深度学习的关系

Figure 2-5 Relationship of AI, ML and DL

ML algorithms are primarily employed for three fundamental tasks including Classification, Regression, and Clustering shown in the Figure 2-6.

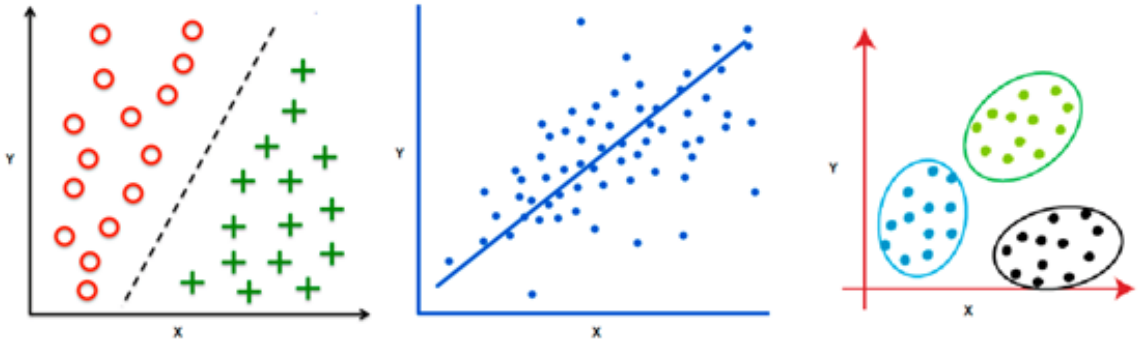



图 2-6 机器学习执行的分类、回归和聚类任务

Figure 2-6 Classification, Regression, and Clustering task performed by the ML

Classification is a supervised learning task where the objective is to assign input data to one of several predefined categories or classes. In this context, the algorithm is trained using labeled data, which consists of input-output pairs, to learn the mapping between features and target labels. Once trained, the model can then classify unseen data based on this learned relationship. Classification models are widely used in various applications, such as image recognition, medical diagnosis, email spam detection and fault diagnosis. Similar to classification, regression is a supervised learning task, but its goal is to predict a continuous numerical value based on input data. In regression tasks, the model learns the relationship between input features and a continuous target variable. This type of machine learning is commonly applied in scenarios where the prediction of a numerical outcome is required, such as in financial forecasting, real estate price prediction, and sensor data analysis. Clustering is an unsupervised learning task aimed at grouping data points into clusters or groups based on their inherent similarities. Unlike classification and regression, clustering algorithms do not rely on

labeled data but instead seek to uncover the underlying structure of the data by grouping similar instances together. This task is particularly useful in exploratory data analysis, pattern recognition, and anomaly detection. Common applications include customer segmentation, image segmentation, and data mining.

The term "deep" in DL distinguishes it from traditional ML algorithms, such as Support Vector Machines (SVM), Artificial Neural Networks (ANN), and other shallow learning methods, by the presence of multiple layers of non-linear transformations. In contrast to shallow neural learning methods, where feature extraction from data samples must be manually performed, deep learning models automate the process of feature representation learning. This is achieved through layer-by-layer transformations of the original data, facilitated by backpropagation, allowing for the creation of hierarchical feature representations that are highly abstract and tailored to specific tasks. A key advantage of deep learning is its ability to perform end-to-end learning, directly transforming raw data into the outcomes of classification and regression tasks, without the need for manual intervention in feature extraction shown int the Figure 2-7.

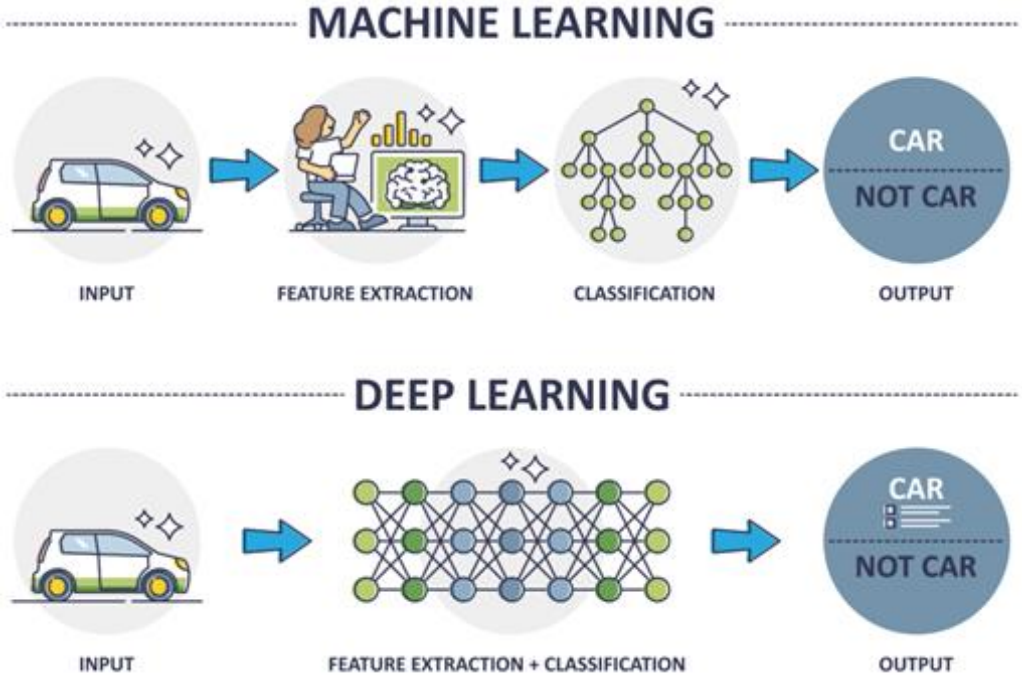



图 2-7 深度学习与传统机器学习算法比较

Figure 2-7 DL and traditional ML algorithms comparison

An Artificial Neural Network (ANN) is a computational model inspired by the structure and functioning of the biological neural networks found in the human brain. It is a fundamental component of ML and DL. The Artificial Neuron (AN) serves as the fundamental computational unit in ANN. Each artificial neuron typically consists of three key components:

Input- Receives weighted inputs from previous neurons or external data sources.

Activation Function- Applies a mathematical transformation to introduce non-linearity, enabling the network to learn complex patterns.

Output- Produces the neuron's final output, which is passed to subsequent layers or serves as the network's prediction.

The structural representation of an AN is illustrated in Figure 2-8. In AN, inputs represent the raw data that is processed by AN to simulate the functionality of biological neurons. Each input is associated with a corresponding weight, which determines its influence on the AN activation. The mathematical relationship governing the components is defined as follows:

$$u_i = \sum_j w_{ij} x_j + \theta_j \qquad (2-6)$$

$$Y_i = f(u_i) \qquad (2-7)$$

where $x_j$ represents the input signals to the neuron, $w_{ij}$ denotes the weights associated with each input connection, $\theta_j$ is the bias term, which allows the model to shift the activation function, $u_i$ represents the weighted sum of inputs before activation, $f(u_i)$ is the activation function, which introduces non-linearity and determines the neuron's output $Y_i$.

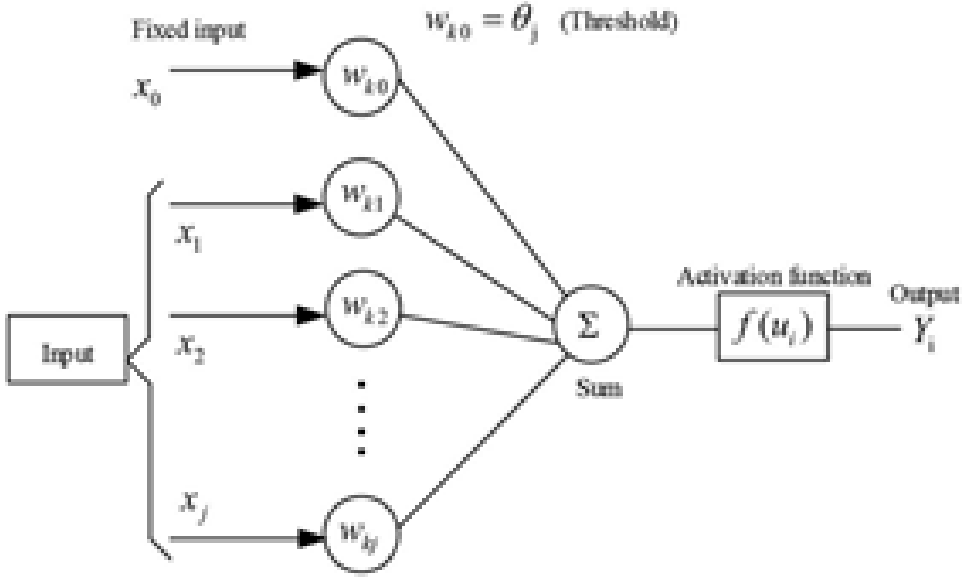



图 2-8 人工神经的基本结构

Figure 2-8 Basic structure of an AN

There are several types of activation functions commonly used in neural networks shown int the Figure 2-9, each with unique properties that make them suitable for different tasks.

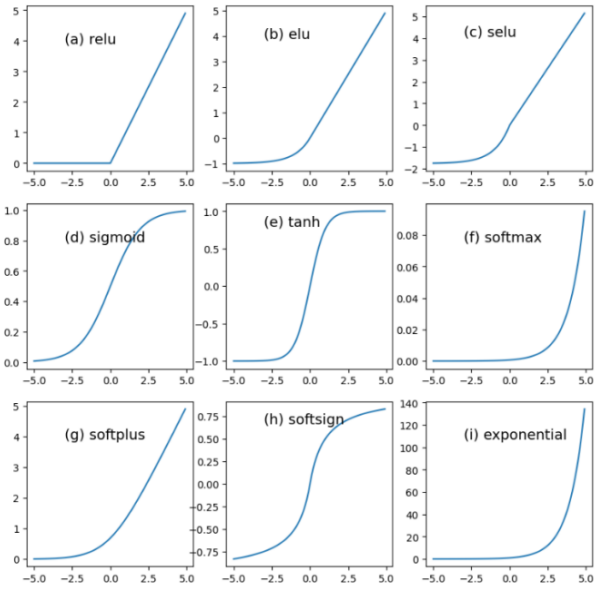



图 2-9 常见的激活函数

Figure 2-9 Popular Activation Functions

An ANN consists of multiple interconnected ANs, which are organized into three main layers:

Input Layer – Receives raw data (e.g., sensor readings, images, or numerical values).

Hidden Layers – Intermediate layers where neurons apply activation functions and extract hierarchical features from the input data.

Output Layer – Produces the final result, such as a classification label or a predicted value.

The structural representation of an ANN is illustrated in Figure 2-10.

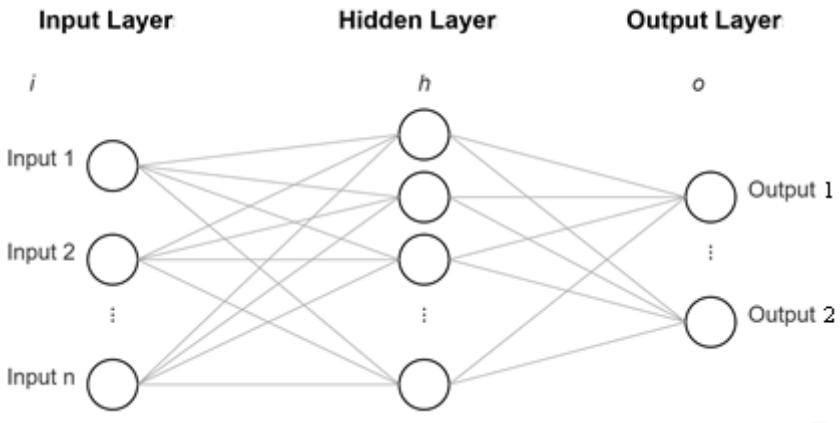



图 2-10 人工神经网络的基本结构

Figure 2-10 Basic structure of an ANN

The training process in Artificial Neural Networks (ANNs) consists of two primary stages: forward propagation and backpropagation Figure 2-11.

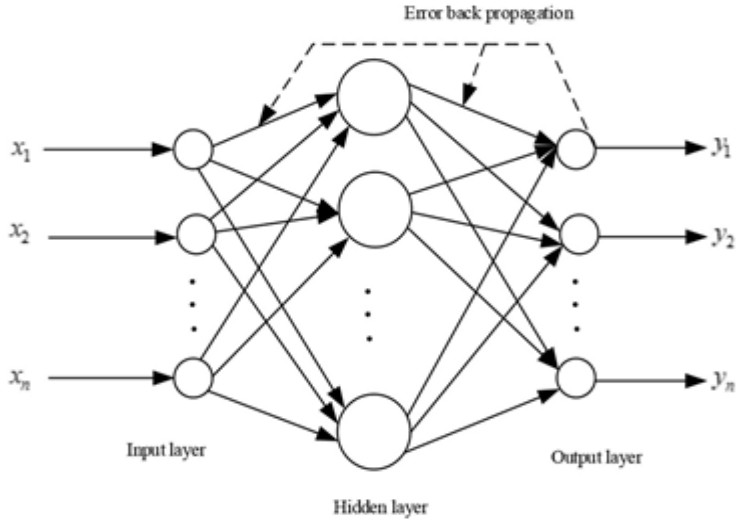



图 2-11 人工神经网络训练过程

Figure 2-11 ANN training process

During forward propagation, the output at each layer is transmitted exclusively to the neurons in the subsequent layer. If the output layer does not yield the desired result, the system employs backpropagation to adjust the connection weights within the network. This iterative process continues until the error is reduced to an acceptable level of accuracy. The learning principle of an ANN involves processing input through the hidden layers to generate an output. The error between the actual and predicted outputs is then calculated, and the error function is used to update the connection weights and neuron thresholds. Once the error is minimized, the relationship between input and output is effectively established. In contrast to traditional ANNs, which typically feature one or two hidden layers, DNNs are characterized by multiple hidden layers shown in the Figure 2-12.

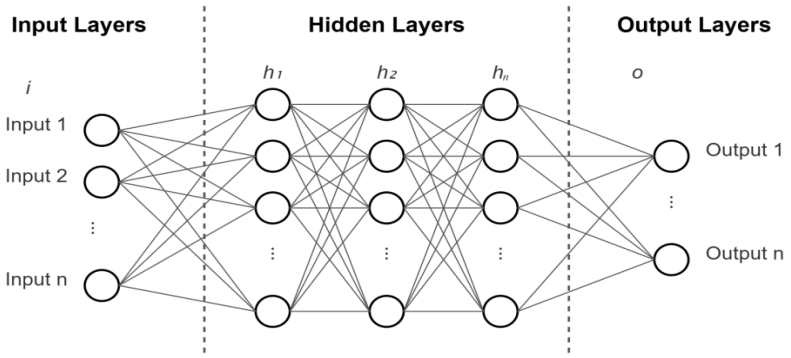



图 2-12 深度神经网络的基本结构

Figure 2-12 Basic structure of an DNN

This deeper architecture enables DNNs to learn progressively abstract representations of the input data, allowing them to capture complex and intricate

patterns that may remain undetectable in shallower network structures. To train a DNN, the same forward propagation and backpropagation principles apply, but with the added complexity of managing many layers of weights and activations.

Typical DL architectures include Deep Belief Networks (DBN)[53], Autoencoders (AE)[54], Convolutional Neural Networks (CNN)[55], and Recurrent Neural Networks (RNN)[56]. Over the past few years, the rapid advancement of DL techniques has led to the development of numerous new architectures, which have been successfully applied to intelligent Fault Detection and Prediction (FDP) tasks. Notable examples of these emerging architectures include Generative Adversarial Networks (GAN)[57], Transformers[58], and Graph Neural Networks (GNN)[59]. Additionally, Convolutional Neural Networks (CNN) have experienced a resurgence, driven by significant progress in the field of computer vision. A more detailed discussion of these DL architectures and their applications in FDP will be presented in the subsequent chapter.

## 2.5 现代深度学习技术在故障诊断中的应用（Modern Deep Learning Techniques for FDP）

DL based approaches can be categorized based on the type of supervision into two primary types: unsupervised methods and supervised methods. Unsupervised methods aim to uncover inherent patterns or structures within unlabeled data, while supervised methods focus on learning complex, non-linear relationships between input data and their corresponding labeled outputs. More specifically, supervised methods can be further classified according to their objectives, such as processing specific data types or extracting distinctive features. The Figure 2-13 illustrates the categorization of the major DL-based approaches commonly applied in intelligent Fault Detection and Prediction (FDP). Moreover, hybrid and semi-supervised frameworks are emerging to bridge the gap between unsupervised and supervised paradigms. For instance, pretraining a deep model on unlabeled data using unsupervised learning and fine-tuning it with a small labeled dataset has proven effective in low-resource settings. In addition, self-supervised learning—where the model learns from automatically generated labels or pretext tasks—is gaining attention for its ability to leverage large volumes of unlabeled operational data in industrial settings.

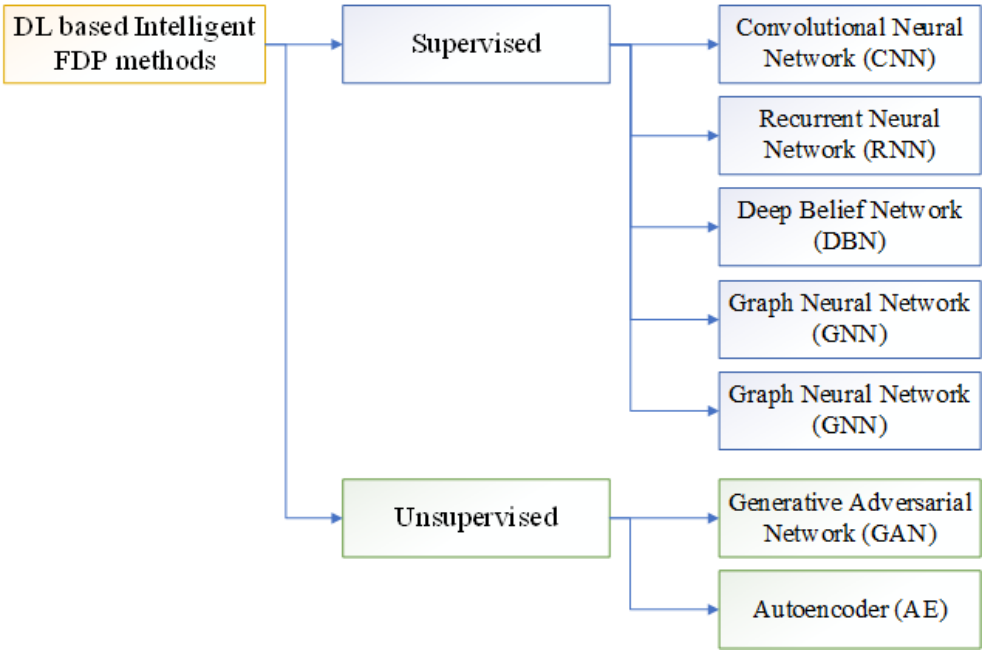



图 2-13 智能故障诊断中的深度学习技术分类

Figure 2-13 The categorization of deep learning techniques in intelligent FDP

A more detailed discussion of these approaches will be provided in the subsequent sections.

### 2.5.1 Supervised DL Methods

Supervised learning methods utilize a training dataset consisting of input-output pairs to train models, enabling them to produce the desired outputs. In the context of intelligent FDP, supervised learning techniques can be employed to extract task-specific features from various types of sensory data, thereby enhancing the model's ability to effectively address the specific requirements of the task.

(1) Convolutional Neural Network (CNN)

The CNN is inspired by the biological mechanisms of visual perception. It exhibits distinct structural features, such as local connections, weight sharing, and pooling, which collectively endow CNNs with robust capabilities in feature learning and representation. Currently, CNNs are predominantly applied in fault diagnosis tasks, though their ability to perform status trend analysis or fault prognosis of equipment remains limited. Within the domain of intelligent FDP, three primary configurations are commonly encountered: 1) conversion of other sensory data into image, 2) formats the use of 1-D CNNs for signal processing, and 3) monitoring sensors are implemented as cameras. Typically, the monitoring variables observed by sensors are one-dimensional signals, which differ from two-dimensional images. To harness the powerful feature learning capabilities of Convolutional Neural Networks (CNNs), many researchers

have explored the conversion of one-dimensional signals into two-dimensional images, which can then be processed by CNNs for classification or recognition tasks. For instance, Ref[60] proposes an intelligent fault diagnosis method for aeroengine sensors by combining CNNs with time-frequency analysis, transforming the signal recognition problem into an image recognition problem. An example pipeline for this process is depicted in Figure 2-14.

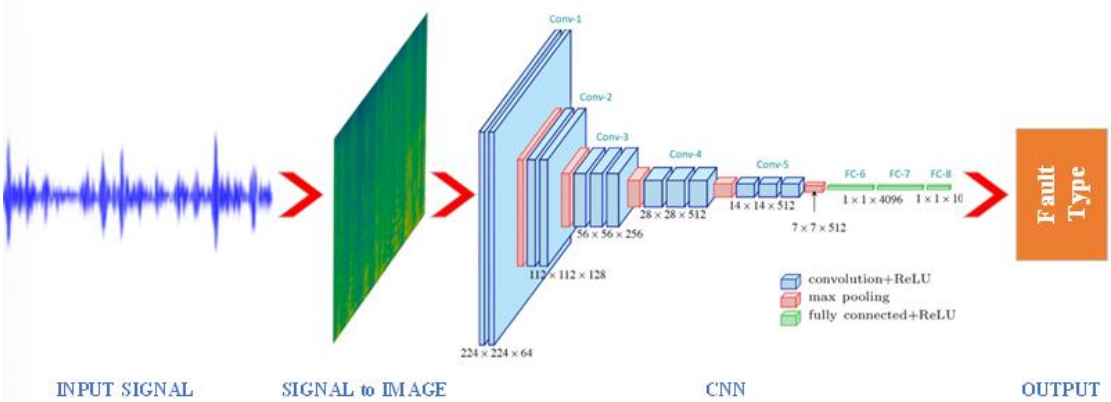



图 2-14 基于信号到图像转换的典型 CNN 故障诊断流程

Figure 2-14 A typical CNN fault diagnosis pipeline based on signal-to-image conversion

A significant focus of such studies is on the conversion of one-dimensional signals into two-dimensional images, using methods such as wavelet transform[60,61,62], S-transform[63], and phase space reconstruction[64]. These time-frequency distribution images, generated through such transformations, often exhibit simpler backgrounds compared to natural images. The quality of these transformation methods directly impacts the performance of the CNN. If the distinction between two-dimensional images representing fault and non-fault signals is minimal, the classification accuracy of the CNN may be compromised. Two-dimensional convolution operations can be decomposed into two one-dimensional convolutions, applied vertically and horizontally. This has led to the exploration of adapting two-dimensional CNNs for one-dimensional data, resulting in the development of 1-D CNNs[65,66], which are particularly suited for processing temporal signals[67]. This approach is inherently well-suited for sensor data and has gained widespread use in intelligent Fault Detection and Prediction (FDP) in recent years. One notable example within 1D-CNNs is the Wide Deep Convolutional Neural Network (WDCNN)[68], which has garnered attention for its capability to process raw vibration signals without requiring pre-processing or feature engineering. WDCNN employs wide kernels in its initial convolutional layer to mitigate high-frequency noise, followed by deeper layers with smaller kernels to capture detailed hierarchical features. A WCNN show in the Figure 2-15.

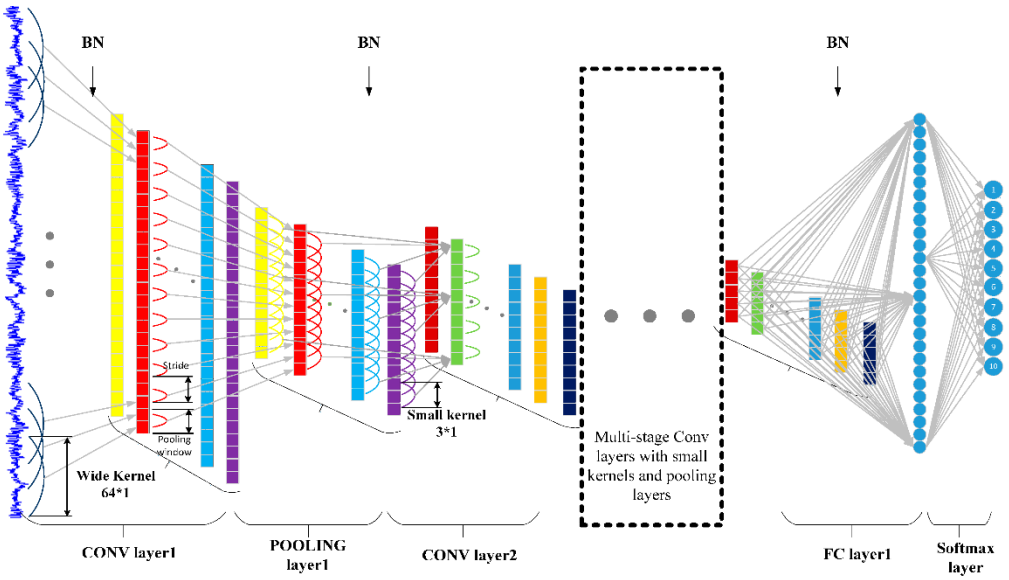



图 2-15 一维 CNN (WDCNN) 架构

Figure 2-15 1D CNN (WDCNN) architecture

Additionally, Ref[69] proposes a 1-D CNN-based method for automatic feature learning in rub-impact fault diagnosis using raw vibration signals from a rotor system, while Ref[70] develops a fault identification model leveraging the robust feature extraction and complex data analysis capabilities of 1-D CNNs. Due to its inherent flexibility, many advanced techniques from 2-D CNNs, such as attention mechanisms[71], lightweight designs[72], and dilated convolutions[73], can be effectively adapted to 1-D CNNs to enhance signal feature extraction. When a device fault can be detected via visual evidence, such as through pixel-level reflections captured by a camera, CNN-based methods tend to yield superior diagnostic results, particularly in fields such as machinery and circuit analysis. For example, Ref[74] proposes a fault diagnosis strategy for rotating machinery utilizing infrared thermal images processed by CNNs. Similarly, Ref[75] incorporates an attention mechanism within a CNN framework to efficiently extract fault features from analog circuits. In another study, Ref[76] employs an encoder-decoder-style CNN to detect cracks on device surfaces amidst complex backgrounds. While the diagnosis of such image data can effectively capture qualitative fault trends, it often falls short of providing precise quantitative descriptions of the faults. While Convolutional Neural Networks (CNNs) offer an alternative approach for processing various types of condition monitoring data, they are not without limitations. Firstly, the conversion of signal data into images involves a quantization process, which can lead to the loss of important signal details during the projection onto pixel bins. As a result, subtle abnormalities, particularly in

the early stages of fault development, may be overlooked due to the nature of convolution and pooling operations. Additionally, the design of conversion methods must be carefully considered to avoid overfitting. Moreover, CNN-based Fault Detection and Prediction (FDP) methods face challenges in achieving real-time diagnosis due to their relatively high computational overhead, stemming from the complexity of multiple layers. These issues will be addressed and discussed in Chapter3.

(2) Recurrent Neural Network (RNN)

In contrast to other architectures, Recurrent Neural Networks (RNNs)[56] are designed with the assumption that input and output are not independent, aiming to capture long-term dependencies from sequential or time-series data. RNNs consist of non-linear recurrent units with directed cycles, coupled with unit hidden states, which enable the preservation of time-series information. This structure allows the hidden layer's state to be influenced not only by the current input data but also by previous computations, enhancing the network's ability to model dynamic behaviors. RNNs are theoretically well-suited for non-linear time-series forecasting and serve as universal approximators for dynamic systems. Notable variants of RNNs, such as Gated Recurrent Units (GRU)[77,78] and Long Short-Term Memory (LSTM) networks[79–81], have emerged as highly effective methods for Fault Detection and Prediction (FDP) using time-series data. A comparison of the basic units of these architectures is provided in Figure 2-16.

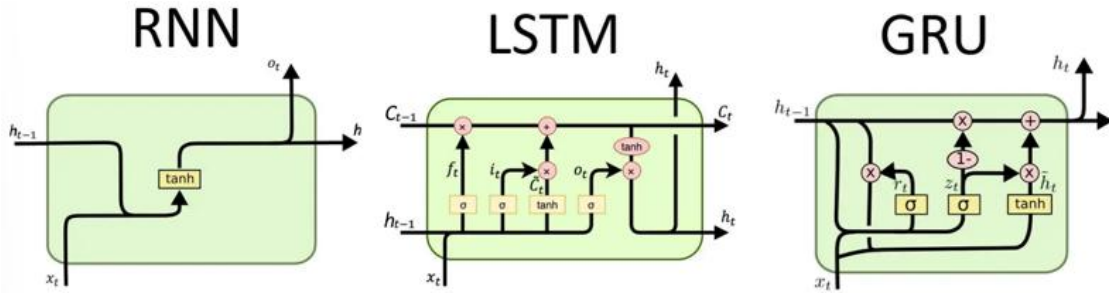



图 2-16 常见的 RNN 架构

Figure 2-16 Common RNNs architectures

Given the continuous collection of long-term condition monitoring data, RNN-based methods are increasingly sought after in intelligent Fault Detection and Prediction (FDP) applications. For example, Ref[81] introduces a convolutional LSTM that simultaneously extracts time-frequency domain features and models their long-term dependencies in vibration signals from bearings. Ref[82] applies LSTM for fault diagnosis and Remaining Useful Life (RUL) estimation using time-series data from aeroengines. In Ref[83], an RNN is employed to implement early warning systems

during the fault creep period for nuclear power machinery, integrating techniques such as principal component analysis, wavelet analysis, and Bayesian inference models. Additionally, Ref[56] proposes a fault prognosis approach using LSTM to model the degradation sequence of equipment, leveraging concatenated features and operational state indicators for RUL estimation. On one hand, the unique structure of recurrent units with directed cycles allows RNNs to effectively model time-series data. On the other hand, this same structure results in slower training times compared to other architectures, such as CNNs, which presents significant computational challenges for industrial computing centers. Additionally, RNNs are highly sensitive to the quality of training data, and when fault features are weak or distorted by noise, maintaining robust performance becomes difficult.

(3) Deep Belief Network (DBN)

Traditional neural networks are more computationally efficient when limited to a few hidden layers, making them suitable for solving relatively simple mapping problems. The Deep Belief Network (DBN) is a network constructed by stacking Restricted Boltzmann Machines (RBMs), which are a special type of generative stochastic neural network consisting of visible and hidden units. A basic example of a DBN with two hidden layers is illustrated in Figure 2-17.

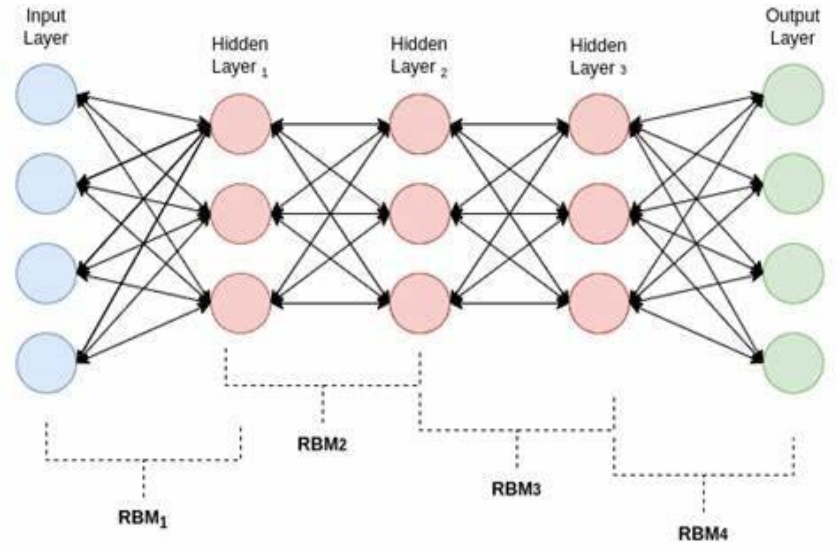



图 2-17 DBN 架构

Figure 2-17 DBN architecture

DBNs can be trained through the pre-training of the stacked RBMs. With multiple hidden layers, DBNs eliminate the reliance on prior knowledge and can adaptively extract fault features for diagnosis. Additionally, DBNs are capable of processing non-linear, high-dimensional data, effectively mitigating issues such as the curse of

dimensionality. As a result, DBNs are particularly well-suited for fault diagnosis in the context of Construction Machinery Big Data. Numerous studies based on Deep Belief Networks (DBN) have been conducted in the field of fault diagnosis, with applications spanning across various industries, including aircraft engines[84], reciprocating compressors[85,86], gearboxes[87–90], rolling bearings[91–94], and power transformers[95,96]. In these studies, DBNs are typically employed either as classifiers in a supervised manner or to replace traditional signal processing techniques for feature extraction in an unsupervised manner. Despite being a foundational technique in deep learning (DL), DBNs require careful tuning of numerous parameters, and improper handling of these parameters can negatively impact their generalization ability and limit accuracy, especially when compared to more modern DL techniques. Consequently, DBNs are increasingly being integrated with other architectures, such as Convolutional Neural Networks (CNNs), to enhance performance and achieve more accurate results.

(4) Graph Neural Network (GNN)

While the aforementioned deep learning techniques effectively capture hidden features and model inherent knowledge from input data in an end-to-end manner, many overlook the inter-dependencies between data points or the various physical measurements collected from multiple sensors[97]. Since the pioneering work of[98], which applied neural networks to directed acyclic graphs, Graph Neural Networks (GNNs) have emerged as a powerful tool for handling data characterized by complex spatiotemporal relationships[99]. While traditional deep learning methods are adept at uncovering patterns in Euclidean domains, an increasing amount of data is generated from non-Euclidean domains, often represented as graphs with intricate spatiotemporal relationships among objects. GNNs are designed to model these relationships through graph representations, which consist of feature nodes and adjacency edges, and focus on tasks such as node classification (at the node level), edge classification and link prediction (at the edge level), and graph classification (at the graph level)[97,100]. GNNs can be integrated with other architectures and extended into more advanced models like Graph Convolutional Networks (GCNs)[101], Graph Attention Networks (GATs)[102], and Graph Autoencoders (GAEs)[103]. A typical graph structure in a GNN is represented by a node feature matrix, an adjacency matrix, and a set of weighted edges. Information is propagated across the graph's edges using graph operations such as graph convolutions, which enable the model to learn meaningful node or graph representations. Among the various GNN models, the Graph Convolutional Network

(GCN) is the most commonly used, with many of its operations having counterparts in Convolutional Neural Networks (CNNs). For example, GCN performs convolutions on nodes to aggregate information from neighboring nodes via weighted edges, uses the ReLU function for non-linear activation, and applies pooling layers to reduce dimensionality. Although there are some subtle differences in the implementation of these operations, the underlying concepts are quite similar. Due to its ability to model complex relationships in data, Graph Neural Networks (GNNs) have recently attracted significant attention from researchers in the Fault Detection and Prediction (FDP) community. One of the main challenges in applying GNNs to FDP is determining the appropriate method for constructing and representing the graph[99]. Figure 2-18 illustrates an example of a fault diagnosis pipeline based on Graph Convolutional Networks (GCN).

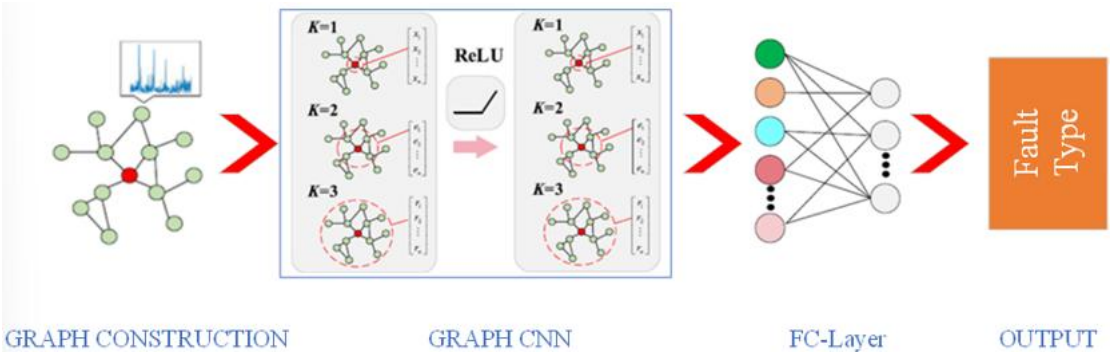



图 2-18 典型的 GCN 故障诊断流程

Figure 2-18 A typical GCN fault diagnosis pipeline

For instance, Ref[101] introduces a GCN-based fault diagnosis approach that constructs an association graph from prediagnostic results and refines the graph by incorporating both measurements and prior knowledge, yielding promising diagnostic outcomes. In the context of time-series data, Ref[97] develops three types of graphs for fault diagnosis and prognosis, each tailored to univariate and multivariate time-series data. Additionally, Ref[104] proposes an interaction-aware GNN for fault diagnosis in complex industrial processes. This approach transforms sensor signals into a heterogeneous graph with multiple edge types and uses the GNN to extract fault features from one particular edge type, enabling the model to learn implicit interactions between sensor signals.

(5) Transformer

Originally developed for natural language processing, the attention mechanism is a technique designed to model sequence dependencies, enabling a model to focus on

specific elements within the input and break down a problem into a sequence of attention-based reasoning tasks[105,106]. Over time, the attention mechanism has been integrated into various deep learning architectures, including Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs). The Transformer architecture[107] represents a departure from traditional recurrent and convolutional structures, utilizing only multi-head self-attention (MSA), multi-layer perceptrons (MLPs), and fully connected layers[108] to capture long-term dependencies between elements in a sequence. This architecture does not rely on the distance between elements, allowing it to comprehensively consider global information across the entire sequence. Figure 2-19 illustrates an example of a fault diagnosis pipeline using the Transformer architecture.

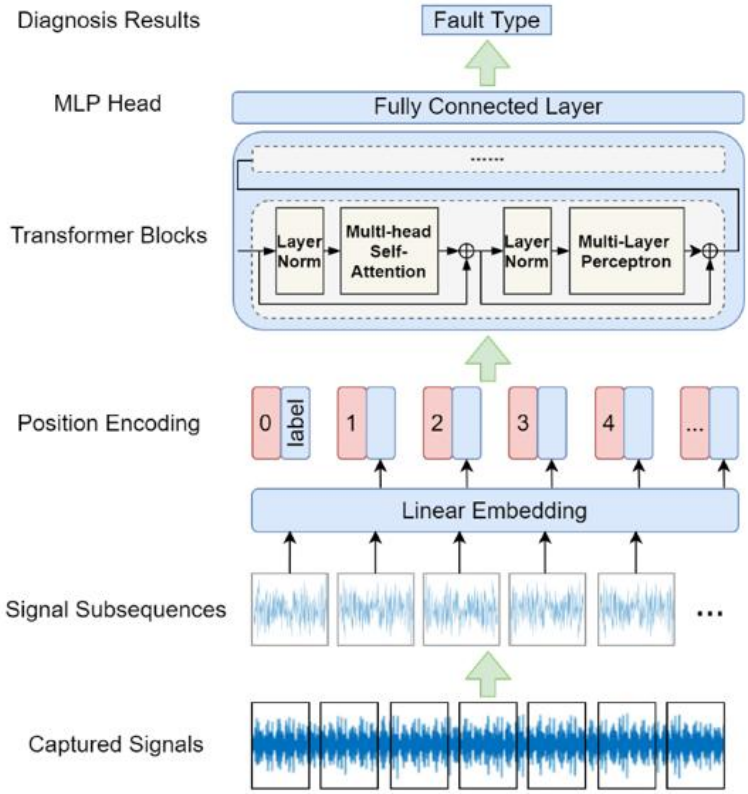



图 2-19 基于变压器的故障诊断示例流程

Figure 2-19 An example pipeline of transformer-based fault diagnosis

The captured signals are initially segmented into subsequences based on their original positions. These subsequences are then mapped into high-dimensional vectors via linear embedding, followed by the application of trainable position encoding to retain the positional information of the signal. The resulting vectors are then passed through multiple stacked Transformer blocks, where long-range dependencies are modeled using layer-normalized multi-head self-attention (MSA) and multi-layer

perceptrons (MLPs). Finally, the extracted features are fed into the MLP head, a fully connected layer, to produce the classification results. Standard loss functions typically used for classification tasks are employed in the model's training process. Due to its exceptional ability to model global information, the Transformer architecture has outperformed other models in feature extraction for various tasks, making it a prominent research focus in Fault Detection and Prediction (FDP) in recent years. For instance, Ref[108] introduces a time-series Transformer that uses raw vibration signals for fault diagnosis in rotating machinery, aiming to capture translation invariance and long-term dependencies through a novel time-series tokenizer. In contrast, Ref[109] designs a time-frequency Transformer, incorporating an innovative tokenizer and encoder module to extract effective abstractions from the time-frequency representation of vibration signals. Ref[58] employs an integrated Vision Transformer (ViT) combined with a soft voting fusion method to achieve high accuracy and generalization in bearing fault diagnosis. For Remaining Useful Life (RUL) prediction, Ref[110] proposes a Transformer-based encoder-decoder structure with dual-aspect encoders to extract features from both sensor data and time steps simultaneously, while adaptively focusing on the most critical portions of the input and processing long data sequences. Transformer-based FDP methods are increasingly being adopted as powerful feature extractors and for time-series data processing, owing to their superior performance in capturing long-range dependencies compared to CNNs and RNNs. However, due to their strong capability in long-distance modeling, Transformers tend to have a relatively weaker ability to capture local information, a limitation that has prompted efforts to combine Transformers with CNNs or RNNs to compensate for this shortcoming. Additionally, the computational efficiency of Transformers, given their complex structure, remains a significant challenge and a key area of focus in the deep learning community. Despite their remarkable potential, further exploration is needed to fully address these limitations and optimize Transformer-based methods for FDP applications.

### 2.5.2 Unsupervised DL Methods

Unsupervised deep learning (DL) methods do not rely on labeled data, and as such, they must extract the underlying structure and patterns from the input data. These methods typically do not address Fault Detection and Prediction (FDP) tasks directly, but rather support ancillary tasks that are equally important, such as feature reduction and data generation.

(1) Generative Adversarial Network (GAN)

An essential requirement for supervised deep learning methods is the availability of a large volume of training samples. However, in many real-world scenarios, the training data is often limited and imbalanced, which manifests in the disparity between the number of positive and negative samples, as well as the availability of known fault patterns. This issue is commonly referred to as the problem of small sample size or small data. Traditional over-sampling techniques struggle to effectively capture the data distribution and are prone to overfitting[111].

In 2014, Generative Adversarial Networks (GANs)[112], an unsupervised learning method, achieved significant success in computer vision tasks. GANs consist of two competing networks: a generator, which creates synthetic samples, and a discriminator, which evaluates the authenticity of the generated samples. Through a minimax game between these two networks, GANs can generate realistic data that fits within the distribution of the original training data, offering a significant improvement over traditional over-sampling techniques, such as the Synthetic Minority Over-sampling Technique (SMOTE)[113].

As a result, GANs have demonstrated exceptional performance across various domains beyond computer vision. In the field of Fault Detection and Prediction (FDP), GANs have been increasingly adopted and have shown promising results compared to other architectures. Recent studies have successfully employed GAN-based architectures to generate synthetic vibration signals, current waveforms, and sensor readings that closely mimic real fault conditions. This not only enriches the dataset but also enhances the generalization capability of supervised models trained thereafter. Additionally, variants such as Conditional GANs (cGANs) and Wasserstein GANs (WGANs) have been introduced to improve training stability and control over generated sample characteristics, further broadening their applicability in industrial FDP tasks. The fundamental concept of data augmentation using GANs is illustrated in Figure 2-20.

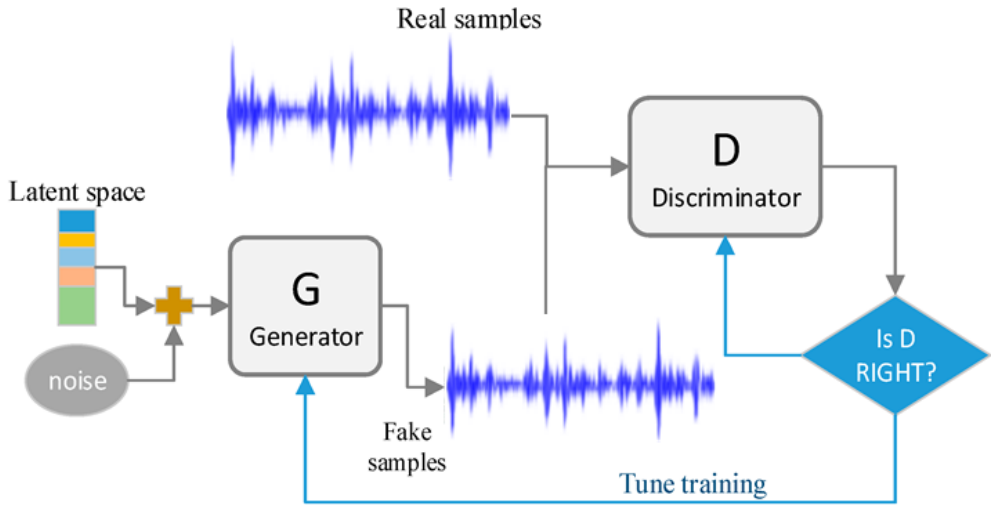



图 2-20 使用 GAN 进行数据增强

Figure 2-20 Data augmentation using GAN

Initially, GANs were primarily adopted for the generation of normal or faulty samples, either for image data or signal data. Figure 2-20 presents an example of a GAN-based approach for data augmentation in the training of deep fault diagnosis models. The ability of GANs to model data distributions can be further leveraged for fault diagnosis applications. For instance, the trained generator can be used to repair a faulty sample, and the fault can be identified by comparing the generated sample with the original[114]. Furthermore, the adversarial learning strategy employed by GANs has been widely utilized to address the challenge of domain shift in data distribution, a common problem in fault diagnosis under varying operational conditions or environments. Specifically, when the distribution of available training data in the source domain differs from that of the data to be tested in the target domain, it becomes difficult for the trained model to generalize effectively[115]. This issue poses significant challenges in Construction Machinery applications, where fault data often vary across different conditions and environments. Due to its unique and powerful capabilities, Generative Adversarial Networks (GANs) have garnered significant attention in the field of intelligent Fault Detection and Prediction (FDP) for real industrial systems. Current research primarily focuses on utilizing the adversarial nature of GANs to generate more realistic samples and address cross-domain adaptation challenges in intelligent FDP. For example, Ref[111] introduces an InfoGAN-based failure prediction algorithm, employing an auxiliary GAN to enforce consistency between the generated samples and their corresponding labels. Ref[116] proposes the use of deep feature-enhanced GANs to ensure the accuracy and diversity of synthetic samples, thereby improving the performance of imbalanced fault diagnosis in rolling bearings. To

address the issue of limited data availability in industrial settings, particularly for damaged conditions, Ref[65] suggests a multilabel 1-D GAN to generate damage data for industrial equipment, resulting in improved fault diagnosis accuracy with the generated data. Ref[117] utilizes a domain-adversarial training approach, combining labeled samples from an auxiliary domain with unlabeled samples from the target domain, enhancing the adaptability of the auxiliary domain samples to the target domain and improving transfer performance.

Despite GANs' ability to generate samples that follow the same distribution as the original data, evaluating the quality of generated 1-D signals remains a challenge, unlike in image generation tasks. Moreover, ensuring the convergence of the adversarial training process to the desired outcome is another key challenge. Additionally, given that generating faulty samples is a primary objective, integrating prior knowledge from domain experts to enhance the generation process remains a critical issue to be explored for real-world industrial applications.

(2) Autoencoder (AE)

Autoencoders (AEs) are unsupervised neural network architectures that assume the output, once encoded and decoded, is identical to the input. In this sense, the encoder component of the AE can be used for feature reduction, converting high-dimensional input data into low-dimensional encoded vectors. The encoder-decoder framework is also widely adopted by other deep learning architectures, such as Convolutional Neural Networks (CNNs). A simple architecture of an AE is shown in Figure 2-21.

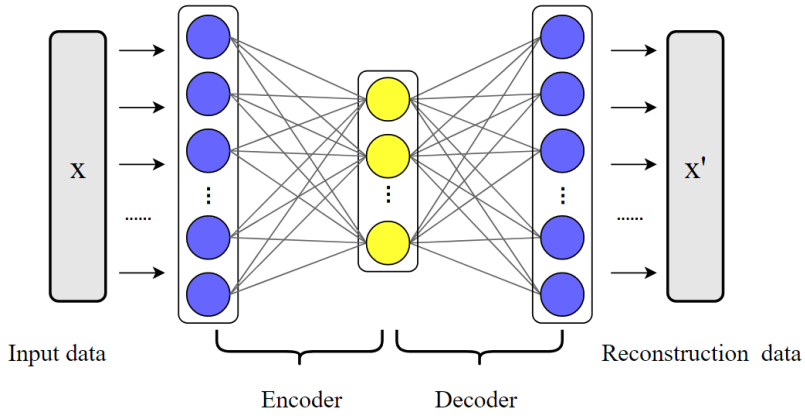



图 2-21 基本自编码器架构

Figure 2-21 Basic AE architecture

AEs can be categorized into various types, including standard AEs[118–120], denoising AEs[121,122], sparse AEs[123], variational AEs[124], and contractive AEs[125], among others. AEs have been extensively utilized for feature extraction and

fault classification, demonstrating remarkable capabilities in non-linear dimensionality reduction and feature extraction, as well as robustness in practical Fault Detection and Prediction (FDP) applications. For example, Ref[126] designs a sparse AE to automatically extract degradation indicators for fault detection in multi-component systems. Ref[127] employs multi-layer sparse AEs as a method for multi-sensor feature fusion and extraction, combined with Deep Belief Networks (DBNs) for bearing fault diagnosis. To achieve enhanced performance, stacked AEs are often preferred in various scenarios, and the boundaries between different AE types are becoming increasingly blurred, leading to hybrid architectures such as sparse denoising AEs. Despite these advantages, AEs still face challenges in extracting meaningful features, which can be difficult due to their inherent properties. Furthermore, their performance is generally highly dependent on the quality and diversity of the training samples.

## 2.6 总结（Summary）

This chapter provided an extensive overview of foundational theories and methodologies pertinent to fault diagnosis in construction machinery, emphasizing data-driven diagnostic approaches. Initially, it defined key terminologies and concepts associated with fault diagnosis and prognosis, clarifying the mathematical formulations necessary for understanding and addressing fault-related issues in complex machinery systems. Subsequently, the chapter explored three primary categories of fault diagnosis methods: knowledge-based, model-based, and data-driven techniques. Knowledge-based methods leverage expert insights, making them intuitive but limited by the necessity of comprehensive prior knowledge. Model-based methods depend on accurate physical or mathematical modeling of system dynamics; these methods offer precise diagnostics but may suffer due to modeling inaccuracies or computational complexity. Data-driven methods, specifically highlighted due to their flexibility and adaptability, were examined in detail, illustrating how machine learning techniques such as neural networks, support vector machines, information fusion, and signal processing are utilized for robust fault diagnosis. Further discussion focused on advanced deep learning techniques, outlining their strengths in automatic feature extraction, handling high-dimensional data, and managing non-linear relationships. Techniques like Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Deep Belief Networks (DBNs), Graph Neural Networks (GNNs), Transformers, Generative Adversarial Networks (GANs), and Autoencoders (AEs) were detailed to showcase their applicability in complex fault diagnosis scenarios, each

with unique advantages and specific implementation considerations. In conclusion, this comprehensive analysis of fault diagnosis methods highlights the necessity of selecting appropriate diagnostic techniques based on specific application requirements, data availability, computational resources, and operational conditions. It lays a solid theoretical foundation for developing robust and intelligent fault diagnosis systems tailored for practical engineering applications.

# 3 云边协同故障诊断框架
# 3 Cloud-Edge Collaborative Fault Diagnosis Framework

The rapid development of information technology has significantly accelerated the evolution of intelligent manufacturing, and this trend is increasingly influencing various industries, including construction machinery. In the context of construction machinery, more and more equipment is being connected to the internet, allowing vast amounts of data to be collected across all stages of the machinery lifecycle. The cloud-based intelligent manufacturing paradigm is paving the way for new applications and services that analyze this data and foster large-scale collaboration in machinery maintenance and fault diagnosis. However, factors such as network unavailability, high bandwidth requirements, and latency can hinder the real-time performance of these systems, especially in critical applications. Edge computing offers a solution by extending cloud computing, storage, and networking capabilities to the edge, thus addressing these challenges. This chapter introduces a layered reference architecture for a fault diagnosis system for construction machinery, built on the principles of cloud-edge collaborative computing.

## 3.1 云计算与边缘计算的概念(Concepts of Cloud Computing and Edge Computing)

### 3.1.1 Cloud Computing

Cloud computing is a computational paradigm that delivers services through the internet, where resources (such as processors and storage) are provided to users in the form of services, which can be accessed on-demand over the network and paid for based on usage. Cloud computing supports on-demand network access to a configurable pool of shared computing resources that can be quickly configured and released with minimal management effort or service provider interaction[128]. The key characteristics of cloud computing are as follows:

Large-Scale, distributed: Cloud service providers offer services to users by pooling multiple geographically distributed server resources, including virtual machines, processors, memory, and network bandwidth. This characteristic allows the cloud to provide powerful computing capabilities to meet user demands.

On-Demand Service: Cloud users can configure resources as needed without any intervention from the cloud service provider. Users can independently configure

computing power, servers, storage, and network resources. This approach to resource management offers greater flexibility and enables rapid response to changes in service demands.

High Availability and Scalability: Cloud providers take measures to ensure that services are more reliable, such as data redundancy and fault tolerance to guarantee high availability. Additionally, the scale of the cloud is dynamic and can be adjusted based on demand, ensuring it meets the specific needs of applications.

Virtualization: Cloud computing supports users in obtaining application services via the network. The requested resources are provided in the form of services, rather than fixed, tangible entities.

Security: In addition to providing computing services, cloud computing also offers storage services. As a result, cloud providers employ professional security teams to ensure the safety of user data and resources.

Cloud computing provides services in a centralized manner. The traditional cloud computing model, as shown in Figure 3-1, involves data being uploaded by data producers to the cloud computing center for computation and storage. Data consumers then send requests to the cloud computing center and receive the response results from the cloud.

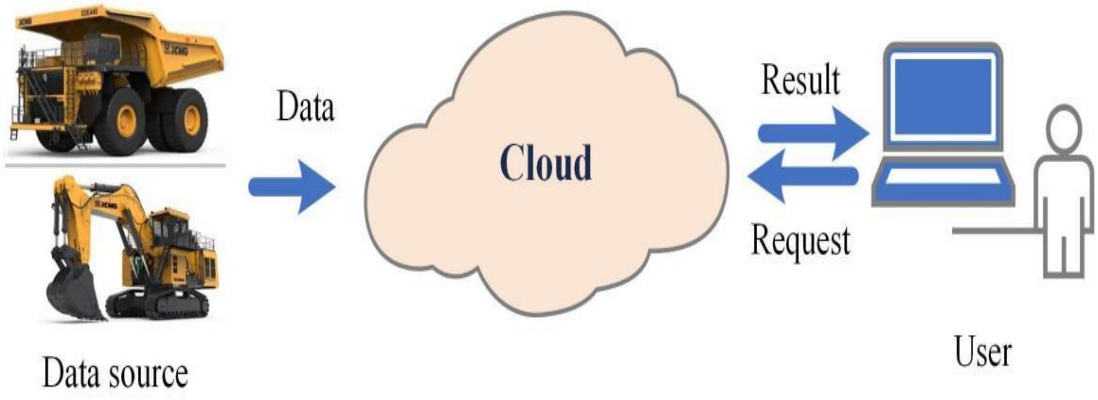



图 3-1 云计算模型

Figure 3-1 Cloud Computing Model

Cloud applications reduce costs by simplifying development complexity and saving on expensive hardware, network bandwidth, cooling equipment, power, and human resources. Users can access these hosted data or applications from cloud servers via terminal devices over the network. Due to these characteristics, more and more applications are expected to migrate to the cloud in the future.

### 3.1.2 Edge Computing

Cloud computing adopts a centralized management approach, where all data must be transmitted to a remote cloud data center for processing. While the highly centralized and integrated resources make cloud computing highly versatile, the explosive growth of IoT devices and data has gradually revealed the limitations of cloud computing in terms of real-time capabilities, network constraints, resource overhead, and security. Cloud servers are often located far from the terminal devices, and the centralized nature of cloud services requires all data to be transmitted to the cloud before processing. This data transmission incurs time delays, and the processing speed is further constrained by factors such as bandwidth, cloud computing capacity, and total task load. The cumulative delay caused by these processes may exceed the system's tolerance, making real-time applications difficult to handle. Since cloud computing requires all data to be transmitted to the cloud for centralized processing, it is highly dependent on a stable and reliable network connection. However, in areas with poor network environments, such as production workshops, rural regions, or mountainous areas, cloud computing cannot provide stable and reliable services. With the generation of massive amounts of data, the transmission of this data consumes considerable network resources, while processing the data requires significant computational and storage resources on the cloud. However, much of this data is unnecessary. For example, in fault diagnosis scenarios, the time a device spends in a fault state is relatively small compared to its entire lifecycle. Most of the data collected by sensors is normal, which is of no value for fault diagnosis. However, transmitting all data to the cloud for processing unnecessarily increases the pressure on cloud resources. Data transmitted to the cloud can vary widely, with some data potentially involving user privacy, such as factory production data. Since the cloud's data processing methods are not visible to the user, this raises potential security concerns. To address these limitations, edge computing has emerged as a new computational paradigm. Edge computing is a distributed open platform that integrates computing, storage, networking, and application capabilities at the network edge, near on-site devices[129]. The edge computing layer is close to the production site, shortening the data transmission path, enabling local data processing and analysis to ensure real-time processing. Additionally, local data processing enhances data security, and edge computing does not rely on high-bandwidth networks, offering offline capabilities as well. The edge computing model, as shown in Figure 3-2, allows the edge to collect data from devices like sensors or smartphones, perform

partial computations such as data storage and preprocessing, and then send the processed data to the cloud for further analysis.

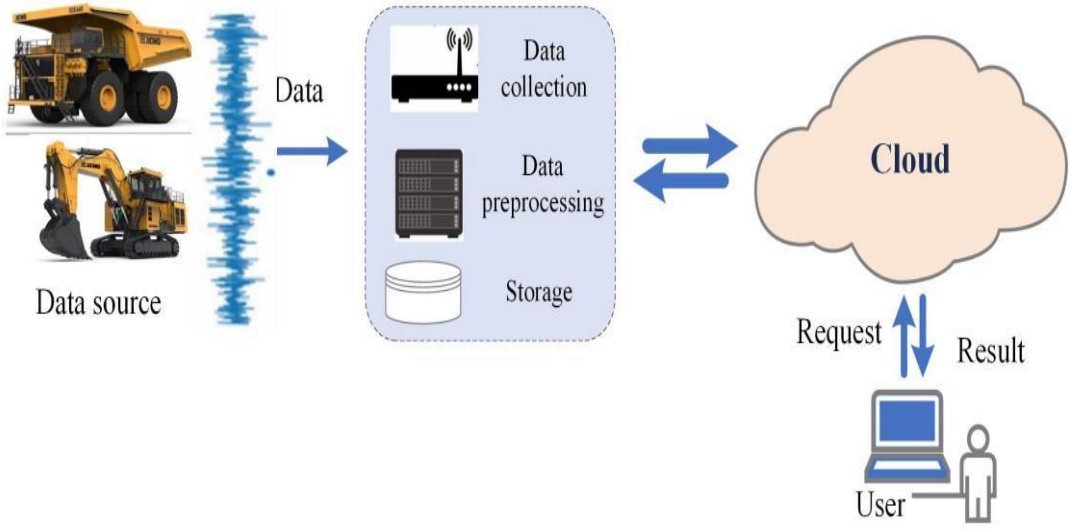



图 3-2 边缘计算模型

Figure 3-2 Edge Computing Model

## 3.2 云边协同概念（Cloud-Edge Collaboration Concept）

The main advantages of cloud computing are massive storage, high-efficiency computation, and wide-area coverage. In contrast, the key advantages of edge computing are real-time data processing, edge resource pools, and support for high mobility and high Quality of Service (QoS). The comparison between cloud computing and edge computing is shown in Table 3-1. From Table 3-1, it can be seen that the application scenarios of cloud computing and edge computing are different. Cloud computing excels in large-scale, long-cycle big data processing and analysis with low real-time requirements, while edge computing is better suited for small-scale, real-time, short-cycle data processing and analysis. Therefore, the relationship between cloud computing and edge computing is not one of replacement but rather complementarity and collaboration. Through the close collaboration of cloud and edge, the demands of various application scenarios can be better met, thereby amplifying the application value of both. Edge computing, which is closer to the terminal devices, serves as the data collection and preprocessing unit and can support cloud applications. Cloud computing has powerful processing capabilities that allow for big data analysis optimization and model training, after which the trained models or business rules are sent to the edge. The edge computing then operates based on the new models or business rules. The capabilities and concepts of cloud-edge collaboration mainly include three types: resource collaboration, data collaboration, and service collaboration:

表 3-1 云计算与边缘计算的比较

Table 3-1 Comparison between Cloud Computing and Edge Computing

| Feature | Cloud Computing | Edge Computing |
|---|---|---|
| Definition | Computing resources and services are provided over the internet from remote data centers. | Computing happens closer to the data source, at or near the "edge" of the network. |
| Latency | Higher latency due to data travel to distant servers. | Lower latency due to proximity to data sources. |
| Processing Location | Centralized, typically in large data centers. | Decentralized, typically at the data source (e.g., IoT devices). |
| Data Storage | Data is stored in centralized cloud servers. | Data is stored locally or in nearby edge devices. |
| Scalability | Highly scalable, as cloud providers offer vast resources. | Less scalable than cloud, depends on local resources. |
| Bandwidth Requirements | High bandwidth required for data transfer to and from the cloud. | Lower bandwidth needed as data is processed locally. |
| Security | Relies on centralized security protocols and measures. | Security is distributed and depends on edge device security. |
| Fault Tolerance | Dependent on the central cloud data centers. | More resilient as local edge devices can continue functioning. |
| Use Cases | Data-heavy applications (e.g., big data, AI, etc.). | Real-time applications (e.g., IoT, autonomous vehicles). |
| Examples | Ali Cloud, Huawei Cloud, AWS, Google Cloud, Microsoft Azure. | IoT devices, smart sensors, edge gateways, 5G. |

Resource Collaboration: Edge nodes can provide basic infrastructure resources such as computing, storage, and networking. They can independently manage and schedule local resources or collaborate with the cloud, accepting and executing cloud-issued resource scheduling management strategies. Data Collaboration: Edge nodes are responsible for data collection, preprocessing and simple analysis of raw data according to models or business rules, and then upload the results and related data to the cloud. The cloud can store, analyze, and mine the value of massive data. Data collaboration between the edge and cloud allows for the orderly flow of data between the two, forming a complete data flow path, which facilitates subsequent lifecycle management

and value mining of the data. Service Collaboration: After the cloud trains the models, it sends them to the edge nodes, which perform inference based on the models. The cloud manages the lifecycle of edge-side applications, including deployment, startup, shutdown, deletion, and version updates. The cloud generates application orchestration strategies, and the edge executes the application according to the cloud's strategy.

## 3.3 提出的框架（Proposed Framework）

The proposed could-edge collaborative fault diagnosis framework as illustrated in Figure 3-3 consists of three planes, including the equipment plane, edge plane, and cloud plane, which work collaboratively to collect data from construction machine, transfer, pre-process, inference, etc., to realize the fault diagnosis efficiently. Detail introduction to these planes is elaborated as follows.

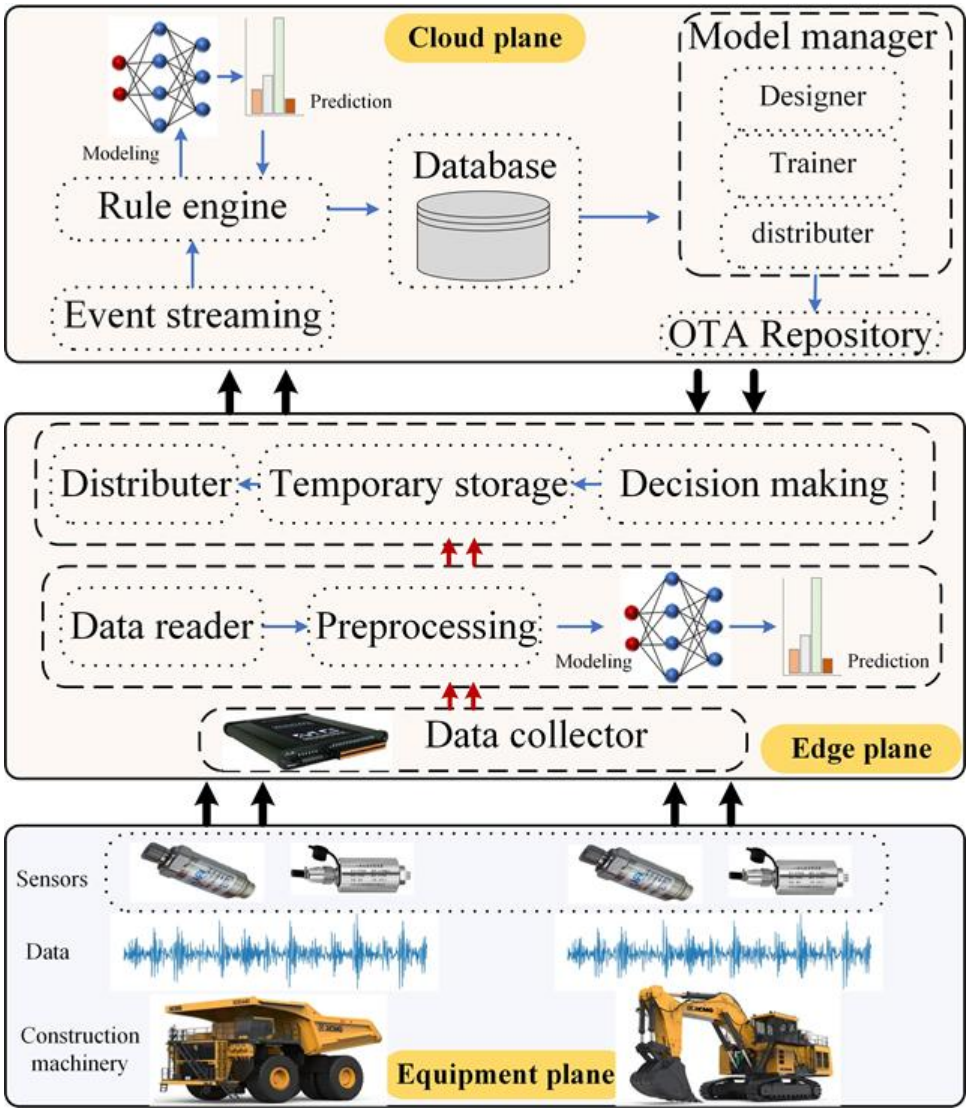



图 3-3 提出的云边协同故障诊断框架

Figure 3-3 Proposed could-edge collaborative fault diagnosis framework

### 3.3.1 Equipment Plane

Equipment plane consists of the machinery required to be diagnose and sensors used for data acquisition. Specifically, this study focuses on fault diagnosis of bearings in construction machinery. Construction machinery plays crucial role in our daily life, they can be used for the construction, excavation, and maintenance of infrastructure, buildings, roads, and other projects. These machines are essential for tasks such as digging, lifting, moving materials, and preparing land for construction work. They are typically large, powerful, and designed to handle tough environments, making construction projects faster and more efficient. Rolling bearings, as critical components in construction machinery, support rotational motion and reduce friction between moving parts. A rolling bearing consists of an inner raceway, outer raceway, rolling balls, a cage and seal. The structure of a rolling bearing is shown in Figure 3-4.


图 3-4 滚动轴承结构图

Figure 3-4 Rolling bearing structure diagram

Rolling bearings are essential for ensuring the smooth operation, stability, and efficiency. Therefore, Fault diagnosis of bearings is crucial to prevent catastrophic failures, improve operational efficiency, and ensure the safety of construction equipment.

Vibration analysis is one of the most effective technique used for bearing condition monitoring, vibration signals capture rich information about the bearings operating status. For example, Figure 3-5 presents a vibration signal collected from bearing with inner race fault under operating speed of 25 Hz, sidebands and peaks at the defect frequency which is Ball Pass Inner Race Frequency (BPFI) can be significantly seen in the envelope spectrum of this signal, and so on for other faults. By accurately analyzing vibration signals we can achieve strong fault diagnostic systems. There are various sensors used for vibration signal acquisition include acceleration sensors, velocity sensors, displacement sensors. Among these, acceleration sensors are the most widely

used due to their versatility, cost-effectiveness, durability, ease of use, and ability to provide real-time data.

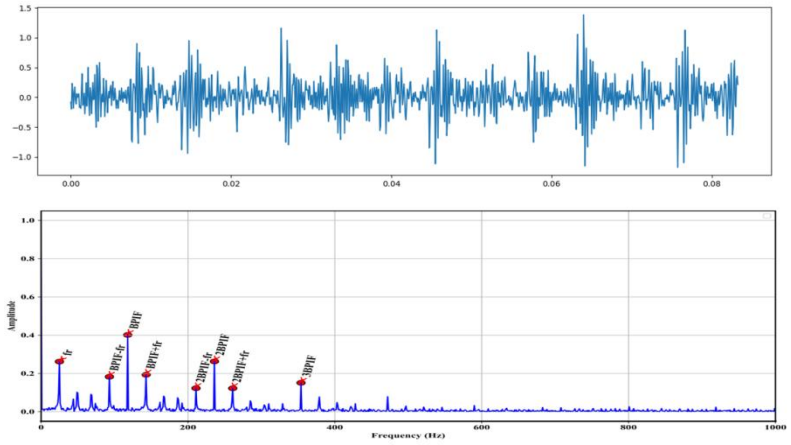



图 3-5 振动信号包络谱分析

Figure 3-5 Vibration signal envelope spectrum analyses

## 3.3.2 Edge Plane

The Edge plane serves as the intermediary layer between the Equipment plane and the Cloud plane, handling the data processing locally to ensure faster fault detection and diagnosis. It consists of data collectors that gather data from the Equipment plane in a centralized location, facilitating easier access and management of the data. The collected data, such as vibration signals from sensors monitoring the condition of the bearings in construction machinery, is then processed by low-power computing units. These computing units, such as industrial PCs (e.g., Raspberry Pi), perform tasks like data reading from data collectors, the data collectors can variate from one manufacturer to another that is why data reading task supports different types of communication protocols like Modbus, CAN, HTTP, MQTT, FTP etc. to ensure interoperability. Data reading task also supports time-based scheduling ensures that data is retrieved at consistent intervals preventing some data be overwritten or lost due to limited storage or network connectivity issues, allowing for real-time monitoring of Equipment plane. Another important task is data preprocessing. Data preprocessing is used to improve fault diagnosis results. In this research case we have used data preprocessing to normalize the sensor data between 0 and 1 or -1 and 1 scale. That is because different sensor signals can have varying scales, amplitudes, or units. Normalization ensures that all input signals are on a consistent scale, preventing any particular signal from dominating others in analysis, particularly in machine learning or fault diagnosis algorithms. By transforming all signals to a uniform scale, algorithms can better analyze

the relative patterns and features in the data. The most important task is fault diagnosis task. The fault diagnosis on the edge achieved by inferencing the edge fault diagnosis model, in this research case this model is an edge-partition of the cloud-edge collaborated DNN model described in Chapter 3 Model Implementation. The fault diagnosis task contains the feature that always keeps track on the edge fault diagnosis model version. if new model is available in the cloud (due to edge compute does not have much computational resources, that is why DNN model training takes place in the cloud server), it will automatically pull from the cloud and updated. Inferenced edge fault diagnosis model returns features and predictions that are used by decision making task. The decision-making task also have access to normalized raw sensor data that is being just diagnosed by fault diagnosis task. The decision-making task taking decision on which data need to be sent to the cloud, based on the decision-making rules. The decision-making rule represents the conditions based on threshold T max min values set for predicted fault. The decision-making flow Algorithm 1 shown in the Table 3-2.

表 3-2 边缘决策流程

Table 3-2 Edge decision-making flow

| Algorithm 1 |
| --- |
| Input: $C$ is prediction confidence vector returned by edge fault diagnosis model, containing the confidence values for each fault class, $T$ is a threshold vector, where each entry $T_i = [T^i min, T^i max]$ contains the minimum and maximum thresholds for each fault class calculated for edge model during model evaluation, $R$ is normalized raw vibration data, $F$ is feature extracted from the normalized raw vibration data. |
| Output: (R or F) and C |
| 1: procedure EDGE_DECISION_MAKING (C, T, R, F) |
| 2: predicted_class = arg max(C) # Find the class with highest confidence |
| 3: max_confidence = C[predicted_class] # Get the confidence of the predicted class |
| 4: T_min, T_max = T[predicted_class] # Get the thresholds for the predicted class |
| 5: if max_confidence < T_min then |
| 6: return R and C |
| 7: else if T_min <= max_confidence < T_max then |
| 8: return F and C |
| 9: else if max_confidence >= T_max then |
| 10: return C |

The algorithm operates as follows: the input consists of the prediction results, represented as a confidence vector $C$. The algorithm identifies the fault class $i$ with

the highest confidence value and compares this value against the confidence thresholds $T_i$ for the current fault class. If the predicted confidence is less than $T^imin$, it indicates that the model was unable to accurately predict the fault. In this case, the raw data and the incorrectly diagnosed class should be sent to the cloud for analysis by a human professional. If the predicted confidence is greater than or equal to $T^imin$ but less than $T^imax$, it suggests that the model has recognized the fault but is uncertain about the specific fault class. Therefore, the extracted features of the fault and the predicted class should be sent to the cloud for further classification. Conversely, if the predicted confidence is greater than or equal to $T^imax$ , the model is highly confident in the prediction. In this case, only the predicted fault type and its associated confidence level should be sent to the cloud for storage. For further details regarding the determination of $T^imin$ and $T^imax$, refer to Chapter 4.4 Cloud-edge decision making threshold values. The decision-making task is flexible and gives ability to set custom rules and actions based on the business needs. Once decision made the decision result goes to the temporary storage waiting for data distribution task. The data distribution task is the key task that is responsible for fault tolerant data transmission to the cloud, In the case of network errors data still is save because it is still saved in the local storage. The distribution task supports multiple network protocols for improved interoperability with cloud and also can control the size of the transmitted data for efficient utilization of network resources.

### 3.3.3 Cloud plane

The Cloud Plane is the head and heart of proposed framework because all the data processed by edge plane eventually riches the cloud for further processing or storage. Training and distribution of the cloud-edge collaborated model also takes place in this plane. That is why could should have powerful computational and storage resources running and fault tolerant cloud server clusters. As we know even ideal systems time to time have some errors, the errors in the cloud bring to data loses and slow respond time of the cloud services, so this research used event streaming approach for overcome these shortages. Event streaming is commonly used to handle high-throughput, real-time data streams, enabling the management and processing of large volumes of telemetry data, events, and other messages in a distributed and fault-tolerant manner. The next challenge, after addressing data handling through event streaming, is processing the data according to evolving business requirements. As business needs are dynamic and subject to change, it is essential to implement a flexible solution that can

adapt to these fluctuations. This research proposes the use of a rule engine as a method to address this challenge, providing the necessary flexibility to accommodate varying business demands. The data handled by event streaming directly flows to the rule engine. Rule Engine is a component that allows users to define, manage, and execute business processes based on incoming data. Incoming data can be of three types that is normalized raw vibration data, feature extracted from the vibration data and fault diagnosis results. There is a business logic Algorithm 2 running in rule engine for processing received data shown in the Table 3-3.

表 3-3 云端业务逻辑（在规则引擎中运行）

Table 3-3 Cloud business logic (running in the rule engine)

| Algorithm 2 |
|---|
| Input: $D$ is the data received by the rule engine, $C$ is prediction confidence vector returned by cloud fault diagnosis model, containing the confidence values for each fault class, $T$ is a threshold vector, where each entry $T_i = [T^i min]$ contains the minimum thresholds for each fault class for cloud model during model evaluation. |
| 1: procedure RULE_ENGIN (D, T) |
| 2: if D has raw_vibration_data then |
| 3: alarm ("Current vibration data fault class is unknown") |
| 4: save D for human analysis |
| 5: else if D has extracted_feature then |
| 6: C = cloud_model. predict(extracted_feature) |
| 7： predicted_class = arg max(C) # Find the class with highest confidence |
| 8: max_confidence = C[predicted_class] |
| 9: T_min = T[predicted_class] # Get the threshold for the predicted class |
| 10: run DECISION_MAKING (D, predicted_class, max_confidence, T_min) |
| 11: else if D has edge_predicted_result then |
| 12: if D has fault, then |
| 13: alarm(D) |
| 14： save D for further analysis and monitoring |
| 15: procedure DECISION_MAKING (D, predicted_class, max_confidence, T_min) |
| 16: if max_confidence < T_min then |
| 17: alarm ("Cloud model could not recognize the fault") |
| 18: save D and predicted_class for human analysis and cloud model fine-tuning |
| 19: else if max_confidence >= T_min then |
| 20: alarm(predicted_class) |
| 21: save predicted_class for monitoring |

The algorithm operates as follows: When raw vibration data is received, the Rule

Engine triggers an alarm indicating that the fault class associated with the current vibration data is unknown, subsequently saving the data to the database for future human analysis. In the case of extracted features, this indicates that the edge model has recognized a fault but is uncertain about the fault class. As a result, the Rule Engine forwards the features to the cloud model for further analysis and prediction. The cloud model is a cloud-partition of the cloud-edge collaborated DNN model described in Chapter 4 Cloud-edge collaborated Fault Diagnosis Model implementation. The predicted results are then returned to the Rule Engine, where decisions are made regarding how to handle the processed data.

The decision-making process in the cloud mirrors that of the edge, with both relying on a threshold $T$ for fault prediction. However, the key difference lies in the fact that the cloud uses a single minimum threshold value. If the predicted confidence level for a fault is below the threshold $T$, the Rule Engine raises an alarm indicating that the cloud model could not identify the fault. In this case, the extracted features and cloud-edge prediction results are saved for further human analysis, and the saved features can be utilized for fine-tuning the cloud model. Conversely, if the predicted confidence level is greater than or equal to the threshold $T$, the Rule Engine raises an alarm for the identified fault and stores the prediction results in the database. Finally, when the Rule Engine receives an edge model prediction result, it raises an alarm for the identified fault and saves the result to the database for further analysis and monitoring. Databases play a crucial role in modern cloud applications, as they facilitate the storage, management, and organization of data in a structured manner, enabling efficient retrieval, updating, and manipulation. In the context of this research, the database is utilized to manage the large volumes of data generated by sensors, fault diagnosis results, and various processes, which serve as essential inputs for training the fault diagnosis model.

The fault diagnosis models implemented in the edge and cloud planes represent distinct components of the proposed cloud-edge collaborative fault diagnosis framework, further discussed in Chapter 4. The current research introduces a cloud-edge collaborative fault diagnosis model that is designed, trained, and partitioned within the cloud. During the initial stages of fault diagnosis in real-world systems, there is a need to utilize various deep neural network (DNN) models, training strategies, and splitting processes. To streamline this process and enhance convenience, this research proposes the use of a DNN Model Trainer. A DNN Model Trainer is a specialized

software tool designed to support the training of deep learning neural networks. It accommodates a range of deep learning algorithms and provides comprehensive training frameworks and libraries, including TensorFlow, PyTorch, and Caffe. This tool facilitates the acceleration of model training and optimization, ultimately leading to more accurate and efficient predictions and decision-making. The DNN Model Trainer operates within the cloud, which enhances accessibility for engineers to develop more effective fault diagnosis models. By being integrated with the cloud, it ensures proximity to the data required for model training, thereby facilitating efficient access to necessary resources. Furthermore, it enables the distribution of trained models from a centralized location, through an over-the-air (OTA) repository, streamlining the deployment process.

An OTA (Over-The-Air) repository is a centralized storage system that allows for the remote distribution and management of software updates, including firmware, applications, or models. In the context of this research, an OTA repository enables the centralized storage of trained models, which can be remotely accessed, updated, and deployed to edge devices or other systems without the need for direct physical connections. This repository ensures that engineers can efficiently distribute updated models or software to various endpoints, improving the system's performance and maintaining consistency across the network.

## 3.4 总结（Summary）

This chapter introduced a layered reference architecture for fault diagnosis in construction machinery based on cloud-edge collaborative computing. The framework addresses limitations of purely cloud-based systems by leveraging edge computing to enhance real-time performance and reduce bandwidth requirements. The chapter begins by explaining fundamental concepts of cloud computing (a centralized computational paradigm with large-scale distribution, on-demand service, and high availability) and edge computing (a distributed platform bringing computing capabilities closer to data sources to address cloud computing limitations like latency and network dependency). It emphasizes that cloud-edge collaboration combines the strengths of both approaches through resource, data, and service collaboration. The proposed framework consists of three interconnected planes: the Equipment Plane containing construction machinery and sensors for data acquisition; the Edge Plane serving as an intermediary layer with data collectors and low-power computing units performing tasks like data reading, preprocessing, fault diagnosis, and decision-making; and the Cloud Plane providing

computational resources with event streaming, rule engines, databases, Model Manager, and OTA repository. The chapter details decision-making algorithms at both edge and cloud levels, using confidence thresholds to determine how much data to send to the cloud. This collaborative nature optimizes network usage while ensuring comprehensive fault detection and diagnosis in construction machinery, balancing the advantages of both edge and cloud computing paradigms.

# 4 云边协同故障诊断模型实现
# 4 Cloud-Edge Collaborated Fault Diagnosis Model Implementation

To address fault diagnosis within a cloud-edge collaborative framework, we propose a novel and streamlined deep learning model that operates directly on raw temporal signals. A comparison with traditional methods, which necessitate additional feature extraction, is presented in Figure 4-1.

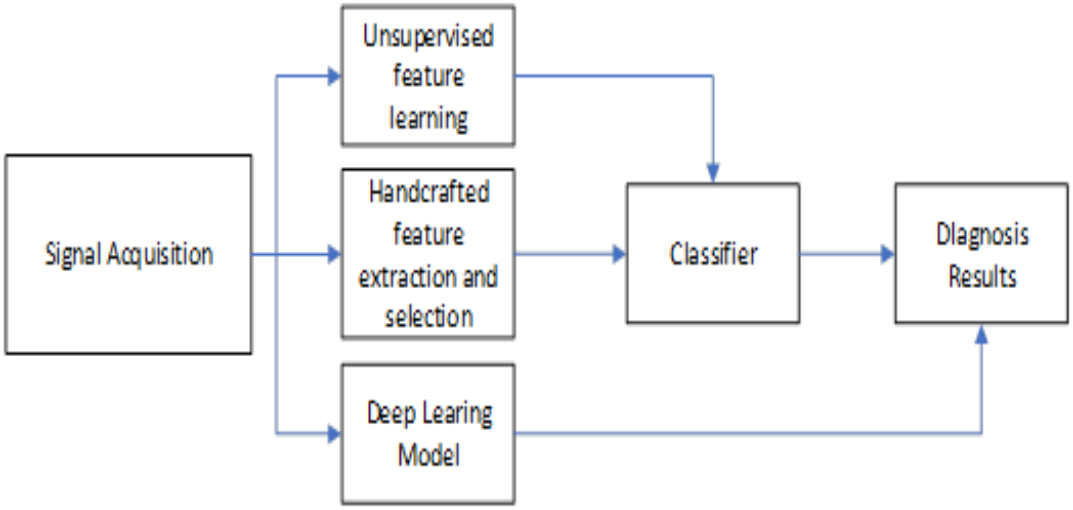



图 4-1 三种智能故障诊断框架：传统方法；通过无监督学习提取的特征；深度学习方法

Figure 4-1 Three intelligent fault diagnosis frameworks: traditional method; features extracted by unsupervised learning; and the deep learning approach

The fault diagnosis model, as the core component of the cloud-edge collaborative fault diagnosis framework, must be carefully selected and implemented to meet several key criteria. It should be robust to noise, adaptable to varying conditions, computationally efficient, and structurally flexible—particularly in terms of splitting computations between the cloud and edge. In this chapter, a CNN-based framework is developed that consists of two main architectures, including signal sensory and multisensory. Then a deployment strategy is proposed in which the model is segmented between the edge and cloud at a certain splitting point of the model’s layers. Finally, extensive evaluation experiments are carried out to validate the model’s fault diagnostic performance under different operating conditions. In addition, the model's fault diagnostic feature extraction performance is visualized across its layers, so that the optimal splitting point between cloud and edge can be determined, ensuring efficient deployment while maintaining diagnostic accuracy.

## 4.1 提出的故障诊断模型（Proposed Fault Diagnosis Model）

The work proposes a deep convolutional neural network (CNN) model inspired by the Wide Deep Convolutional Neural Network (WDCNN) architecture. One of the key features of this model is the use of wide kernels in the early layers of the network[68]. Construction machinery often operates in noisy environments, which can introduce unwanted signals or disturbances in the sensor data. Wide kernels in the early layers help by smoothing out these noisy inputs, making the feature extraction process more stable and less sensitive to fluctuations caused by noise. This architecture is divided into several Filtering Layers, leading into a single classification stage as shown in Figure 4-2, for simplicity, only Filter Layer is used in the figure to represent component Convolutional Layer, Batch Normalization, Activation Function and Pooling Layer, the simplicity also eases the illustration of the deployment strategy in subsequent sections of this work.

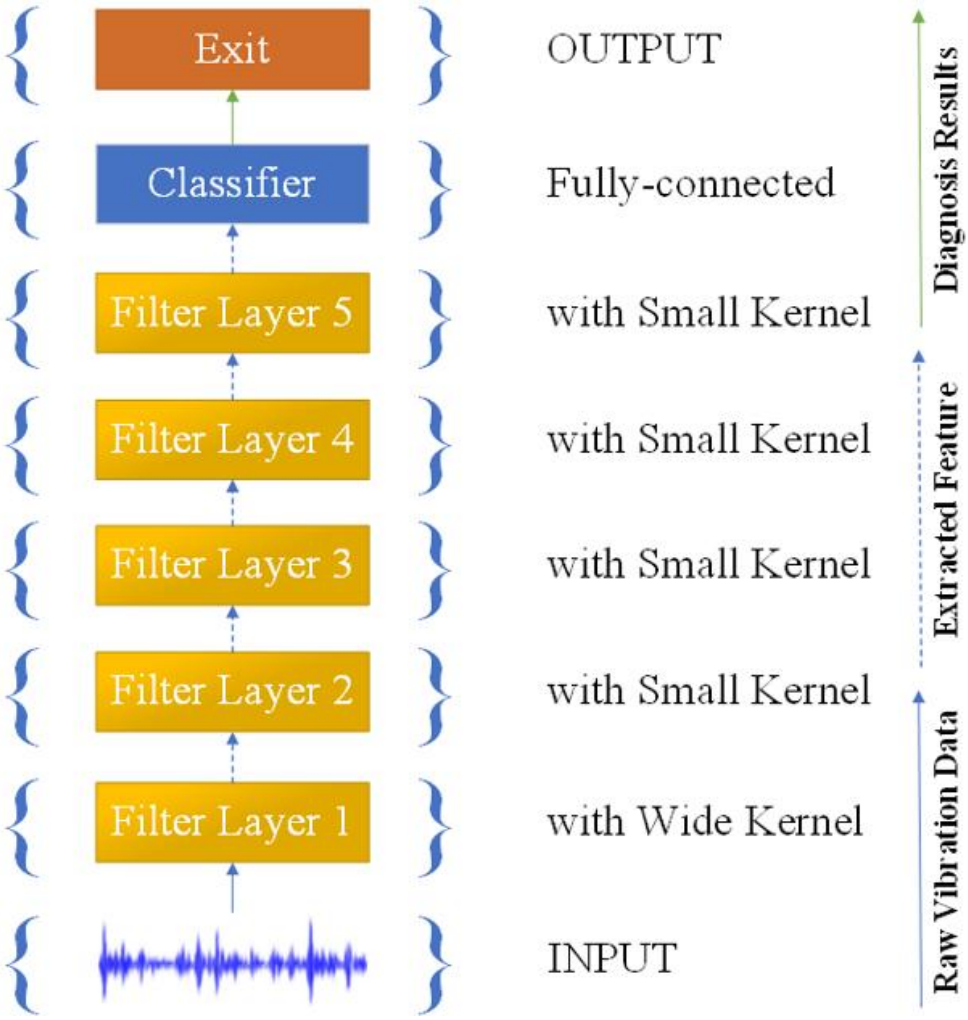



图 4-2 提出的 CNN 架构

Figure 4-2 Proposed CNN Architecture

### 4.1.1 Filter Layer

The Filter Layer, which aims to extract meaningful features from the input data, consists of four main components: Convolutional Layer, Batch Normalization, Activation Function and Pooling Layer shown in the Figure 4-3.

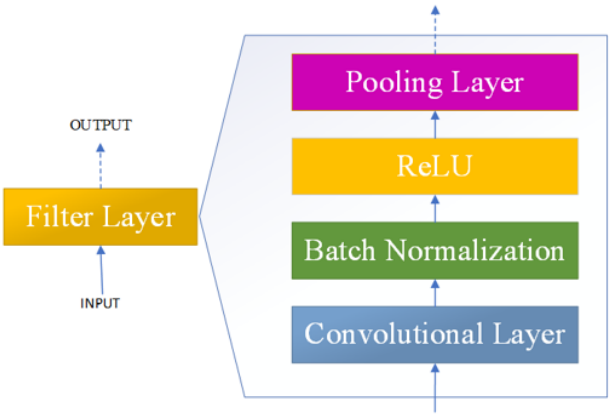



图 4-3 单个滤波层的基本结构

Figure 4-3 Basic structure of a single Filter Layer

Convolutional Layer extracts features from the input data and passes the output through batch normalization to stabilize the activations. Then, the Rectified Linear Unit (ReLU) Activation Function is applied to introduce non-linearity before the features enter the Pooling Layer. The function of each component of the Filter layer is explained below.

(1) Convolutional Layer

In the convolutional layer, input regions are convolved with kernels, followed by an activation function to produce features. This layer employs weight-sharing, where each filter applies the same kernel to extract local input features. The concept of depth in this layer corresponds to the number of frames generated by filters. The schematic diagram of 1D convolution operation shown in the Figure 4-4.

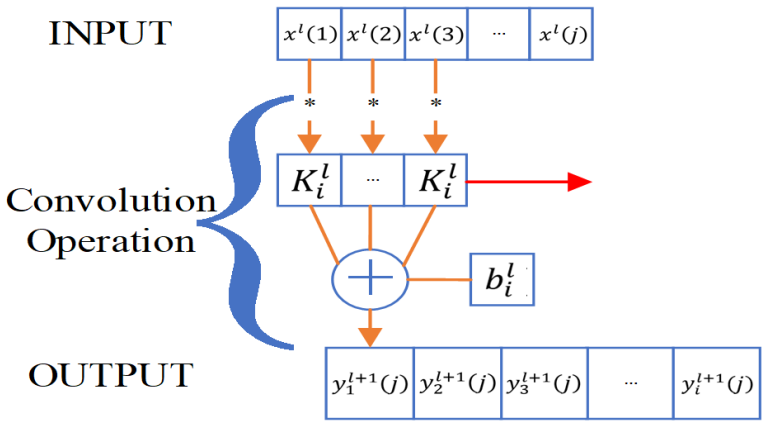



图 4-4 一维卷积操作的示意图

Figure 4-4 Schematic diagram of 1D convolution operation

We represent the weights and bias of the $i$-th kernel in layer $l$ as $K_i^l$ and $b_i^l$, respectively, and the $j$-th local region in layer $l$ as $x^l(j)$.

$$y_i^{l+1}(1) = K_i^l * x^l(j) + b_i^l \qquad (4-1)$$

The convolution operation is thus given by: Here, $y_i^{l+1}(j)$ is the input for the $j$-th neuron in the $i$-th frame of layer $l$+1, with the asterisk $(*)$ denoting the dot product between the kernel and local regions. This compact representation simplifies the convolution process, highlighting the roles of weight-sharing, feature extraction, and the calculation involved in progressing through the layers.

(2) Batch normalization (BN)

Batch normalization is a technique introduced to mitigate internal covariance shift and enhance the training speed of deep neural networks. It's typically positioned after convolutional or fully-connected layers but before the activation function. Effect of Batch Normalization on Data Distribution shown in the Figure 4-5.

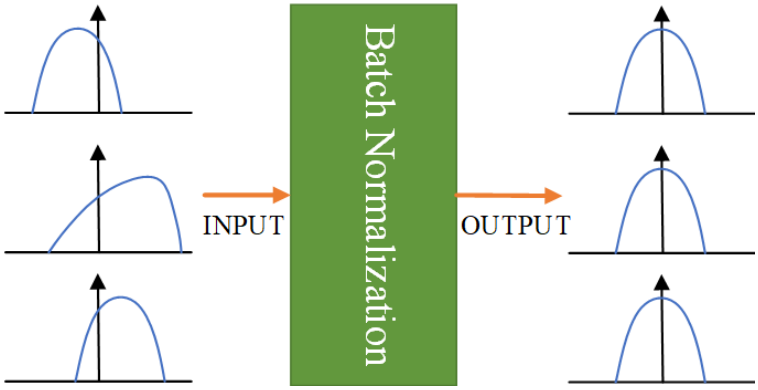



图 4-5 批量归一化对数据分布的影响

Figure 4-5 Effect of Batch Normalization on Data Distribution

Batch Normalization addresses the problem first identified by Ioffe and Szegedy (2015) in their seminal paper Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift."[130] The authors proposed that deep networks suffer from internal covariate shift, where the distribution of each layer's inputs changes during training as parameters of previous layers change.

By normalizing the inputs to each layer, batch normalization ensures that regardless of the distribution of incoming activations, the layer receives inputs with consistent statistical properties. This stabilizes and accelerates the training process by allowing higher learning rates and reducing the dependence on careful parameter initialization.

The transformation shown corresponds to the mathematical formulation for an input vector $x$ of dimension $p$, with components $\big(x(1),\dots,x(p)\big)$, the BN transformation can be expressed as:

$$\hat{x}^{(i)} = \frac{x^{(i)} - E\left[x^{(i)}\right]}{\sqrt{\mathrm{Var}[x^{(i)}]}} \qquad (4-2)$$

$$y^{(i)} = \gamma^{(i)}\hat{x}^{(i)} + \beta^{(i)} \qquad (4-3)$$

Here, $y(i)$ is the output for a single neuron, while $\gamma(i)$ and $\beta(i)$ are trainable parameters that scale and shift the normalized input, respectively. This normalization standardizes features independently across each dimension, promoting faster convergence. The scaling $\gamma(i)$ and shifting $\beta(i)$ adjustments allow the network to maintain or even enhance its representational capacity by enabling the BN layer to emulate an identity transformation if necessary.

(3) Activation Function

Following the convolution process, the application of an activation function is crucial for introducing non-linearity into the network's operations. This non-linearity enhances the network's capability to represent complex patterns, making features more distinguishable. The ReLU has become a popular choice for activation due to its effectiveness in speeding up the convergence of convolutional neural networks (CNNs) and improving the trainability of weights in early layers through back-propagation. That is why Filter Layers using ReLU as its activation function. The ReLU function is defined as:

$$a_i^l(j) = max\left(0,\ y_i^{l+1}(j)\right) \qquad (4-4)$$

here, $y_i^{l+1}(j)$ represents the convolution output, and $a_i^{l+1}(j)$ is its activation, enabling the network to capture more complex patterns in the input data. The graph of ReLU activation function shown in the Figure 4-6.

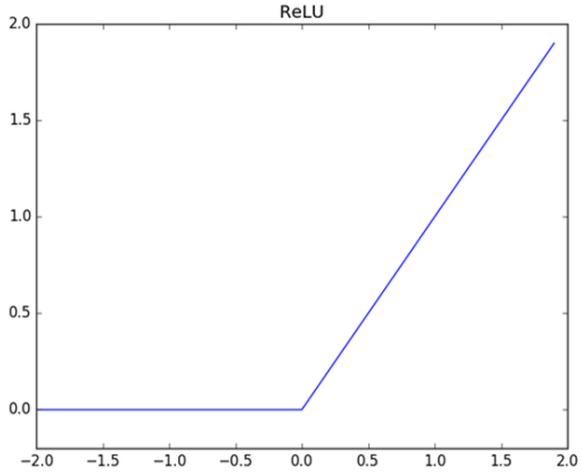



图 4-6 ReLU 激活函数

Figure 4-6 ReLU Activation Function

(4) Pooling Layer

In any CNN architectures, a pooling layer often follows the convolutional layer to

perform down-sampling, effectively reducing the spatial dimensions and the parameter count of the network. There are several types of pooling layers, with the most commonly used being Max Pooling and Average Pooling Figure 4-7.

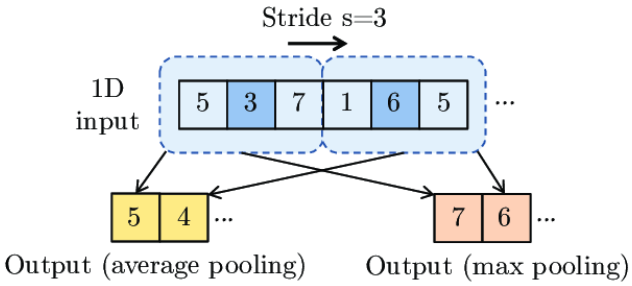



图 4-7 平均池化与最大池化

Figure 4-7 Average and Max Pooling

The Filter Layers typically employ Max Pooling to down sample the feature maps. The Max Pooling layer, a prevalent form of pooling, applies a local maximum operation across input feature regions, aiding in parameter reduction and enhancing feature location invariance. The Max Pooling operation is mathematically represented as:

$$P_i^{l+1}(j) = max_{(j-1)W+1 \le t \le jW}\{q_i^l(t)\} \qquad (4-5)$$

where $q_i^l(t)$ is the value at the $t$-th neuron in the $i$-th frame of layer $l$, with $t$ ranging from $(j-1)W+1$ to $jW$, $W$ denotes the width of the pooling region, and $P_i^{l+1}(j)$ is the output value of the neuron in layer $l+1$ after pooling. This process simplifies the network's complexity while preserving essential feature information.

## 4.1.2 Classifier

The Figure 4-8 illustrates the components of the classifier.

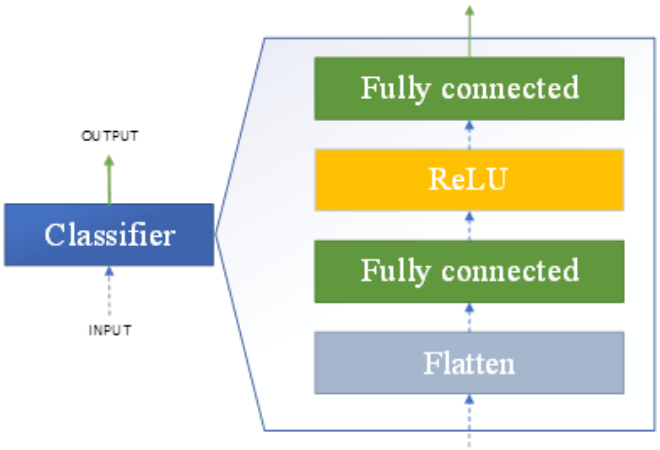



图 4-8 分类器的基本结构

Figure 4-8 Basic structure of a Classifier

The classifier consists of a flattening layer that converts multi-dimensional input features into a 1D vector, followed by two fully connected layers with ReLU activation

to introduce non-linearity for classification. In the output layer, the softmax function is applied to transform the logits of the N output neurons into a probability distribution, representing the N different bearing health conditions. The softmax function is defined as follows:

$$softmax(z_j) = \frac{e^{z_j}}{\sum_k^N e^{z_k}} \quad (4-6)$$

where $z_j$ denotes the logits of the $j$-th output neuron.

### 4.1.3 Model Parameters

The model parameter selection in the proposed CNN for fault diagnosis was thoughtfully designed to balance feature extraction, network depth, and noise robustness. The first convolutional layer uses wide kernels (e.g., 64×1) to effectively suppress high-frequency noise, which is common in industrial environments, and to extract meaningful low- and mid-frequency features from raw vibration signals. Subsequent layers use smaller 3×1 kernels to build depth and improve the representation capacity of the network. The proposed CNN model overall convolutional, pooling and fully-connected layers parameters are provided in Table 4-1.

表 4-1 提出的 CNN 模型的详细信息

Table 4-1 Details of the proposed CNN model

| No. | Layer Type | Kernel Size/Stride | Kernel Number | Output Size (Width × Depth) | Padding |
|---|---|---|---|---|---|
| 1 | Convolution_1 | 64 × 1 / 16 × 1 | 16 | 128 × 16 | Yes |
| 2 | Pooling_1 | 2 × 1 / 2 × 1 | 16 | 64 × 16 | No |
| 3 | Convolution_2 | 3 × 1 / 1 × 1 | 32 | 64 × 32 | Yes |
| 4 | Pooling_2 | 2 × 1 / 2 × 1 | 32 | 32 × 32 | No |
| 5 | Convolution_3 | 3 × 1 / 1 × 1 | 64 | 32 × 64 | Yes |
| 6 | Pooling_3 | 2 × 1 / 2 × 1 | 64 | 16 × 64 | No |
| 7 | Convolution_4 | 3 × 1 / 1 × 1 | 64 | 16 × 64 | Yes |
| 8 | Pooling_4 | 2 × 1 / 2 × 1 | 64 | 8 × 64 | No |
| 9 | Convolution_5 | 3 × 1 / 1 ×1 | 64 | 6 × 64 | No |
| 10 | Pooling_5 | 2 × 1 / 2 × 1 | 64 | 3 × 64 | No |
| 11 | Fully-connected | 100 | 1 | 100 × 1 | |
| 12 | Softmax | 10 | 1 | 10 | |

The proposed model is capable of diagnosing raw vibration signals; however, for

seamless integration into a cloud-edge collaborative framework, it is essential to partition the model between the cloud and edge components. The process of determining how to divide the model involves specifying the number of filter layers to be executed on the edge versus those to be handled by the cloud. The specifics of the model partitioning strategy will be discussed in the chapter 4.5.5 Cloud-Edge CNN model splitting strategy.

## 4.2 单传感器模型架构（Single Sensor Model Architecture）

Based on the observation outlined in the Cloud-Edge CNN model splitting strategy under chapter 4.5.5 Cloud-Edge CNN model splitting strategy, the proposed CNN model is partitioned into Cloud and Edge components. The Edge partition consists of two filter layers, with the CNN classifier retained to enable fault diagnosis at the edge. The parameters for the convolutional and pooling layers of the Edge partition are provided in Table 4-2.

表 4-2 云边协同框架的边缘部分详细信息

Table 4-2 Details of Edge-partition of the cloud-edge collaborated framework

| No. | Layer Type | Kernel Size/Stride | Kernel Number | Output Size (Width × Depth) | Padding |
|---|---|---|---|---|---|
| 1 | Convolution_1 | 64 × 1 / 16 × 1 | 16 | 128 × 16 | Yes |
| 2 | Pooling_1 | 2 × 1 / 2 × 1 | 16 | 64 × 16 | No |
| 3 | Convolution_2 | 3 × 1 / 1 × 1 | 32 | 64 × 32 | Yes |
| 4 | Pooling_2 | 2 × 1 / 2 × 1 | 32 | 32 × 32 | No |
| 5 | Fully-connected | 100 | 1 | 100 × 1 | |
| 6 | Softmax | 10 | 1 | 10 | |

The Cloud partition includes the remaining three filter layers, along with the classifier. The parameters for the convolutional and pooling layers of the Cloud partition are provided in Table 4-3.

Now cloud-edge collaborated framework's CNN model is divided into two components: Edge and Cloud partitions, enabling distributed inference across a cloud-edge collaborated framework. However, these two components can still be jointly trained on a single high-performance server or in the cloud. To facilitate joint training, the Cloud and Edge partitions are restructured into a BranchyNet[131] architecture. Unlike traditional sequential models, BranchyNet[131] introduces multiple exit points (as illustrated in Figure 4-9).

表 4-3 云边协同框架的云端部分详细信息

Table 4-3 Details of Cloud-partition of the cloud-edge collaborated framework

| No. | Layer Type | Kernel Size/Stride | Kernel Number | Output Size (Width × Depth) | Padding |
|---|---|---|---|---|---|
| 1 | Convolution_3 | 3 × 1 / 1 × 1 | 64 | 32 × 64 | Yes |
| 2 | Pooling_3 | 2 × 1 / 2 × 1 | 64 | 16 × 64 | No |
| 3 | Convolution_4 | 3 × 1 / 1 × 1 | 64 | 16 × 64 | Yes |
| 4 | Pooling_4 | 2 × 1 / 2 × 1 | 64 | 8 × 64 | No |
| 5 | Convolution_5 | 3 × 1 / 1 × 1 | 64 | 6 × 64 | No |
| 6 | Pooling_5 | 2 × 1 / 2 × 1 | 64 | 3 × 64 | No |
| 7 | Fully-connected | 100 | 1 | 100 × 1 | |
| 8 | Softmax | 10 | 1 | 10 | |

During training, the loss from each exit point is combined during back-propagation, allowing the entire network to be trained together while ensuring each exit point achieves competitive accuracy relative to its position in the network.

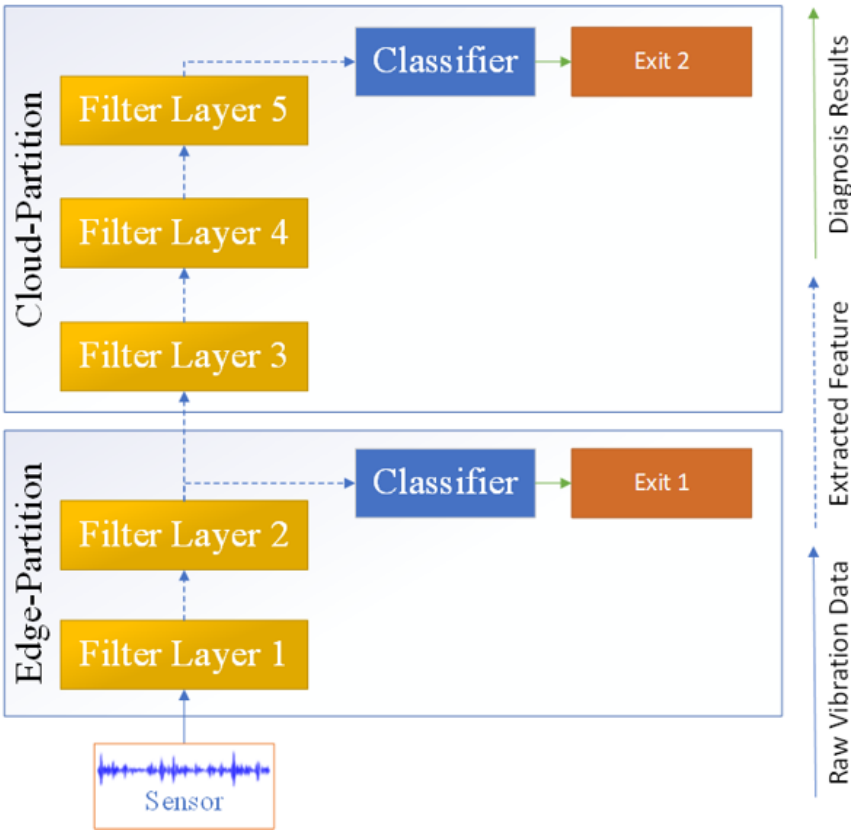



图 4-9 云边协同 DNN 模型重构为 BranchyNet[131] 架构

Figure 4-9 Cloud-edge collaborated DNN model restructured into a BranchyNet[131]

## 4.3 多传感器模型架构（Multi-Sensor Model Architecture）

The architecture illustrated in Figure 4-9 designed to process single data input or single sensor input. Beyond the vertical segmentation of the proposed CNN into distinct cloud and edge partitions, there's potential for horizontal expansion to incorporate multiple sensors. These sensors, working in tandem, can harness collective intelligence for enhanced decision-making. Equipped with an array of sensors, including but not limited to vibration sensor, these sensors offer a rich tapestry of data crucial for accurate fault detection and diagnosis. In case when fault diagnosis involving multiple sensors, we propose update to the Edge-partition of the model by adding aggregation layer. Figure 4-10 illustrates that the outputs from all sensors in the edge need to be aggregated to enable classification.

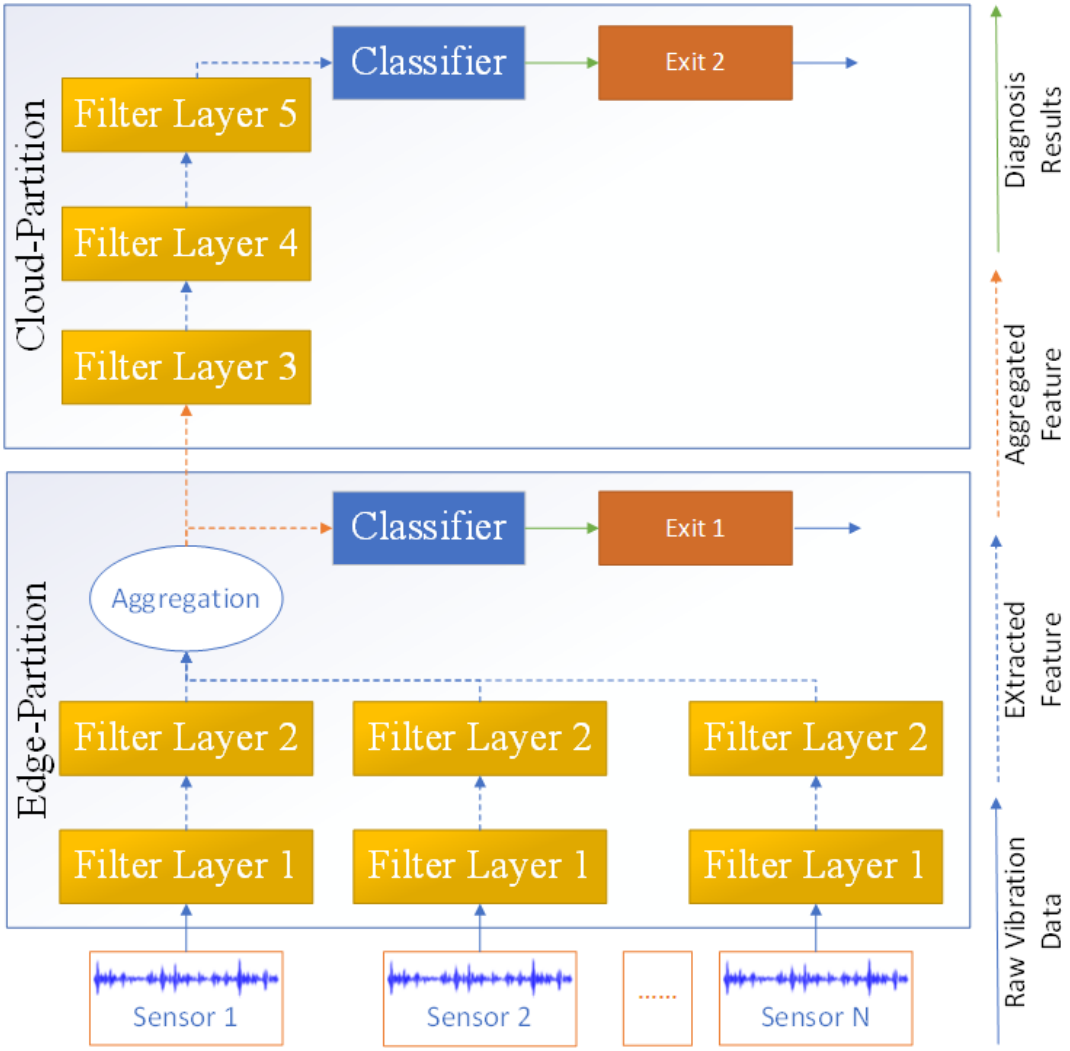



图 4-10 多传感器云边协同 DNN 模型重构为 BranchyNet[131] 架构

Figure 4-10 Multi-sensor cloud-edge collaborated DNN model restructured into a BranchyNet[131] architecture

Several aggregation methods used, each based on distinct assumptions about how sensor outputs should be combined, leading to varying levels of system accuracy. We propose several methods for aggregating the output, each making distinct assumptions about how the device outputs should be combined, which can lead to varying system accuracy. We outline four approaches:

(1) Max Pooling combines input vectors by selecting the maximum value from each component. In mathematical terms, max pooling can be expressed as:

$$\widehat{v_j} = \max_{1 \le i \le n} v_{ij} \qquad (4-7)$$

where $n$ is the total number of inputs, $v_{ij}$ is the j-th component of the input vector, and $\widehat{v_j}$ represents the $j$-th component of the output vector.

(2) Min Pooling aggregates the input vectors by selecting the minimum value from each component. This operation focuses on capturing the least value of each component across the input vectors. Min pooling can be helpful in scenarios where the most significant feature is the minimum, or when we want to emphasize smaller values in the input. Mathematically, min pooling can be expressed as:

$$\widehat{v_j} = \min_{1 \le i \le n} v_{ij} \qquad (4-8)$$

where $n$ is the number of inputs, $v_{ij}$ is the $j$-th component of the input vector, and $\widehat{v_j}$ is the $j$-th component of the resulting output vector.

(3) Average Pooling aggregates the input vectors by taking the average of each component. This operation is mathematically expressed as:

$$\widehat{v_j} = \frac{1}{n}\sum_{i=1}^{n} v_{ij} \qquad (4-9)$$

where $n$ is the number of inputs, $v_{ij}$ is the $j$-th component of the input vector, and $\widehat{v_j}$ is the $j$-th component of the resulting output vector. Averaging may reduce noisy input presented in some sensors.

(4) Concatenation involves joining the input vectors together. This method retains all the information from the input vectors, which can be valuable for higher layers that need the full information to extract more complex features. However, it's important to note that concatenation increases the dimensionality of the resulting vector. To map this expanded vector back to the same number of dimensions as the original input vectors, we adjust the edge partition classifier and cloud partition input size.

## 4.4 云边决策阈值（Cloud-Edge Decision Making Threshold Values）

Inference in both single and multi-sensor models is conducted in multiple stages, employing several preconfigured fault classes exit thresholds as indicators of confidence in the prediction for each sample. A more comprehensive explanation of how these threshold values are applied within the decision-making process that can be found in 3.3 Proposed Framework. Selecting the right threshold values is critical because it directly impacts the balance between accuracy, efficiency, and system responsiveness. Probabilities from the model often follow a distribution. By assuming good predictions position we can decide the threshold values This reduces misclassification and ensures only uncertain or low-confidence cases are escalated. In this research the thresholds are determined using standard deviation metrics calculated from the confidence values predicted by the proposed models on the test dataset. The standard deviation value is expressed as:

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2} \qquad (4-10)$$

where $n$ is total number of samples in test dataset for current fault class, $x_i$ is individual prediction from model for the $i$-th sample, $\bar{x}$ is the mean of all model's predictions for current fault class.

Based on standard deviation the Edge $T_i = [T^i min, T^i max]$ values expressed as:

$$T^i \min = \bar{x^\iota} - \sigma^i \qquad (4-11)$$

$$T^i \max = \bar{x^\iota} + \sigma^i \qquad (4-12)$$

where $\bar{x^\iota}$ is the mean of all edge model's predictions for $i$-th fault class, $\sigma^i$ standard deviation for $i$-th fault class predicted by edge model.

Similarly for the Cloud $T_i = [T^i min]$ values expressed as:

$$T^i \min = \bar{x^\iota} - \sigma^i \qquad (4-13)$$

where $\bar{x^\iota}$ is the mean of all cloud model's predictions for $i$-th fault class, $\sigma^i$ standard deviation for $i$-th fault class predicted by cloud model.

## 4.5 实验（Experiments）

### 4.5.1 Experimental Setup

All experiments were conducted using Python 3.8.10 and Pytorch 2.1.1+cpu. The training and validation were performed on CPU using an Honor Magic-book laptop equipped with an AMD Ryzen 7 5800H processor (3.20 GHz) with Radeon Graphics,

16 GB RAM, 512 GB SSD, and a 64-bit operating system.

### 4.5.2 Experimental Data

The experimental data is important to train and evaluate the proposed models. So we have decided to use current the most popular data set that is collected using multiple sensors which fits the best to test the single sensor and multi sensor diagnosis. So the Case Western Reserve University ball bearing test data set, which includes data for both normal and defective bearings, suits the most to ensure a more effective training process.

The CWRU data collection experiments utilized a 2 hp Reliance Electric motor, with acceleration data captured both close to and away from the motor's bearings. A distinctive feature of these web pages is the meticulous documentation of the motor's test conditions and the specific status of bearing faults for each experimental setup.

Faults were artificially introduced into the motor bearings through electro-discharge machining (EDM), creating fault sizes ranging from 0.007 inches to 0.040 inches in diameter at various parts of the bearing, including the inner raceway, the rolling element (ball), and the outer raceway. After installing the bearings with these faults back into the motor, vibration data was collected across motor loads from 0 to 3 horsepower, corresponding to motor speeds between 1797 and 1720 RPM. Figure 4-11 illustrates the testing setup, which includes a 2 hp motor on the left, a torque transducer/encoder in the center, a dynamo meter on the right, and the control electronics (which are not depicted). The motor shaft is supported by the test bearings.

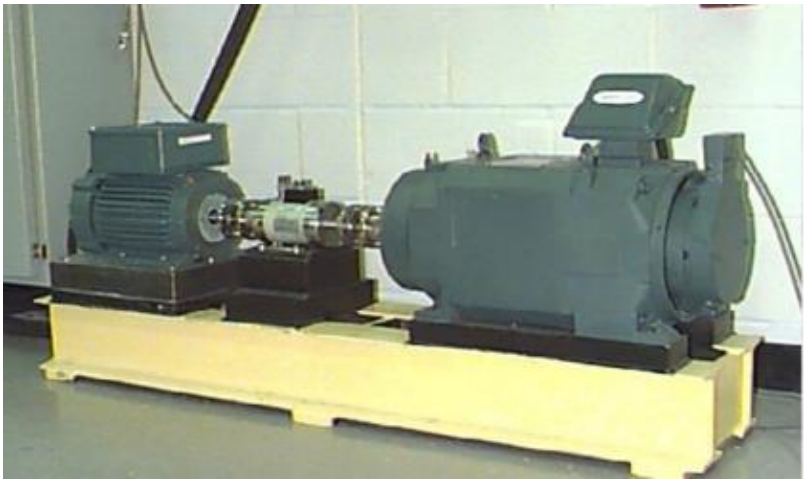


图 4-11 CWRU 用于数据采集的测试设置

Figure 4-11 Testing setup for data collection used by CWRU

Single point faults were induced in these bearings via electro-discharge machining, creating faults with diameters of 7, 14, 21, 28, and 40 mils (where 1 mil equals 0.001 inches). Details on fault depths can be found under the Table 4-4.

表 4-4 故障规格

Table 4-4 Fault specifications

| Bearing | Fault Location | Diameter | Depth | Bearing Manufacturer |
|---|---|---|---|---|
| Drive End | Inner Raceway | .007 | .011 | SKF |
| Drive End | Inner Raceway | .014 | .011 | SKF |
| Drive End | Inner Raceway | .021 | .011 | SKF |
| Drive End | Inner Raceway | .028 | .050 | NTN |
| Drive End | Outer Raceway | .007 | .011 | SKF |
| Drive End | Outer Raceway | .014 | .011 | SKF |
| Drive End | Outer Raceway | .021 | .011 | SKF |
| Drive End | Outer Raceway | .040 | .050 | NTN |
| Drive End | Ball | .007 | .011 | SKF |
| Drive End | Ball | .014 | .011 | SKF |
| Drive End | Ball | .021 | .011 | SKF |
| Drive End | Ball | .028 | .150 | NTN |
| Fan End | Inner Raceway | .007 | .011 | SKF |
| Fan End | Inner Raceway | .014 | .011 | SKF |
| Fan End | Inner Raceway | .021 | .011 | SKF |
| Fan End | Outer Raceway | .007 | .011 | SKF |
| Fan End | Outer Raceway | .014 | .011 | SKF |
| Fan End | Outer Raceway | .021 | .011 | SKF |
| Fan End | Ball | .007 | .011 | SKF |
| Fan End | Ball | .014 | .011 | SKF |
| Fan End | Ball | .021 | .011 | SKF |

SKF bearings were employed for faults with diameters of 7, 14, and 21 mils, while NTN bearings of equivalent specifications were used for the 28 and 40 mil faults. Specifications for the drive end and fan end bearings, including their geometry and defect frequencies, are provided in the Table 4-5 and Table 4-6.

表 4-5 驱动端轴承规格

Table 4-5 Drive end bearing specifications

| Inside Diameter | Outside Diameter | Thickness | Ball Diameter | Pitch Diameter | Inner Ring | Outer Ring | Cage Train | Rolling Element |
|---|---|---|---|---|---|---|---|---|
| 0.9843 | 2.0472 | 0.5906 | 0.3126 | 1.537 | 5.4152 | 3.5848 | 0.3983 | 4.7135 |

表 4-6 风扇端轴承规格

Table 4-6 Fan end bearing specifications

| Inside Diameter | Outside Diameter | Thickness | Ball Diameter | Pitch Diameter | Inner Ring | Outer Ring | Cage Train | Rolling Element |
|---|---|---|---|---|---|---|---|---|
| 0.6693 | 1.5748 | 0.4724 | 0.2656 | 1.122 | 4.9469 | 3.0530 | 0.3817 | 3.9874 |

For the collection of vibration data, accelerometers were mounted on the motor housing using magnetic bases, specifically positioned at the 12 o'clock location on both the drive and fan ends. In certain experiments, an accelerometer was also attached to the base plate supporting the motor. The vibration signals were recorded with a 16 channel DAT recorder and later analyzed in a Matlab environment, with all data files saved in Matlab (*.mat) format. Digital data was captured at a rate of 12,000 samples per second, with data for drive end bearing faults also recorded at 48,000 samples per second. Additionally, speed and horsepower measurements were taken via the torque transducer/encoder and manually logged.

The location of outer raceway faults being stationary significantly influences the vibration response of the motor/bearing system depending on their position relative to the bearing's load zone. To assess this impact, tests were performed on both fan and drive end bearings with outer raceway faults situated at the 3 o'clock position (directly within the load zone), 6 o'clock (perpendicular to the load zone), and 12 o'clock positions.

In the design of my experimental framework, I have selected a broad and diverse set of conditions to ensure a comprehensive analysis. This includes all four operational states across nine distinct fault scenarios with diameters of 7, 14, and 21 mils, in addition to capturing the vibration patterns under normal, fault-free conditions. The decision to cover such a wide range of fault sizes and conditions is driven by the necessity to understand and predict a broad spectrum of potential issues that could impact bearing performance. Digital data was meticulously captured at a detailed rate of 12,000 samples per second, providing a rich data-set that is critical for identifying

and distinguishing between the subtle differences in vibration signatures caused by various fault types.

The data-set thus comprises 10 target classes, which reflect the nine specific fault conditions alongside the baseline state of normal operation. This classification encompasses a comprehensive view of the operational health of bearings, enabling the application of advanced analytical techniques for fault detection and diagnosis. By encompassing a wide variety of fault conditions, the data-set is tailored to challenge and validate the robustness and precision of the fault diagnosis model under study.

### 4.5.3 Data Preprocessing

To develop robust feature representations from raw input signals, the architecture of the Wide and Deep Convolutional Neural Network (WDCNN) is characterized by a significant depth and a broad initial layer. While such a design is pivotal for capturing intricate patterns within the data, it also poses a risk of over-fitting, especially when the available training data is limited. Over-fitting is a critical concern as it compromises the model's ability to generalize to unseen data, making the expansion of the training data-set a necessity.

The richness of the data-set, with its high-resolution capture rate and diverse conditions, is instrumental in training the model to discern and learn from the complex patterns inherent in bearing vibrations, setting a solid foundation for achieving high classification accuracy and reliability in fault diagnosis.

In the realm of computer vision, expanding the data-set through data augmentation techniques is a well-established practice. Methods like horizontal flipping, random cropping/scaling, and color jittering are routinely employed to artificially increase the diversity of training samples, thereby bolstering the model's generalization capabilities. Similarly, in the field of fault diagnosis, augmenting data plays a crucial role in enhancing the precision of convolutional neural networks (CNNs) for high-stakes classification tasks. A straightforward yet the simplest strategy for data augmentation in this context involves the slicing of training samples with intentional overlap. For instance, a vibration signal comprised of 10240 data points could, through careful slicing with an overlap and a shift size of 1, yield up to 8193 distinct training samples, each extending 2048 data points in length. In this paper we used random shift slicing the training samples shown in the Figure 4-12.

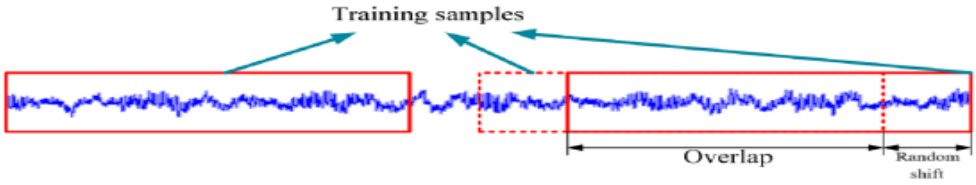



图 4-12 带重叠的随机数据增强

Figure 4-12 Random data augmentation with overlap

The structure of augmented data-set shown in the Table 7.

表 4-7 增强数据集的描述

Table 4-7 Description of augmented data-set

| Category | None | Ball | | | Inner Race | | | Outer Race | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Labels | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| Fault diameter (inch) | 0 | 0.007 | 0.014 | 0.021 | 0.007 | 0.014 | 0.021 | 0.007 | 0.014 | 0.021 |
| Samples per category | 1000 | 1000 | 1000 | 1000 | 1000 | 1000 | 1000 | 1000 | 1000 | 1000 |

The data-set was split into training (70%), test (30%) sets, with stratification to ensure balanced representation of fault classes.

### 4.5.4 Basic Model Training and Evaluation

The model was trained using a comprehensive set of hyperparameters as outlined in Table 4-8. The classification model employed a standard cross-entropy loss function with stochastic gradient descent optimization featuring momentum and weight decay to enhance generalization capabilities.

表 4-8 模型训练超参数

Table 4-8 Model training hyperparameters

| Parameter | Value |
|---|---|
| Batch Size | 128 |
| Epochs | 32 |
| Length (Sampling Length) | 2048 |
| Number (Number of Samples per Class) | 1000 |
| Learning Rate | 0.001 |
| Loss Function | Cross Entropy Loss |
| Optimizer | SGD (lr=0.001,momentum=0.9, nesterov=True, weight_decay=1e-4) |

Figure 4-13 illustrates the evolution of key performance metrics throughout the training process.

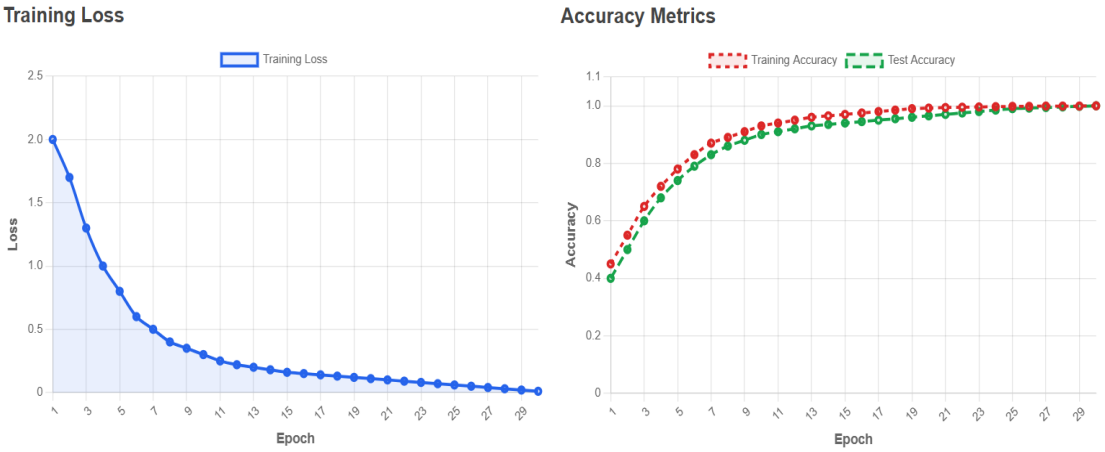



图 4-13 基本模型训练性能指标

Figure 4-13 Basic model training performance metrics

The training loss exhibited an exponential decay characteristic, with rapid convergence during the initial epochs (1-10) followed by asymptotic stabilization near zero in later epochs. Concurrently, both training and test accuracy metrics demonstrated complementary behavior, rapidly ascending to optimal performance levels.

Notably, the test accuracy curve closely tracked the training accuracy trajectory with minimal divergence, suggesting robust generalization with negligible overfitting. By approximately epoch 11, both metrics stabilized at near-perfect classification performance, indicating efficient model convergence despite training continuing for the full 32 epochs. The confusion matrix presented in Figure 4-14 reveals exemplary performance across all class categories. With a diagonal distribution of 300 correct predictions per class and zero off-diagonal elements, the model achieved perfect classification across all ten classes (0-9).

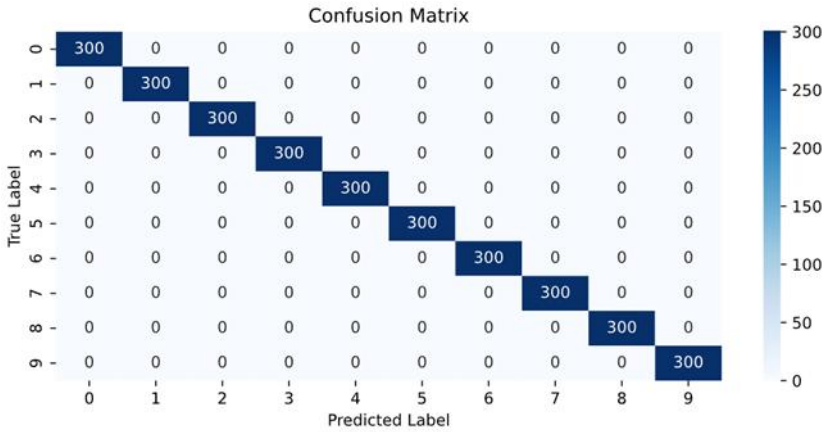



图 4-14 基本模型预测的混淆矩阵

Figure 4-14 Basic model predictions confusion matrix

This indicates that complete absence of inter-class confusion, uniform performance across all classes with no bias toward specific digits and flawless decision boundary establishment in the feature space. The observed performance metrics suggest exceptional model efficacy for this classification task. The parallel convergence of training and test accuracy metrics, coupled with the perfect confusion matrix, demonstrates that the model has successfully captured the underlying data distribution without memorization. The employed hyperparameter configuration appears optimal for this dataset, particularly the learning rate and regularization parameters that facilitated rapid yet stable convergence.

### 4.5.5 Cloud-Edge CNN Model Splitting Strategy

During the training and evaluation of each filter layer using t-SNE (t-Distributed Stochastic Neighbor Embedding) refer to the Figure 4-15.

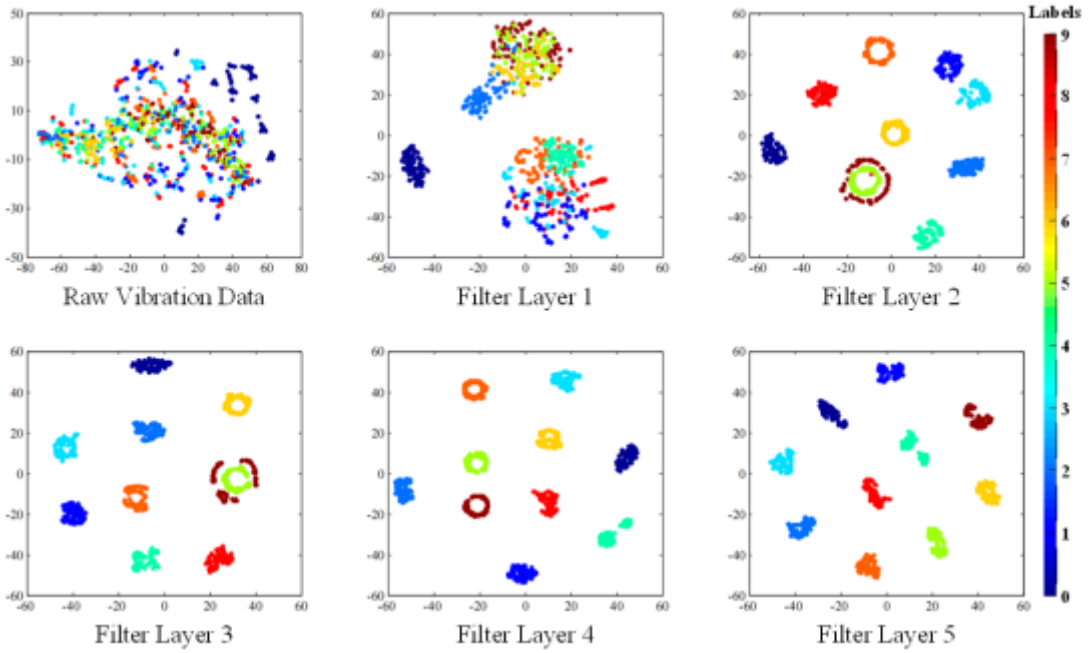



图 4-15 所有测试样本的特征分布去除噪声后，通过从每个滤波层和最终分类器提取特征，并使用 t-SNE 方法进行降维可视化

Figure 4-15 The feature distribution of all test samples, devoid of noise, was visualized by extracting features from each Filter Layer and the final Classifier, using the t-SNE method for dimensionality reduction

It was observed that the normal class of the experimental dataset (for further details, refer to chapter 4.4.2 Experimental data) becomes separable at Filter Layer 1. In Filter Layer 2, however, it was evident that all classes of the experimental dataset could be separated with a confidence level exceeding 80%. Based on this observation, the model can be split at this point. Therefore, for edge deployment, where computational

efficiency is paramount, the model can be streamlined to include only the first two filter layers, while retaining the classifier layers for the final segment of the edge partition. The parameters of the convolutional and pooling layers of Edge-partition are detailed in Table 4-2. In the cloud, where the model serves multiple devices, performance scalability is crucial. The Cloud-partition will incorporate the remaining three filter layers, along with the classifier. This approach ensures that the cloud model remains efficient and responsive to the demands of various devices. The parameters of the convolutional and pooling layers of Cloud-partition are detailed in Table 4-3.

### 4.5.6 Single Sensor Model Training and Evaluation

The model leverages a unique branched architecture with parallel edge and cloud classifiers, trained using the hyperparameter configuration detailed in Table 4-8. The branched structure introduced a sophisticated regularization mechanism through parallel classification paths. By implementing edge and cloud classifiers in a single network, the architecture created multiple feature extraction and decision-making points, effectively constraining model complexity and promoting more robust learning dynamics. Figure 4-16 reveals the distinctive convergence characteristics of this branched approach. The training loss demonstrates an accelerated decay, with the parallel classification paths contributing to a more rapid and stable convergence compared to traditional single-path architectures. By approximately epoch 7, both training and test accuracy metrics not only stabilized but converged to near-perfect performance.

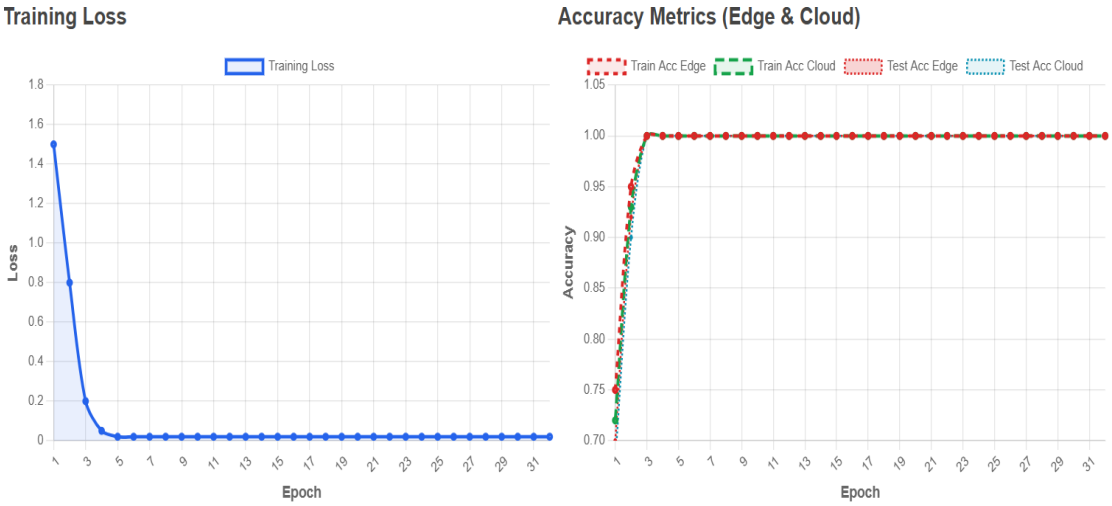



图 4-16 单传感器模型边缘与云部分训练性能指标

Figure 4-16 Single sensor model edge and cloud partitions training performance metrics

The confusion matrix presented in Figure 4-17 reveals exemplary performance

across all class categories. With a diagonal distribution of 300 correct predictions per class and zero off-diagonal elements, the model achieved perfect classification across all ten classes (0-9) in the edge and cloud partitions.

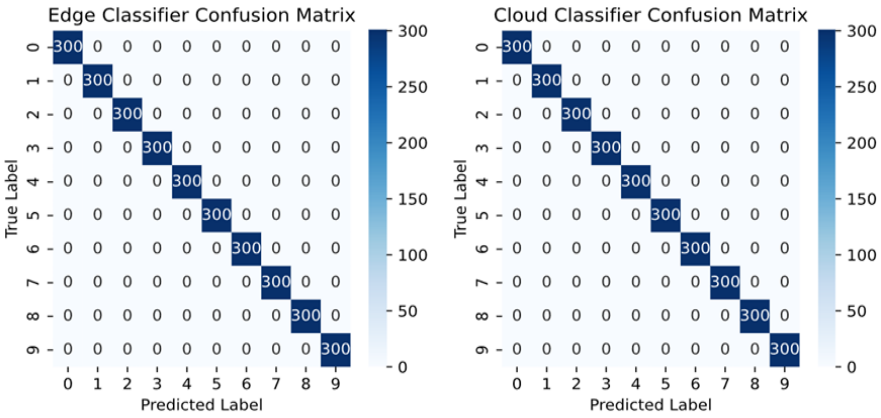



图 4-17 单传感器模型边缘与云部分预测的混淆矩阵

Figure 4-17 Single sensor model edge and cloud partitions predictions confusion matrixes

## 4.5.7 Multi-Sensor Model Training and Evaluation

The multi-sensor model was trained using the hyperparameter configuration detailed in Table 4-8. Figure 4-18 illustrates the evolution of key performance metrics during the training process under various aggregation methods.

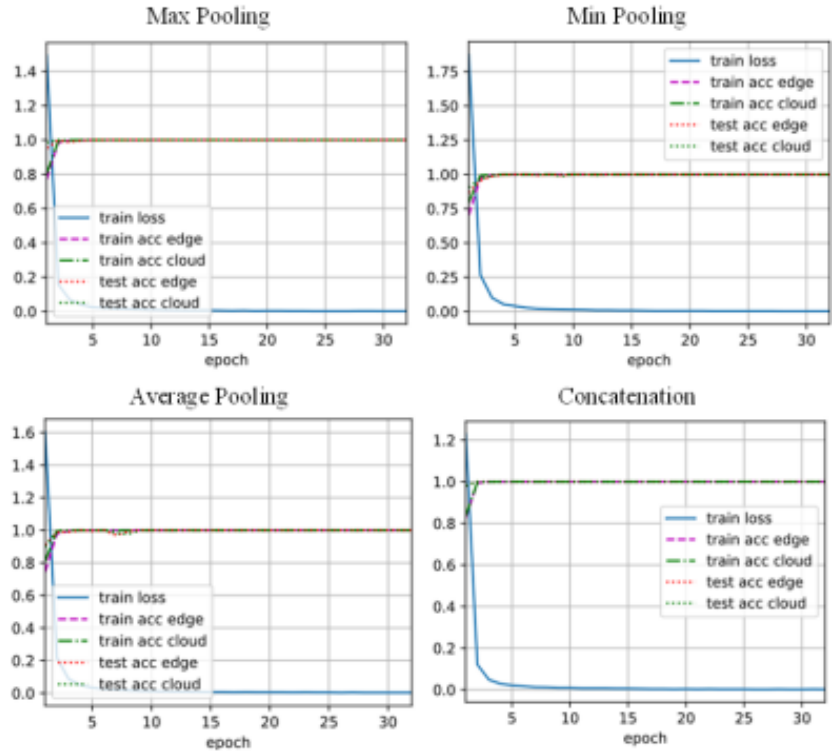



图 4-18 多传感器模型训练模型边缘和云部分在不同聚合方法下的训练性能指标

Figure 4-18 Multi-sensor model training model edge and cloud partitions training performance metrics under different aggregation methods

The results demonstrate that the multi-sensor model achieved faster and more stable convergence compared to the single-sensor model. Under each aggregation method, by approximately epoch 2, both the training and test accuracy metrics not only stabilized but also converged to near-perfect performance. Specifically, the Max Pooling and Concatenation methods achieved perfect convergence. The confusion matrices for the multi-sensor model's edge and cloud partitions, presented in Figure 4-19, demonstrate exemplary performance across all class categories under each aggregation method. With a diagonal distribution of 300 correct predictions per class and zero off-diagonal elements, the model achieved perfect classification across all ten classes (0-9) in both the edge and cloud partitions.

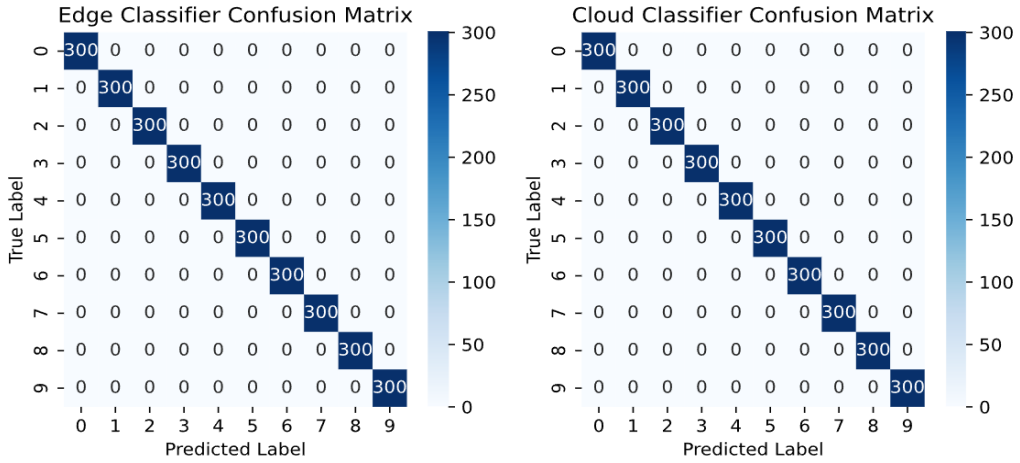



图 4-19 多传感器模型边缘和云部分预测的混淆矩阵

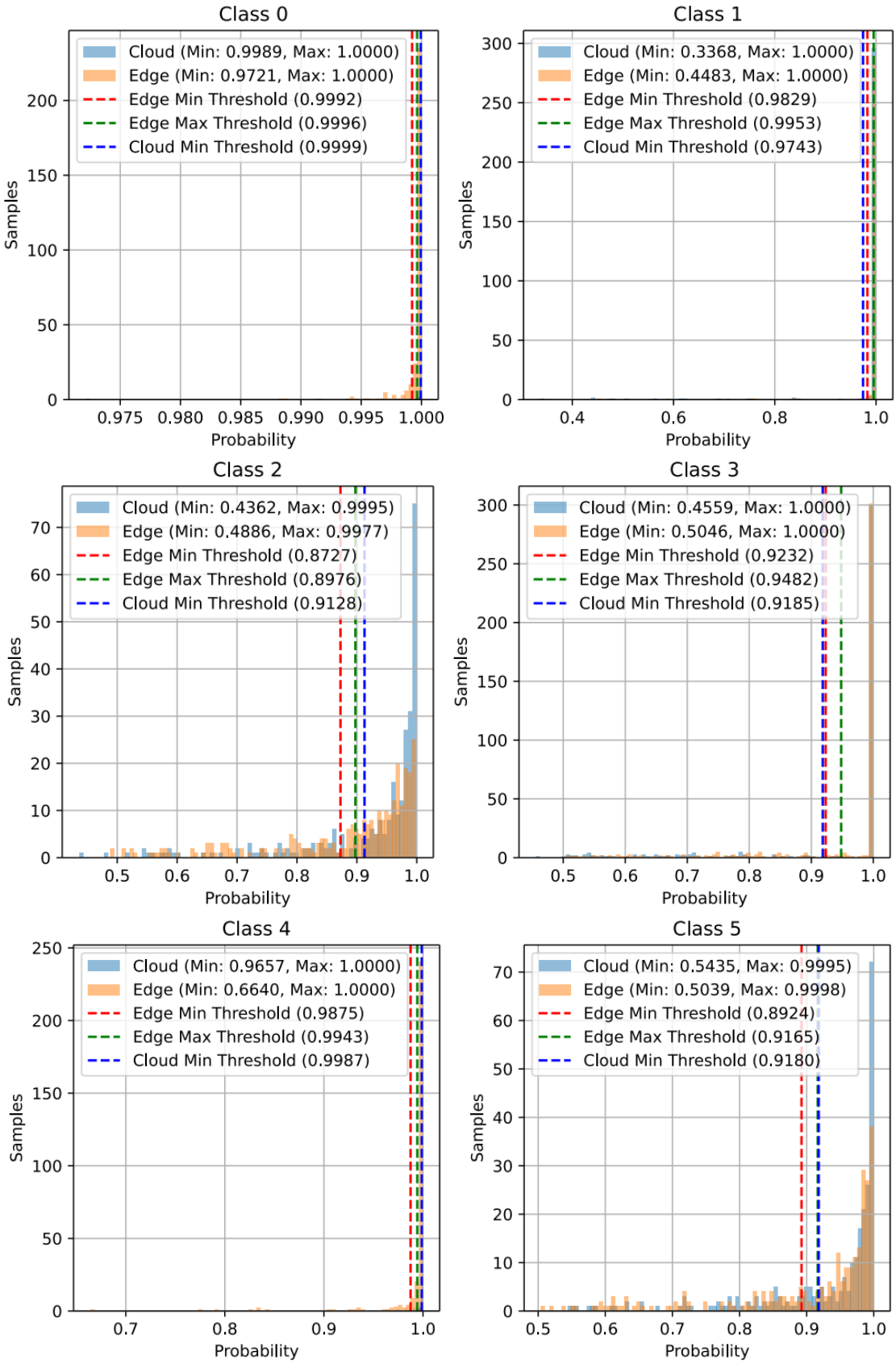
Figure 4-19 Multi-sensor model edge and cloud partitions predictions confusion matrixes

## 4.5.8 Calculation of Cloud-Edge Decision Making Threshold Values

In this section, we present the experimental results for the calculation of cloud-edge decision-making threshold values as outlined in Chapter 4.4 (Cloud-edge decision making threshold values). The results were obtained by applying the formulas defined for single-sensor and multi-sensor models. The thresholds play a critical role in determining whether the data should be processed locally (at the edge) or sent to the cloud for further analysis.

(1) Single sensor model threshold values

The single-sensor model threshold values were calculated using the standard deviation and mean of the predicted confidence levels. The resulting threshold values, computed for each fault classes from the experimental data, are illustrated in Figure 4-20.


Class 0
Cloud (Min: 0.9989, Max: 1.0000)
Edge (Min: 0.9721, Max: 1.0000)
Edge Min Threshold (0.9992)
Edge Max Threshold (0.9996)
Cloud Min Threshold (0.9999)
Samples
Probability
Class 1
Cloud (Min: 0.3368, Max: 1.0000)
Edge (Min: 0.4483, Max: 1.0000)
Edge Min Threshold (0.9829)
Edge Max Threshold (0.9953)
Cloud Min Threshold (0.9743)
Samples
Probability
Class 2
Cloud (Min: 0.4362, Max: 0.9995)
Edge (Min: 0.4886, Max: 0.9977)
Edge Min Threshold (0.8727)
Edge Max Threshold (0.8976)
Cloud Min Threshold (0.9128)
Samples
Probability
Class 3
Cloud (Min: 0.4559, Max: 1.0000)
Edge (Min: 0.5046, Max: 1.0000)
Edge Min Threshold (0.9232)
Edge Max Threshold (0.9482)
Cloud Min Threshold (0.9185)
Samples
Probability
Class 4
Cloud (Min: 0.9657, Max: 1.0000)
Edge (Min: 0.6640, Max: 1.0000)
Edge Min Threshold (0.9875)
Edge Max Threshold (0.9943)
Cloud Min Threshold (0.9987)
Samples
Probability
Class 5
Cloud (Min: 0.5435, Max: 0.9995)
Edge (Min: 0.5039, Max: 0.9998)
Edge Min Threshold (0.8924)
Edge Max Threshold (0.9165)
Cloud Min Threshold (0.9180)
Samples
Probability

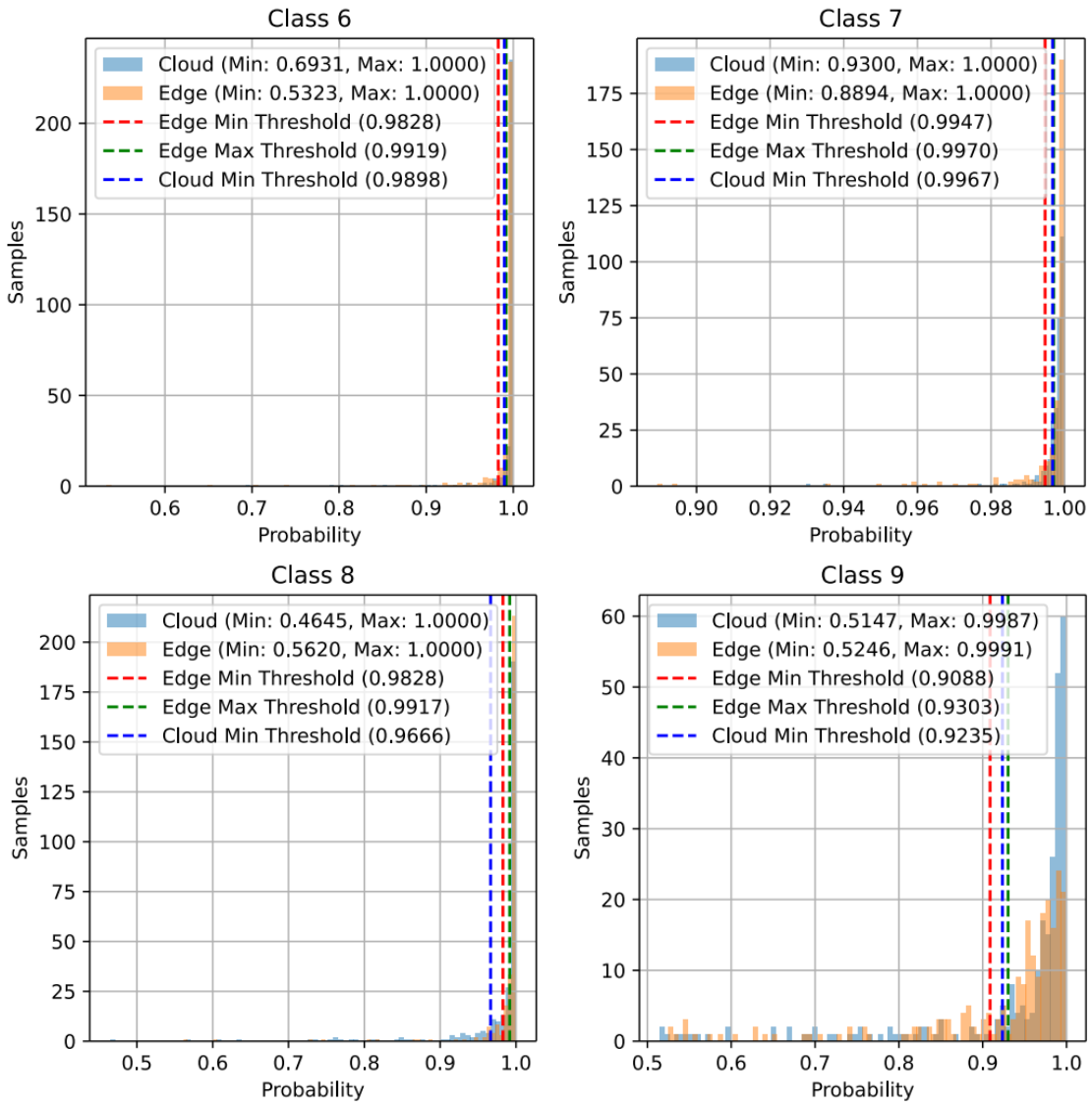


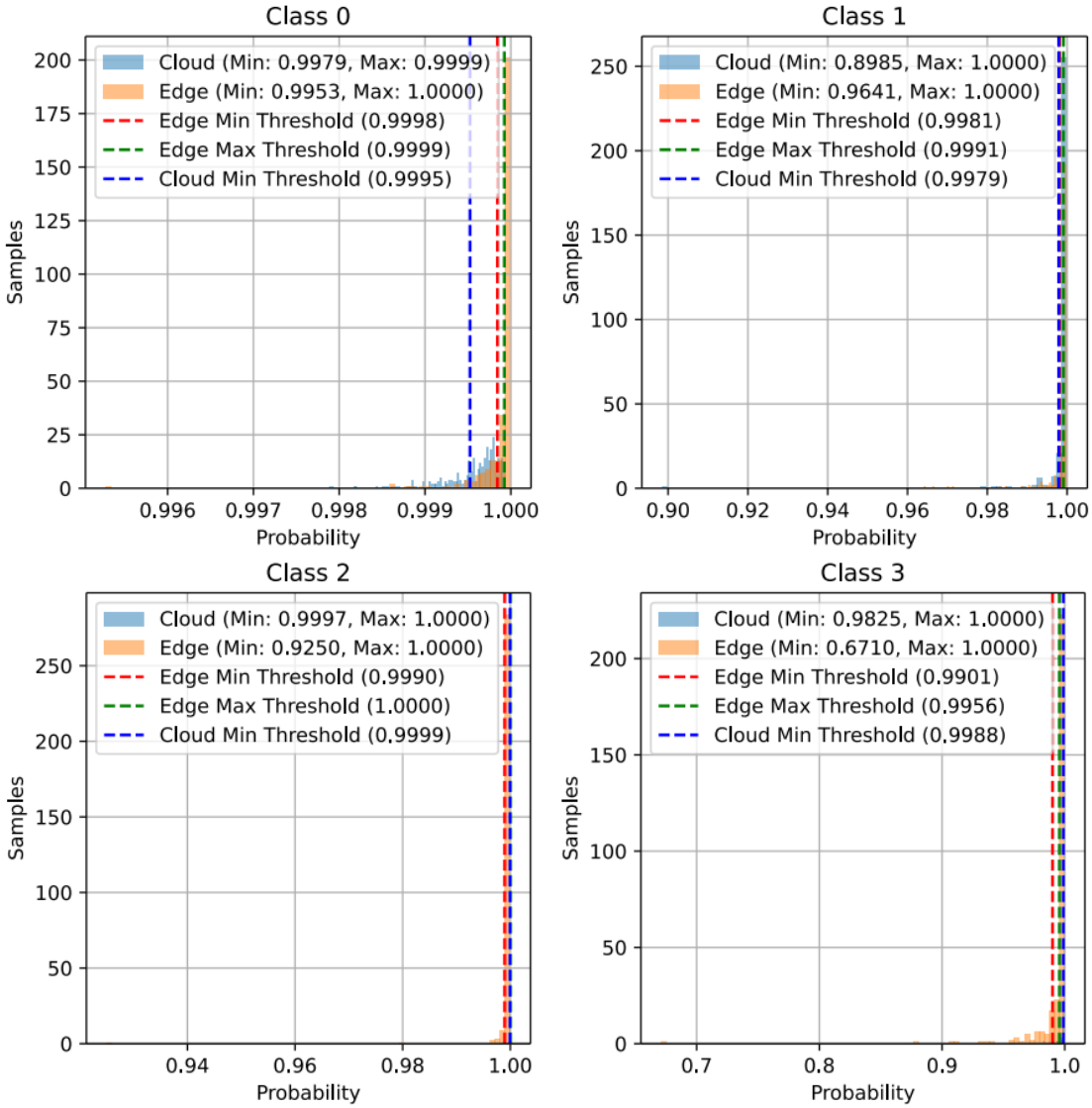

图 4-20 单传感器模型决策阈值

Figure 4-20 Single sensor model decision-making threshold values

Each graph corresponds to a specific fault class (Class 0 to Class 9) and illustrates the predicted probability values for that class. The graphs also include the decision-making thresholds for both the edge and cloud partitions. For each class, the majority of samples exhibit high confidence, with probabilities approaching 1, indicating that the edge partition of the model is generally confident in its fault diagnosis. The edge partition demonstrates a min-max range of predictions (e.g., Class 0 shows a range from 0.9721 to 1.0000), suggesting that the edge system is more conservative in its decision-making and operates within a narrower confidence range. In contrast, the cloud predictions typically exhibit higher minimum probability values (e.g., Class 0 starts at 0.9989), indicating that the cloud system is responsible for handling more uncertain or borderline cases, where the edge model lacks sufficient confidence to make a local decision.

(2) Multi-sensor model threshold values

The multi-sensor model decision-making thresholds were computed for the model utilizing various aggregation methods. The aggregation methods employed to combine features from multiple sensors are detailed in Chapter 4.3 (Multi-Sensor Model Architecture). The resulting threshold values, computed for each fault class under max-pooling, min-pooling, average-pooling and concatenation feature aggregation from multiple sensors based on the experimental data, are presented in Figures from 4-21 to 4-23.



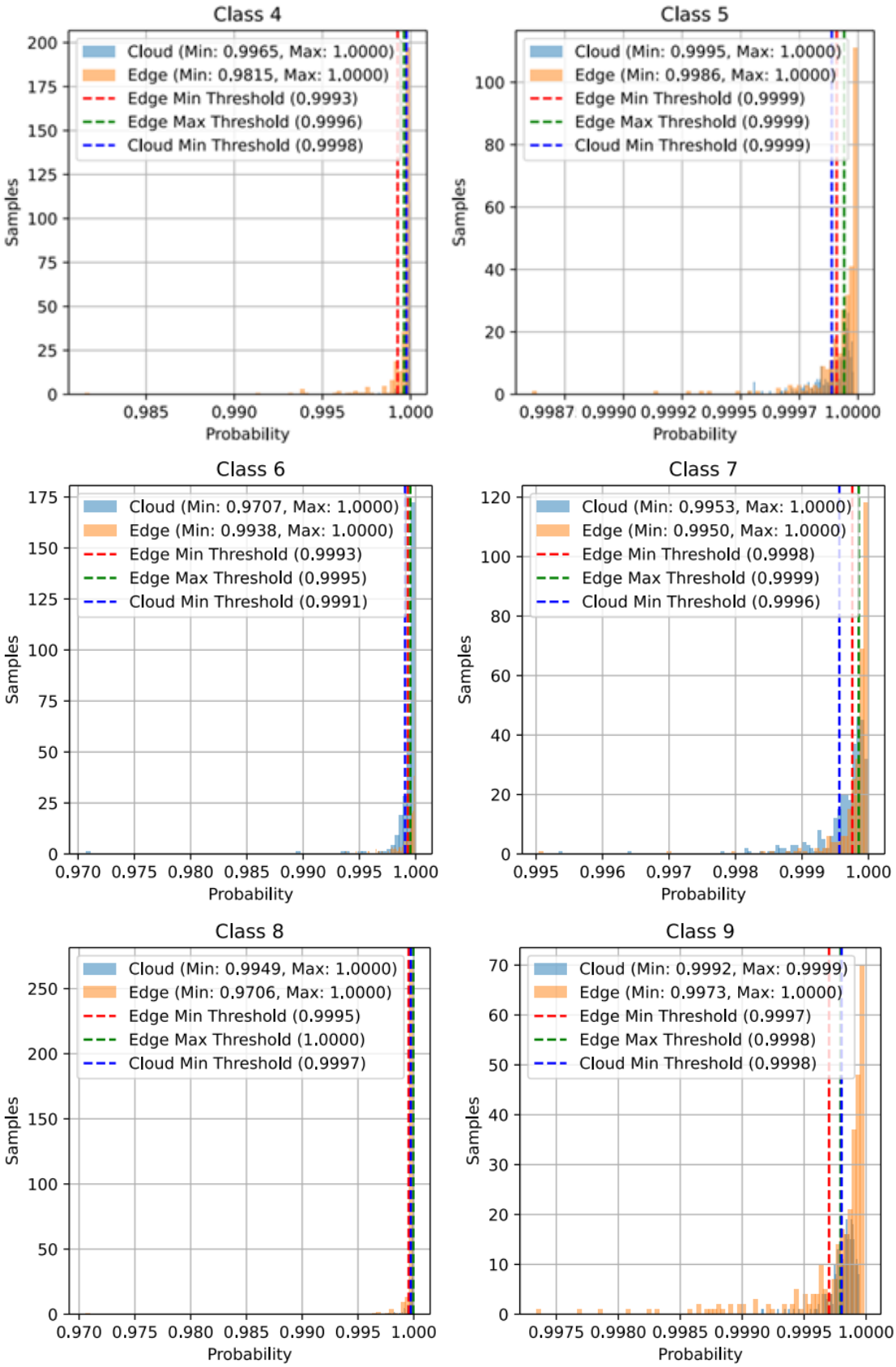
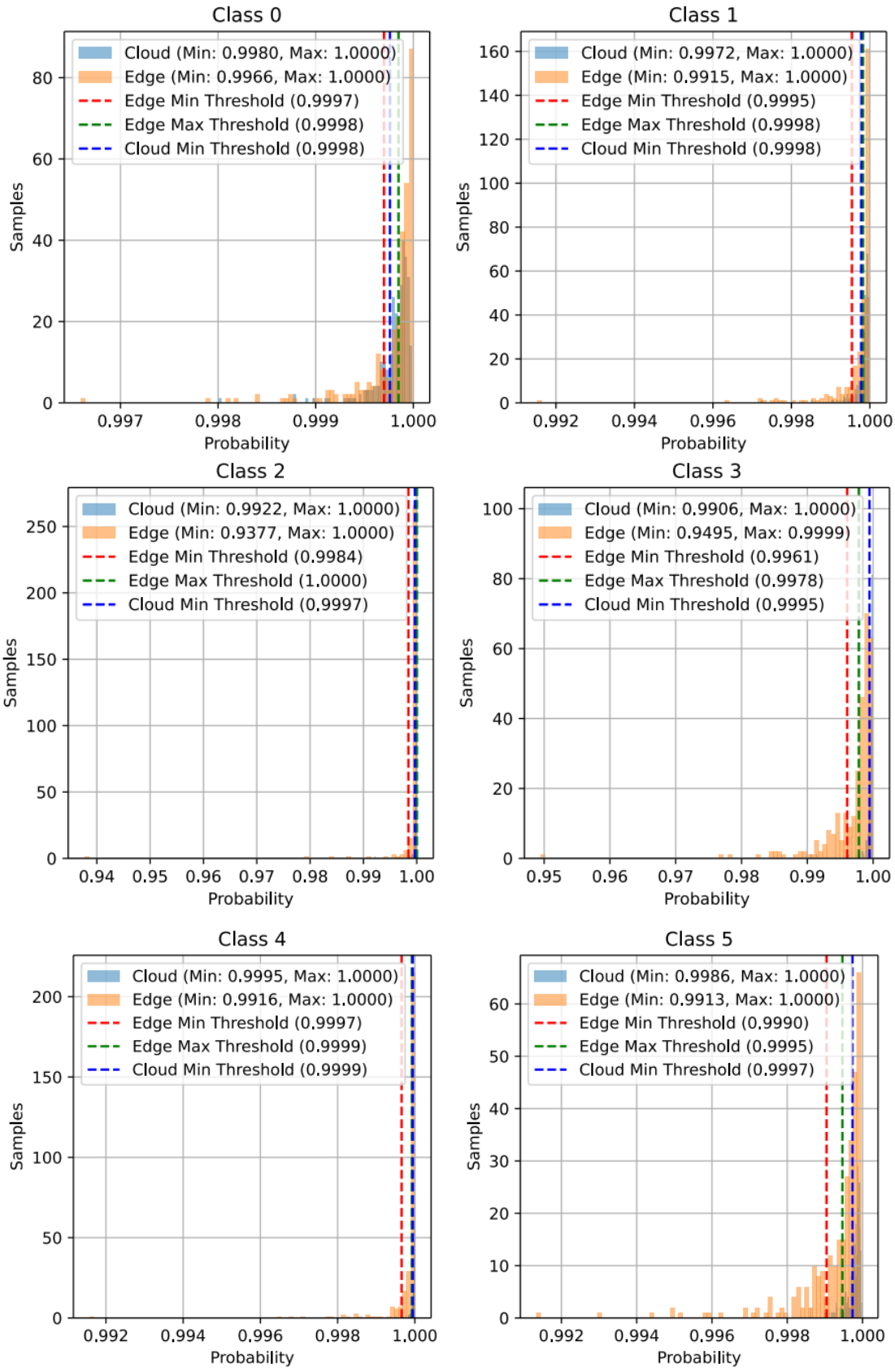



图 4-21 使用最大池化特征聚合的多传感器模型决策阈值

Figure 4-21 Decision-making threshold values for multi-sensor model with max pooling feature aggregation


Class 0
Cloud (Min: 0.9980, Max: 1.0000)
Edge (Min: 0.9966, Max: 1.0000)
Edge Min Threshold (0.9997)
Edge Max Threshold (0.9998)
Cloud Min Threshold (0.9998)
Samples
Probability
Class 1
Cloud (Min: 0.9972, Max: 1.0000)
Edge (Min: 0.9915, Max: 1.0000)
Edge Min Threshold (0.9995)
Edge Max Threshold (0.9998)
Cloud Min Threshold (0.9998)
Samples
Probability
Class 2
Cloud (Min: 0.9922, Max: 1.0000)
Edge (Min: 0.9377, Max: 1.0000)
Edge Min Threshold (0.9984)
Edge Max Threshold (1.0000)
Cloud Min Threshold (0.9997)
Samples
Probability
Class 3
Cloud (Min: 0.9906, Max: 1.0000)
Edge (Min: 0.9495, Max: 0.9999)
Edge Min Threshold (0.9961)
Edge Max Threshold (0.9978)
Cloud Min Threshold (0.9995)
Samples
Probability
Class 4
Cloud (Min: 0.9995, Max: 1.0000)
Edge (Min: 0.9916, Max: 1.0000)
Edge Min Threshold (0.9997)
Edge Max Threshold (0.9999)
Cloud Min Threshold (0.9999)
Samples
Probability
Class 5
Cloud (Min: 0.9986, Max: 1.0000)
Edge (Min: 0.9913, Max: 1.0000)
Edge Min Threshold (0.9990)
Edge Max Threshold (0.9995)
Cloud Min Threshold (0.9997)
Samples
Probability

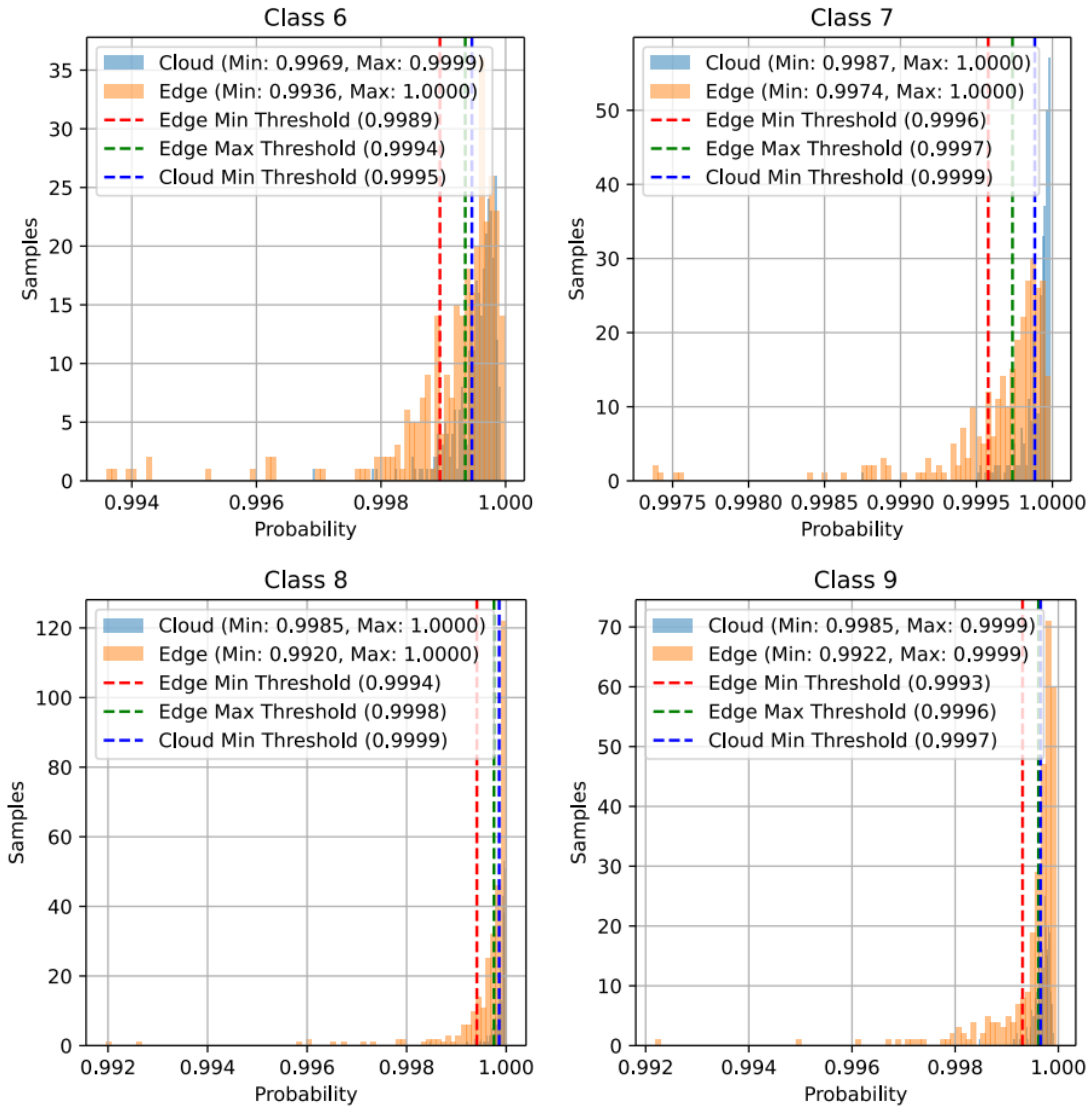
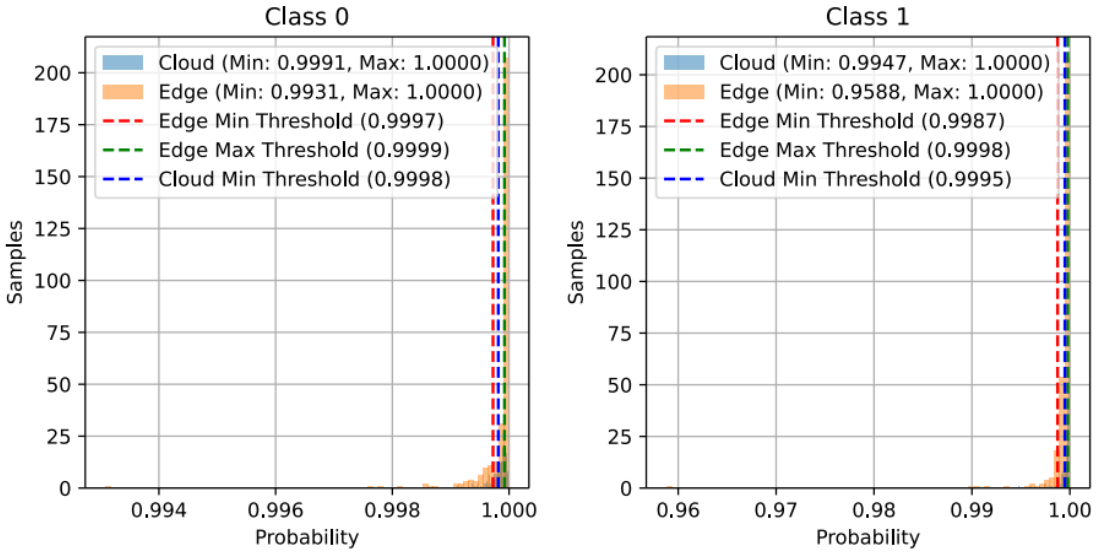
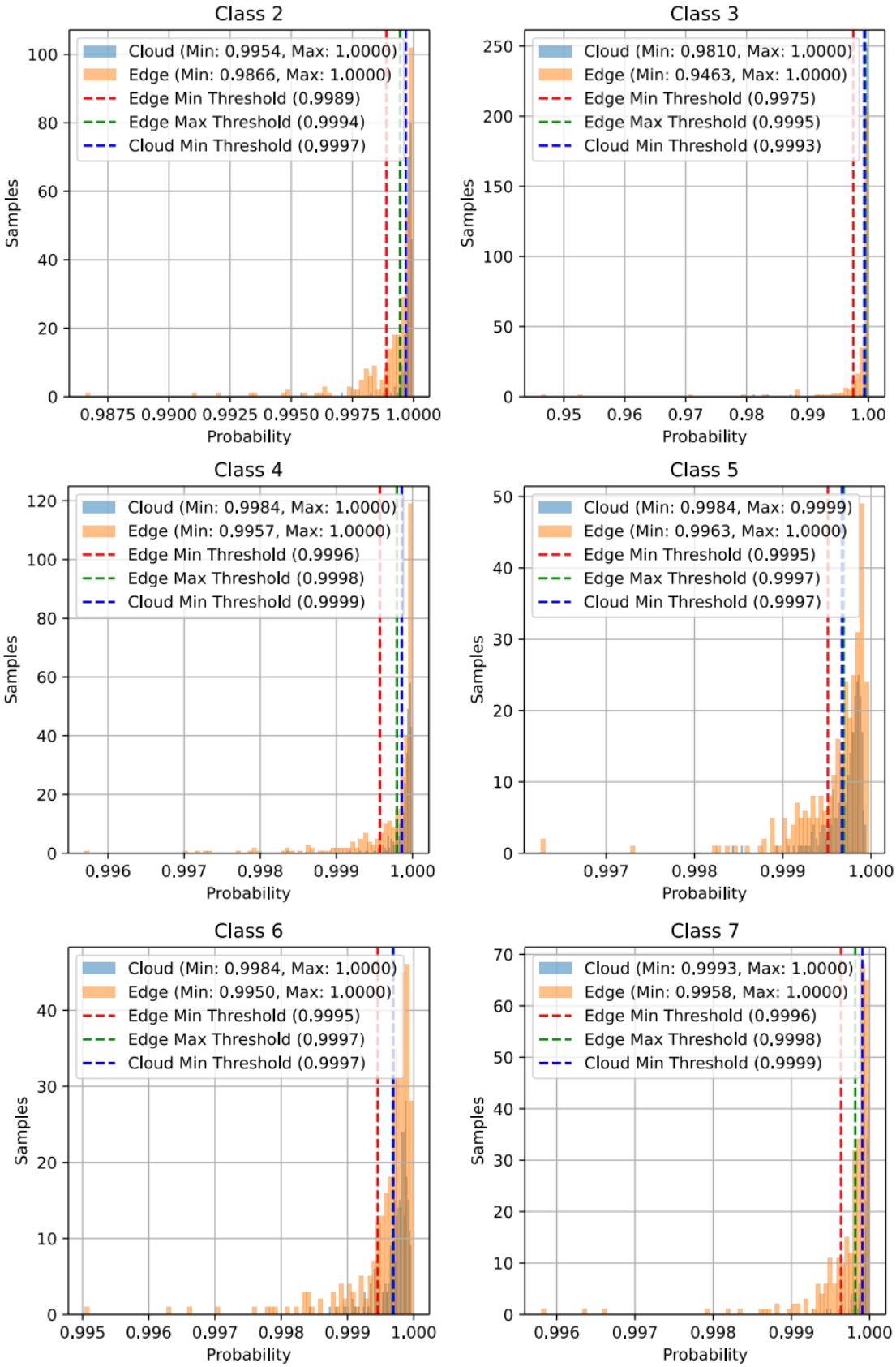



图 4-22 使用最小池化特征聚合的多传感器模型决策阈值

Figure 4-22 Decision-making threshold values for multi-sensor model with min pooling feature aggregation




Class 2
Cloud (Min: 0.9954, Max: 1.0000)
Edge (Min: 0.9866, Max: 1.0000)
Edge Min Threshold (0.9989)
Edge Max Threshold (0.9994)
Cloud Min Threshold (0.9997)
Samples
Probability
Class 3
Cloud (Min: 0.9810, Max: 1.0000)
Edge (Min: 0.9463, Max: 1.0000)
Edge Min Threshold (0.9975)
Edge Max Threshold (0.9995)
Cloud Min Threshold (0.9993)
Samples
Probability
Class 4
Cloud (Min: 0.9984, Max: 1.0000)
Edge (Min: 0.9957, Max: 1.0000)
Edge Min Threshold (0.9996)
Edge Max Threshold (0.9998)
Cloud Min Threshold (0.9999)
Samples
Probability
Class 5
Cloud (Min: 0.9984, Max: 0.9999)
Edge (Min: 0.9963, Max: 1.0000)
Edge Min Threshold (0.9995)
Edge Max Threshold (0.9997)
Cloud Min Threshold (0.9997)
Samples
Probability
Class 6
Cloud (Min: 0.9984, Max: 1.0000)
Edge (Min: 0.9950, Max: 1.0000)
Edge Min Threshold (0.9995)
Edge Max Threshold (0.9997)
Cloud Min Threshold (0.9997)
Samples
Probability
Class 7
Cloud (Min: 0.9993, Max: 1.0000)
Edge (Min: 0.9958, Max: 1.0000)
Edge Min Threshold (0.9996)
Edge Max Threshold (0.9998)
Cloud Min Threshold (0.9999)
Samples
Probability

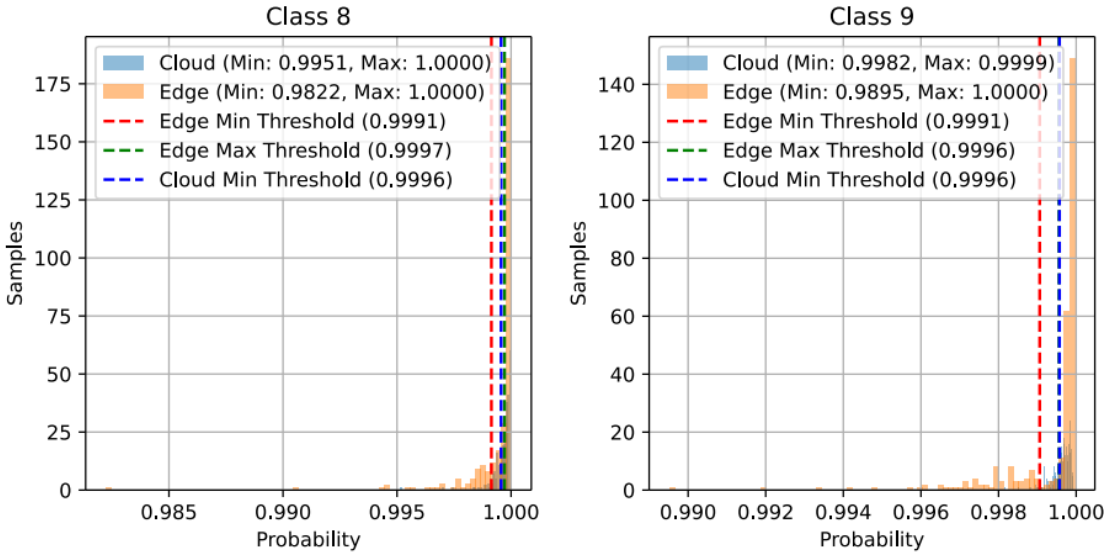
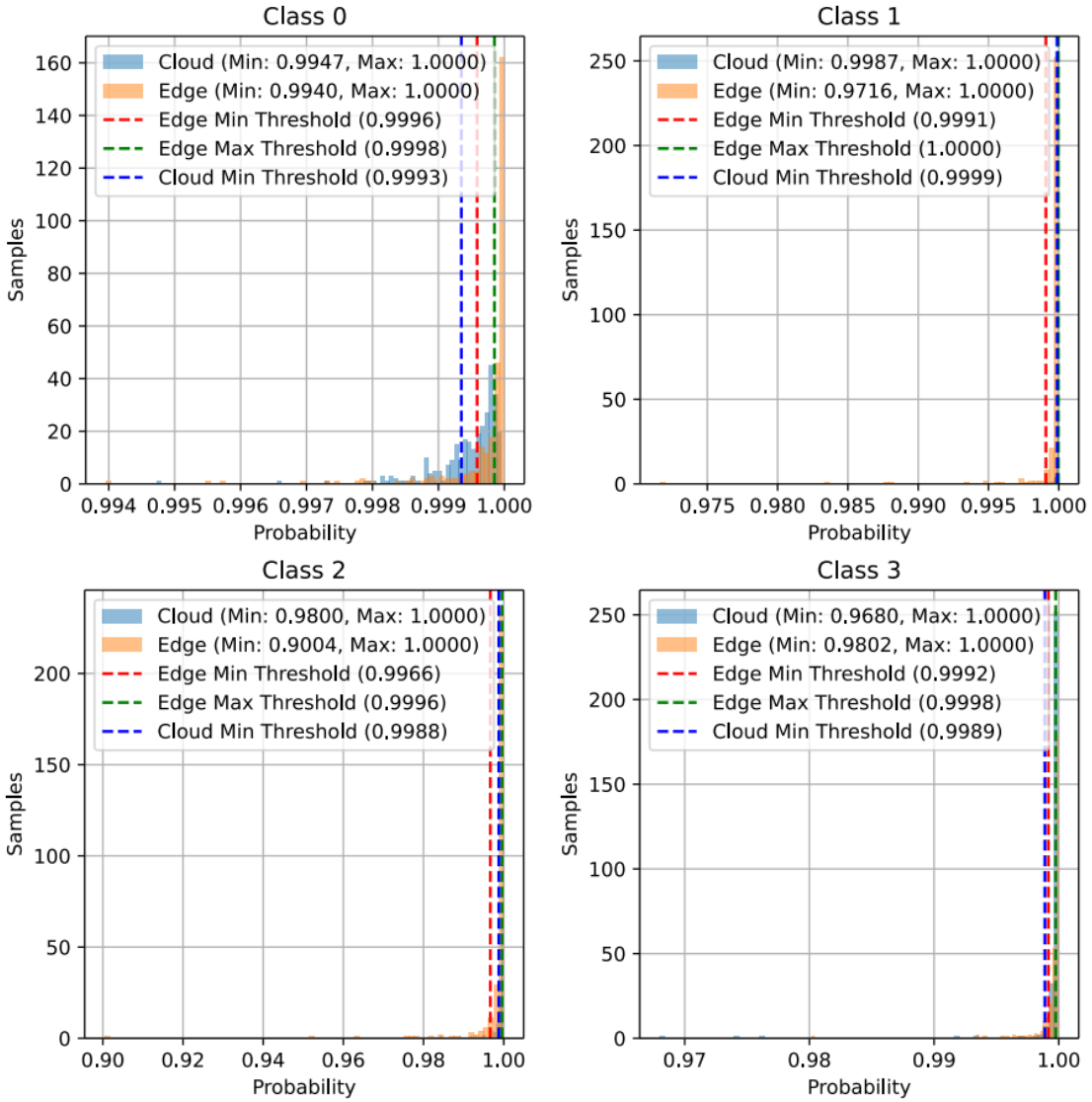



图 4-23 使用平均池化特征聚合的多传感器模型决策阈值

Figure 4-23 Decision-making threshold values for multi-sensor model with average pooling feature aggregation



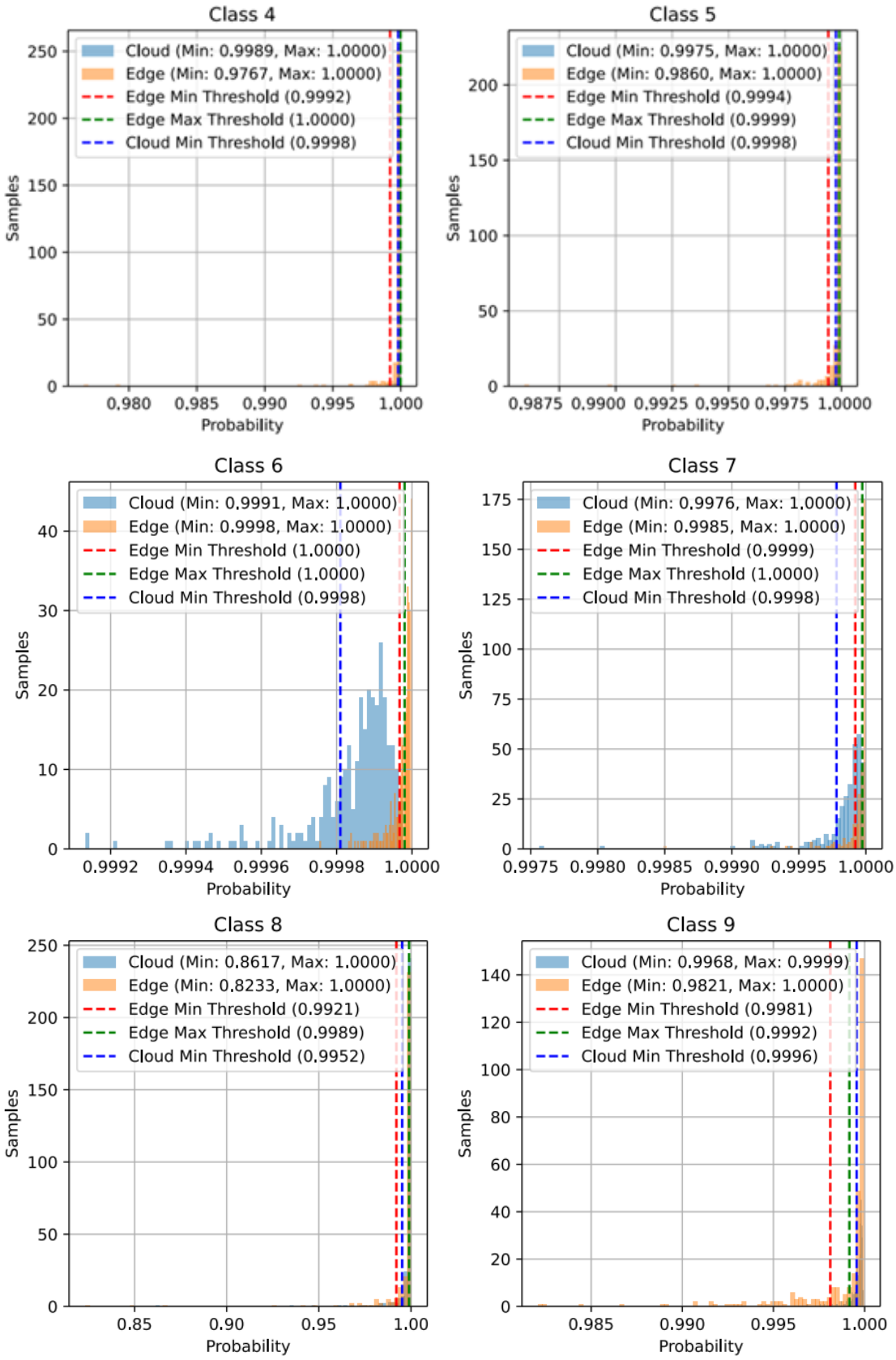



图 4-24 使用连接特征聚合的多传感器模型决策阈值

Figure 4-24 Decision-making threshold values for multi-sensor model with concatenation feature aggregation

For each fault label (0–9), the table lists minimum prediction confidence of the Cloud and Edge for correct classifications under five different configurations:

表 4-9 每类故障的云端与边缘端最小置信度值

Table 4-9 Cloud and edge min values for each fault label

| Fault Lable | Cloud – Min<br>SNone MMax MMin MAvg MCon | Edge – Min<br>SNone MMax MMin MAvg MCon |
|---|---|---|
| 0 | 0.9989 0.9979 0.9980 0.9991 0.9947 | 0.9721 0.9953 0.9966 0.9931 0.9940 |
| 1 | 0.3368 0.8985 0.9972 0.9947 0.9987 | 0.4483 0.9641 0.9915 0.9588 0.9716 |
| 2 | 0.4362 0.9997 0.9922 0.9954 0.9800 | 0.4886 0.9250 0.9377 0.9866 0.9004 |
| 3 | 0.4559 0.9825 0.9906 0.9810 0.9680 | 0.5046 0.6710 0.9495 0.9463 0.9802 |
| 4 | 0.9657 0.9965 0.9995 0.9984 0.9989 | 0.6640 0.9815 0.9916 0.9957 0.9767 |
| 5 | 0.5435 0.9995 0.9986 0.9984 0.9975 | 0.5039 0.9986 0.9913 0.9963 0.9860 |
| 6 | 0.6931 0.9707 0.9969 0.9984 0.9991 | 0.5323 0.9938 0.9936 0.9950 0.9998 |
| 7 | 0.9300 0.9953 0.9987 0.9993 0.9976 | 0.8894 0.9950 0.9974 0.9958 0.9985 |
| 8 | 0.4645 0.9949 0.9985 0.9951 0.8617 | 0.5620 0.9706 0.9920 0.9822 0.8233 |
| 9 | 0.5147 0.9992 0,9985 0.9982 0.9968 | 0.5246 0.9973 0.9922 0.9895 0.9821 |

The multi-sensor model experiments demonstrate that each feature aggregation method offers distinct advantages depending on the nature of the fault classification task. Max Pooling provides the most efficient cloud-edge distribution, with the edge handling the majority of predictions confidently. Min Pooling and Average Pooling increase reliance on the cloud due to the edge model's increased uncertainty, with Min Pooling being better then Average Pooling. Concatenation provides the most detailed and comprehensive fault classification, but at the cost of increased complexity and a broader spread of edge model predictions. The decision-making thresholds for both edge and cloud ensure that computational resources are effectively distributed, allowing for efficient real-time processing and accurate fault diagnosis.

### 4.5.9 Cascade Tests of the Models

In this section, we present the results of the cascade tests conducted on the single-sensor and multi-sensor models. Cascade testing is designed to evaluate the model's performance under sequential stages, where data passes through multiple layers of processing (i.e., the edge layer followed by the cloud layer). This method simulates

real-world operational scenarios, providing insights into how well the edge and cloud partitions of the models work together in a dynamic fault diagnosis process. The tests were carried out in a controlled environment where data from test dataset passed through the models and decision-making thresholds to simulate edge-to-cloud collaboration.

(1) Single sensor model cascade test

The single sensor model performed really well in the cascade test, as illustrated in Figure 4-25.

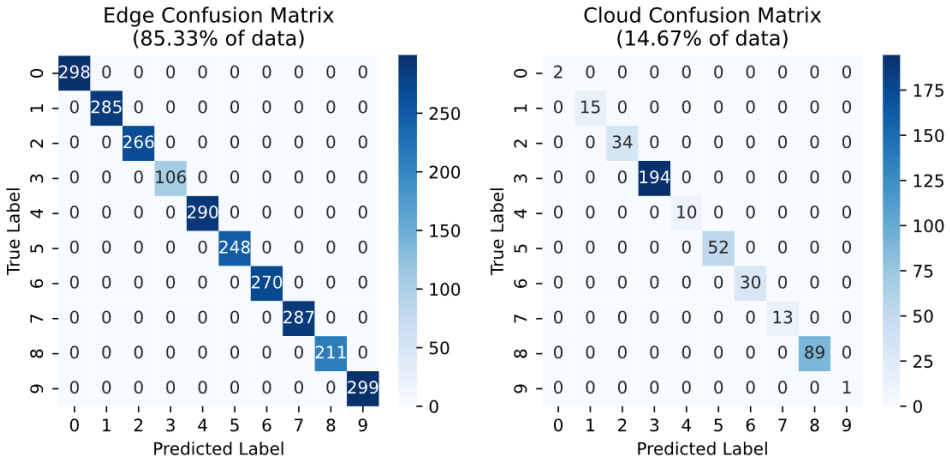



图 4-25 单传感器模型级联测试

Figure 4-25 Single sensor model cascade test

The single sensor model performed really well in the cascade test, as illustrated in Figure 4-25. The Edge component, which processed 85.33% of the data, demonstrated impressive classification accuracy with most of the predictions correctly aligned along the diagonal of the confusion matrix. For example, labels 0 and 9 had nearly perfect predictions, with only a few misclassifications. This indicates that the Edge model can accurately handle the majority of fault diagnosis cases with high efficiency. However, for the remaining 14.67% of data, which the Edge model could not confidently classify, the Cloud component took over. Overall, the performance of the system was highly effective, with the Edge component achieving excellent results and the Cloud model providing a valuable backup for uncertain cases.

（2） Multi-sensor model cascade tests

Multi-sensor model cascade tests aim to evaluate the performance of a system utilizing multiple sensors, which is a more complex setup than a single sensor model.

Figure 4-26 displays the cascade test of the multi-sensor model using max-pooling feature aggregation. The Edge partition of the model performs exceptionally well compared to the single-sensor model, diagnosing the vast majority of cases (93.63% of

the data) with a very low rate of misclassifications. This improved performance is a result of the max-pooling aggregation, which effectively focuses on the most prominent features from multiple sensors, enabling the Edge model to make accurate and rapid predictions. The Cloud partition of the model processes the more complex or uncertain cases, which constitute 6.37% of the test data.

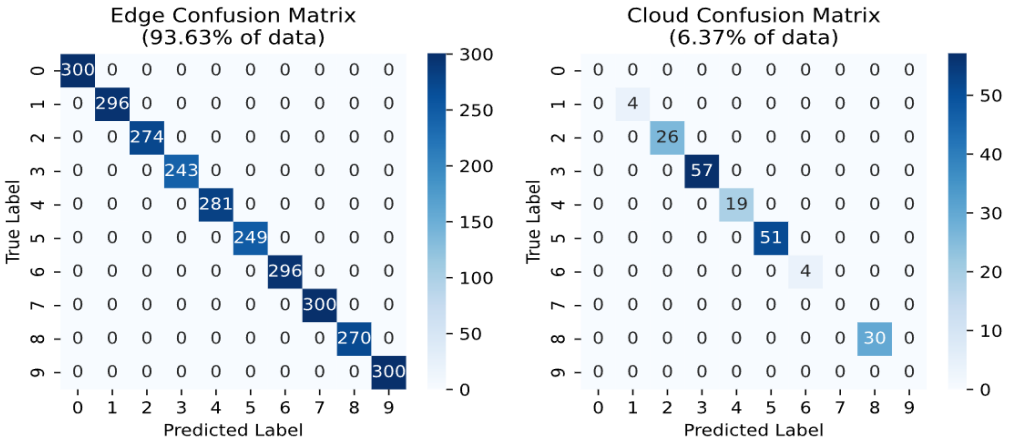



图 4-26 使用最大池化特征聚合的多传感器模型级联测试

Figure 4-26 Cascade test of multi-sensor model with max pooling feature aggregation

Figure 4-27 shows the cascade test of the multi-sensor model with min-pooling feature aggregation. The Edge partition of the model in this test handles 90.50% of the data. This is a slightly lower percentage of the total test data compared to the multi-sensor model with max-pooling feature aggregation (which handles 93.63% of the data). The Edge model in the min-pooling setup processes a significant amount of data but with a slightly reduced accuracy due to the aggregation method, which focuses on weaker features compared to max-pooling. Meanwhile, the Cloud partition of the model handles the remaining 9.50% of the test data.

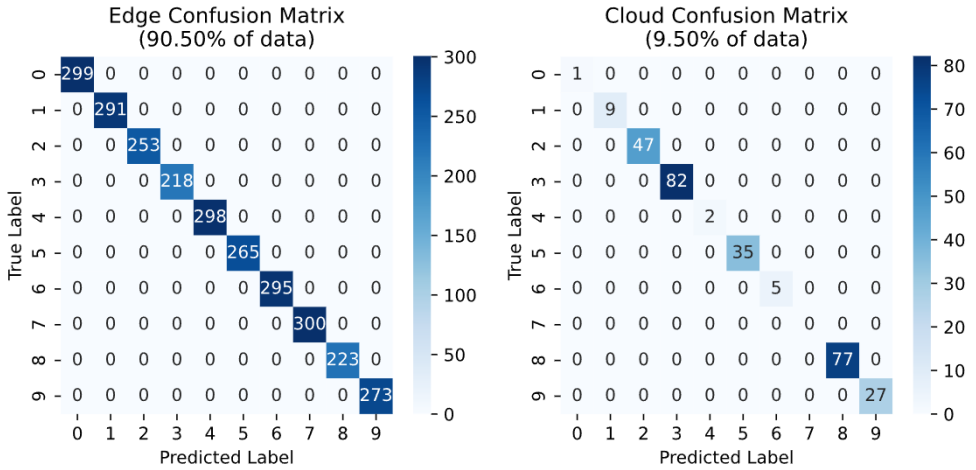



图 4-27 使用最小池化特征聚合的多传感器模型级联测试

Figure 4-27 Cascade test of multi-sensor model with min pooling feature aggregation

Figure 4-28 shows the cascade test of the multi-sensor model with average pooling feature aggregation. The Edge partition of the model handles 90.03% of the data, which is slightly lower than what was achieved in the max-pooling (93.63%) and min-pooling r (90.50%) models, but still represents a significant portion of the data. The Cloud partition processes rest 9.97% of the data. The average pooling method doesn't focus as much on the strongest features as max-pooling and min-pooling, which may explain the slightly less efficient performance in the Edge.

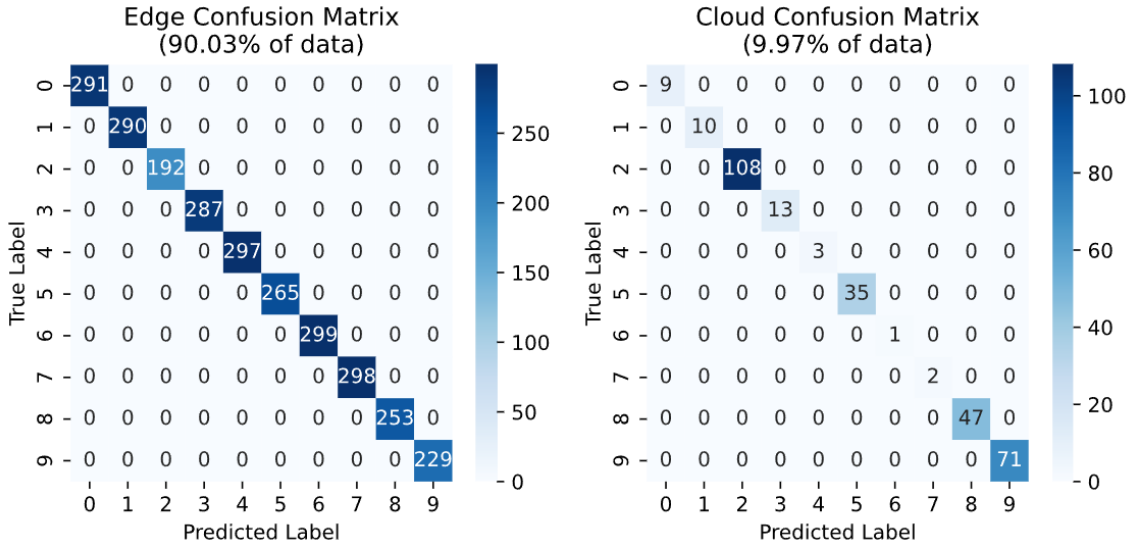



图 4-28 使用平均池化特征聚合的多传感器模型级联测试

Figure 4-28 Cascade test of multi-sensor model with average pooling feature aggregation

The concatenation feature aggregation method leads to very strong performance in the cascade test, as seen in Figure 4-29. The Edge model handles 95.93% of the data with high accuracy, and the Cloud model effectively processes the more complex cases (4.07% of the data) with relatively low misclassification rates. This method benefits from combining features from multiple sensors, providing the model with a richer, more comprehensive feature set, which improves overall diagnostic accuracy.

However, a significant drawback of the concatenation feature aggregation method is that it requires more computational resources. By combining features from multiple sensors, the resulting feature vector becomes larger and more complex. This increase in the dimensionality of the data demands more processing power, which can lead to higher computational costs, particularly in real-time applications where efficiency is critical.

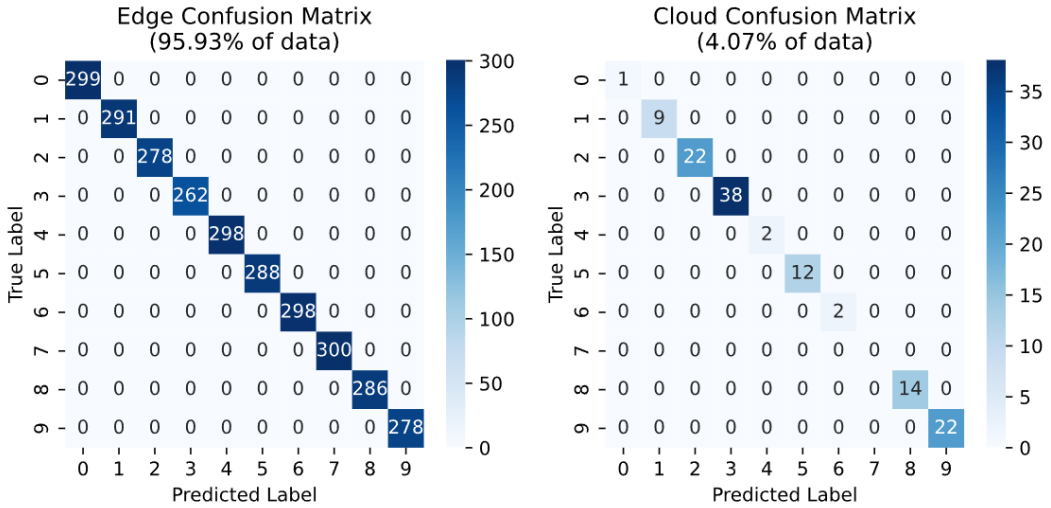



图 4-29 使用连接特征聚合的多传感器模型级联测试

Figure 4-29 Cascade test of multi-sensor model with concatenation feature aggregation

In conclusion, the cascade tests of the single sensor model and multi-sensor models with different feature aggregation methods (max-pooling, min-pooling, average pooling, and concatenation) reveal distinct strengths and trade-offs. The single sensor model is efficient and accurate for simpler fault diagnosis tasks, offering fast processing and lower computational demands, making it ideal for real-time applications. However, its performance can be limited in more complex scenarios that require data from multiple sensors.

The multi-sensor models demonstrate superior performance in handling complex faults. Among the aggregation methods, max-pooling provides the best balance of high accuracy and computational efficiency, with the Edge model processing a large portion of the data (93.63%) with minimal misclassifications. Concatenation offers the highest accuracy for the Edge model (95.93%), but it comes with a higher computational cost, which can be a drawback in resource-constrained environments. Min-pooling and average pooling offer reasonable performance.

## 4.6 总结（Summary）

The chapter introduces a proposed cloud-edge collaborated Deep Neural Network CNN model inspired by the Wide and Deep Convolutional Neural Network (WDCNN). The model is designed to efficiently combine edge and cloud computing resources for fault detection and diagnosis. The model consisting of five filtering stages, is adapted for edge deployment by reducing the number of filtering stages to two while keeping the classifier layers intact. This provides a computationally efficient setup suitable for edge devices with limited resources. The cloud portion, serving multiple devices,

retains three filtering stages to balance performance and scalability. To enable joint training across both cloud and edge partitions, the model is restructured into a BranchyNet[131] architecture, which allows multiple exit points and backpropagates loss from each exit point for optimal performance. This setup facilitates distributed inference while maintaining accuracy across both cloud and edge components. Furthermore, the chapter explores the potential for horizontal expansion by integrating multiple sensors, which together can leverage collective intelligence for better decision-making. The multi-sensor model aggregates sensor data using various techniques, such as max, min, average pooling, and concatenation, to improve classification accuracy. Experimental validation is performed using the Case Western Reserve University (CWRU) dataset, which includes vibration data from a motor system with fault-induced bearings. The dataset includes a range of fault scenarios with different fault sizes, and vibration data is collected at various operational states. The model is trained with this dataset, with an emphasis on augmenting the data to improve the model's robustness and prevent overfitting.

# 5 云边协同系统实现
# 5 Cloud-Edge Collaborated System Implementation

Building upon the intelligent operation and maintenance of construction machinery, this chapter integrates cloud computing, edge computing, deep learning, and software programming technologies to facilitate real-time fault diagnosis for construction machinery. Drawing from the Cloud-Edge Collaborative Fault Diagnosis Framework presented in Chapter 3 and the Cloud-Edge Collaborative Fault Diagnosis Model introduced in Chapter 4, this chapter outlines the design and implementation of a fault diagnosis system for construction machinery using a cloud-edge collaborative computing approach.

## 5.1 系统设计（System Design）

The implementation of the proposed framework, as outlined in Chapter 3, presents a cloud-edge collaborative fault diagnosis system that integrates both cloud and edge processing to facilitate efficient and adaptive industrial fault diagnosis. The architecture comprises several essential components and decision-making processes, which are illustrated in Figure 5-1.

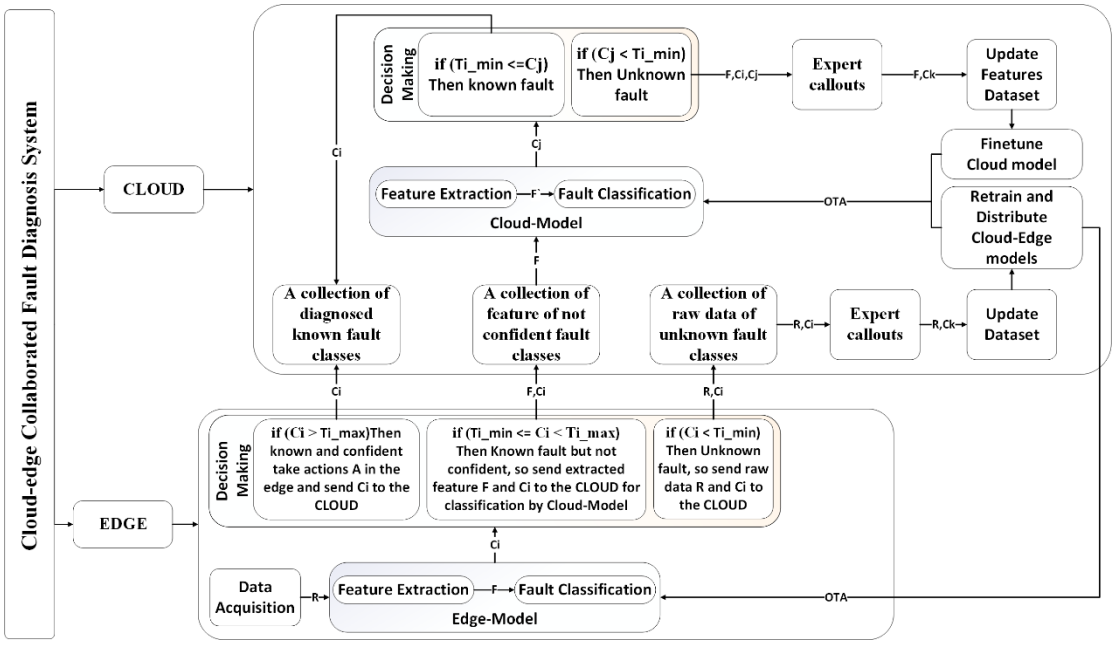



图 5-1 云边协同故障诊断框架的故障诊断图

Figure 5-1 Fault Diagnosis Diagram of the Cloud-edge collaborative fault diagnosis framework

Where R denote the raw vibration data, $Ci$ the confidence value for the $i$-th fault class predicted by the edge, $F$ the features extracted by the edge, and $F'$ the features extracted by the cloud. $Cj$ represents the confidence value for the $j$-th fault class

predicted by the cloud, while $Ck$ is the fault label for the $k$-th fault class as assigned by an expert. The Over-the-Air (OTA) mechanism refers to the process through which new model updates are remotely delivered to both the edge and cloud systems. On the edge side, data is collected and processed from industrial equipment. The edge model performs initial feature extraction and fault classification, filtering out known faults and classifying unknown ones based on the predefined thresholds. On the edge side, data is collected and processed from industrial equipment. The edge model performs initial feature extraction and fault classification, filtering out known faults and classifying unknown ones based on the predefined thresholds. The edge model handles the initial extraction of relevant features from fault signals, performing fault classification for known and unknown fault types. The system can communicate with the cloud to refine the fault diagnosis by sending the features extracted from the edge data for cloud-side analysis. The cloud model processes a collection of known faults, categorized as confident fault classes. If the fault classification confidence level (Ti_min) is lower than a predefined threshold (Cj), the system deems the fault as a known fault, initiating the corresponding corrective actions (A). The cloud model processes a collection of known faults, categorized as confident fault classes. If the fault classification confidence level (Ti_min) is lower than a predefined threshold (Cj), the system deems the fault as a known fault, initiating the corresponding corrective actions (A). Extracted features from faults are analyzed and classified by the cloud model. Known faults are processed first, and unknown faults are handled by gathering raw data, which is then uploaded to the cloud for more in-depth analysis. The cloud then proceeds to update the feature dataset, fine-tune the cloud model, and retrain the system with the newly acquired data, which is subsequently distributed across the edge and cloud systems. Expert callouts are incorporated throughout the process to validate or label fault data, helping refine the dataset and improve the accuracy of model predictions. The cloud continuously updates the models and features, ensuring the system evolves with new fault data. Both cloud and edge models can update one another continuously, with the edge-side models receiving cloud updates via over-the-air (OTA) deployment. Additionally, the cloud can update its models based on feedback from the edge-side and expert interventions, allowing both systems to learn and adapt to new fault scenarios.

## 5.2 设备平面实现（Equipment-Plane Implementation）

To enable data acquisition and real-time monitoring of bearing status parameters, as well as to provide data validation for fault diagnosis algorithms, it is essential to

install appropriate sensors on the primary rotating components of the construction machinery. This setup will facilitate the analysis of potential bearing fault characteristic signals. Therefore, for the data acquisition system, the following types of vibration signal sensors can be selected. The specific criteria for sensor selection are detailed as follows. The vibration sensor is installed on the side of the primary rotating components to measure vibration signals. The CYT9200 vibration sensor, produced by China Tianyu Hengchuang, is selected, as shown in Figure 5-2. The parameters of the vibration sensor are listed in Table 5-1.

图 5-2 振动传感器（CYT9200）

Figure 5-2 Vibration sensor (CYT9200)

表 5-1 振动传感器（CYT9200）参数

Table 5-1 Vibration sensor (CYT9200) parameters

| Function | Parameter | Function | Parameter |
|---|---|---|---|
| Product Model | CYT9200 | Temperature Range | -30~65℃ |
| Measurement Range | 0~50mm/s | Humidity Range | 5~95% |
| Maximum Measurement Error | ≤±1% | Connection Thread | M10×1.5mm |
| Measurement Frequency Range | 5~1000Hz | Sensitivity | 50mm/s ±5% |
| Operating Voltage | DC/24V ±6V | Measurement Direction | Vertical or Horizontal |

## 5.3 边缘平面实现（Edge-Plane Implementation）

The edge system comprises three components: two hardware components—namely, the data acquisition device (WEBDAQ504, as shown in Figure 5-3) and the intelligent vehicle-mounted information terminal (TBOX, as shown in Figure 5-4) along with the software, which is the smart Edge gateway running on the TBOX (Smart Edge gateway, as shown in Figure 5-5). The edge layer is primarily responsible for the collection, preliminary fault diagnosis, and transmission of equipment operational data. The collected data, along with the diagnostic results, are packaged according to the MQTT protocol and subsequently uploaded to the cloud. The intelligent vehicle-mounted information terminal (TBOX) is equipped with network interfaces to

communicate with various devices using protocols such as MQTT, BLE (Bluetooth Low Energy), OPC-UA, Modbus, CAN, and others. Additionally, it features a 4G Unicom SIM card (4G) that facilitates connectivity to the cloud via a Unicom VPN. In situations where network signals are unstable, the collected data is initially cached in local files; once the network connection is restored, the data is batch-transmitted to the cloud.

图 5-3 WebDAQ-504 振动信号采集设备

Figure 5-3 WebDAQ-504 vibration signal acquisition device

图 5-4 智能车载信息终端 TBOX

Figure 5-4 Intelligent vehicle-mounted information terminal TBOX

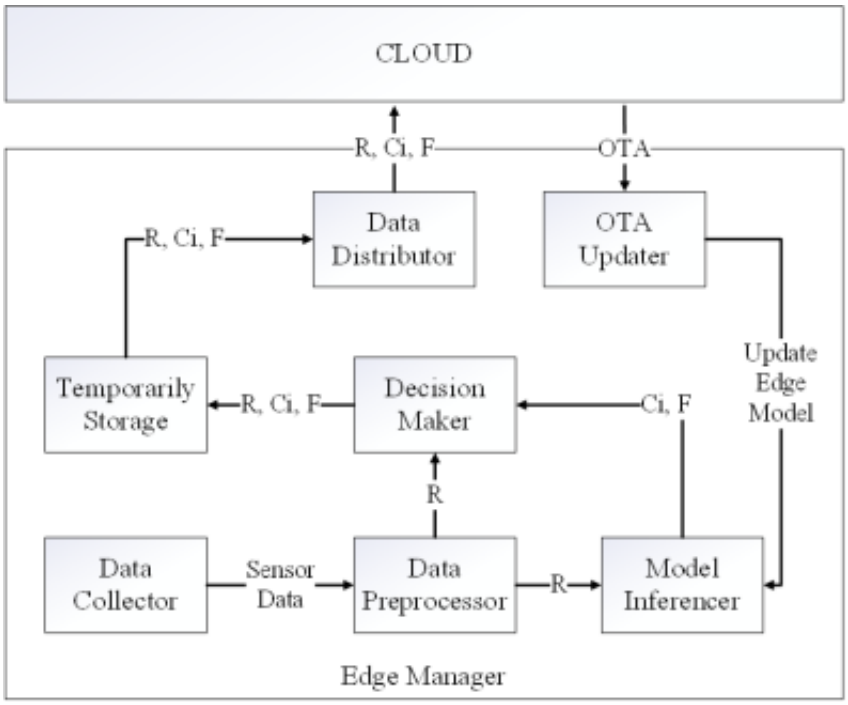



图 5-5 智能边缘网关软件

Figure 5-5 Smart edge gateway software

The Smart Edge gateway software is composed of several key components, each playing a critical role in managing and processing data within the edge computing system.

Edge Manager: The Edge Manager oversees the functions of each module within the edge computing system, ensuring the smooth operation and coordination of the various components in the edge layer.

Data Collector: The Data Collector is responsible for gathering raw data from data acquisition devices or sensors. Its primary role is to retrieve data from multiple sources, including sensors, equipment, or other monitoring devices.

Data Preprocessor: The Data Preprocessor performs essential preprocessing tasks on the raw data before it is sent for further analysis. It transforms the collected data into a clean, usable format, making it suitable for downstream systems such as data analysis, machine learning, or fault diagnosis models.

Model Inferencer: The Model Inferencer invokes edge-based deep learning models and performs feature extraction on the preprocessed data to generate inferences or predictions. It is critical for utilizing machine learning or deep learning models to analyze data and provide insights related to fault detection, prediction, or classification.

Decision Maker: The Decision Maker temporarily stores data—whether processed or raw—according to predefined strategies based on the edge thresholds. It ensures that the data is handled appropriately and stored in alignment with the system's operational strategies, making it available for later decision-making or further analysis.

OTA Updater: The OTA Updater manages the downloading and updating of the latest versions of edge software or deep learning models via Over-The-Air (OTA) updates. It ensures that edge devices are running the most up-to-date software or models, thus improving functionality, performance, and security.

Temp Storage: The Temp Storage serves as a temporary repository where data, managed by the Decision Maker, is stored for a limited period before further processing or transmission. It acts as a buffer, ensuring that both raw and processed data are available for future access by other system components or decision-making processes.

Data Distributer: The Data Distributer is responsible for transferring data from the Temp Storage to the platform (typically the cloud or a central system) for further processing, analysis, or long-term storage. It ensures that data is efficiently and securely uploaded to the appropriate platform.

Both the Data Collector and Data Distributer support multiple network protocols,

enhancing their flexibility and integration within the system.

## 5.4 云平面实现（Cloud-Plane Implementation）

The cloud plane serves as the foundational element of the entire system, primarily responsible for the management, allocation, and scheduling of server cluster resources. At the base layer of the cloud is the server cluster, where virtualization and resource management are facilitated using private cloud components, which also provide container orchestration and scheduling services. On top of this infrastructure, open-source tools such as Hadoop, Spark, relational and non-relational databases, and data warehouses are deployed to establish the cloud's comprehensive infrastructure, supporting the runtime environment for the application layer. The cloud architecture built on this infrastructure is depicted in Figure 5-6.

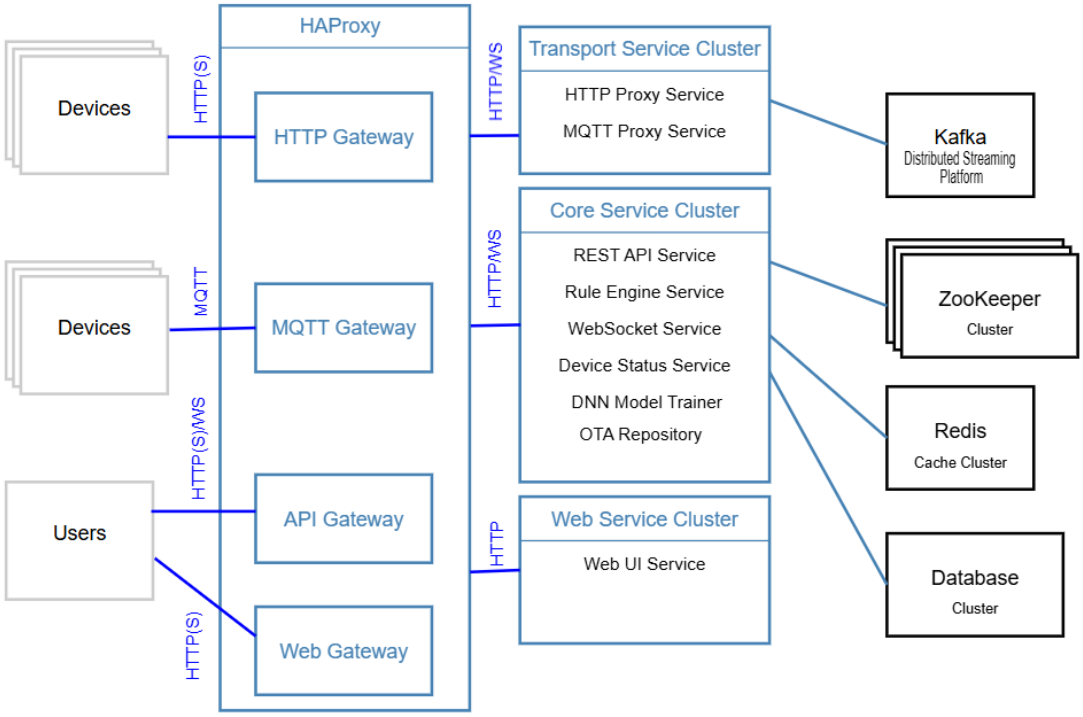



图 5-6 云平面系统架构

Figure 5-6 Cloud plane system architecture

This architecture, based on a microservices framework, offers key advantages such as high scalability, flexibility, and robust stability. In this architecture: The Transmission Microservice Set delivers transmission services through MQTT and HTTP. The Core Microservice Set provides REST API calling services, rule engine services, WebSocket subscription services, and device status detection services. The system employs a hybrid data storage approach: For small volumes of structured data that require high

query response performance, such as device information and system configuration, relational databases like PostgreSQL are utilized. For large-scale, real-time device telemetry data, NoSQL databases such as Cassandra are used. To further enhance system performance, the platform layer incorporates the distributed coordination system ZooKeeper to manage requests from devices to core service components. Additionally, in-memory data storage with Redis is employed for caching data, such as device information and session details.

The DNN Model Trainer Architecture shown on the Figure 5-7.

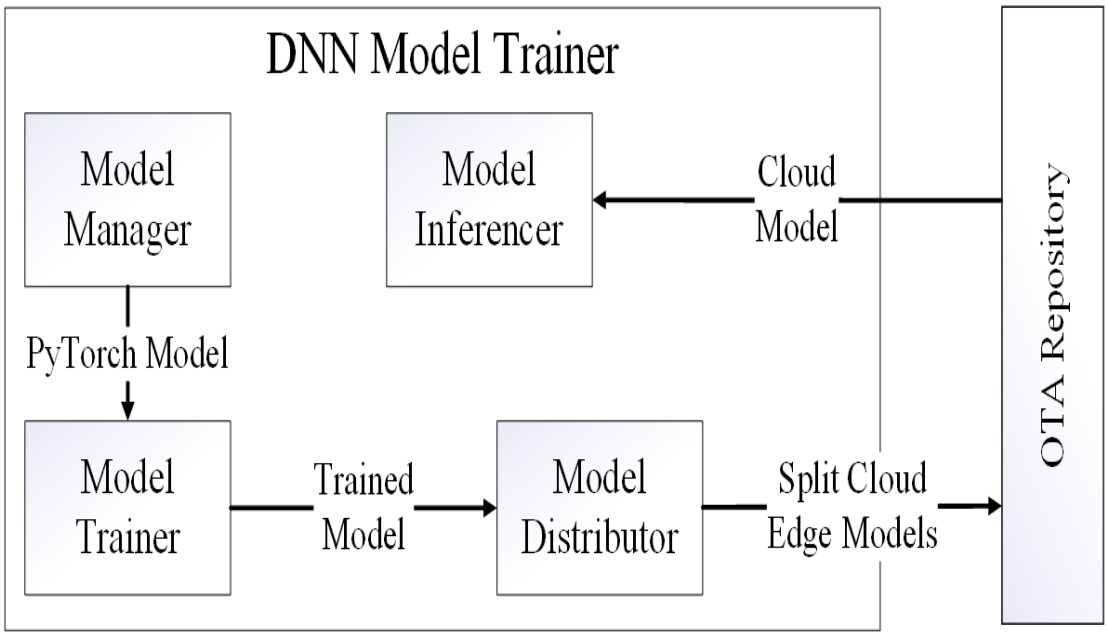



图 5-7 DNN 模型训练器架构

Figure 5-7 DNN Model Trainer Architecture

The DNN Model Trainer consists of several key components, each playing a crucial role in the management and execution of deep learning models across both the cloud and edge layers.

Model Manager: The Model Manager is responsible for overseeing the lifecycle of deep learning models, including their creation, modification, deletion, retrieval, training, deployment, and inference strategies. It ensures the proper management of models at both the cloud and edge layers, coordinating all activities related to model operations.

Model Trainer: The Model Trainer is tasked with training deep learning models according to the strategies outlined by the Model Manager. It adheres to the guidelines set by the Model Manager to ensure correct and efficient training, utilizing appropriate datasets, algorithms, and configurations. The Model Trainer also plays a critical role in implementing various training strategies, such as data preprocessing, model

optimization, and evaluation.

Model Distributor: The Model Splitting and Distributor is responsible for distributing the trained models to the OTA repository, in alignment with the strategies defined by the Model Manager. This component ensures that the models are accessible for deployment and inference on the edge, cloud, or other systems as required.

Model Inference: The Model Inference component is responsible for invoking and executing the trained models based on the inference strategies set by the Model Manager. After a model is trained and stored, Model Inference handles both real-time and batch inference tasks, utilizing the model for predictions, classifications, or decision-making. It ensures that the appropriate model is applied according to predefined strategies and operational needs.

## 5.5 原型实验（Prototype Experiments）

The cloud-edge collaborative fault diagnosis platform has been implemented and tested on mining machinery from Xuzhou Construction Machinery Group, specifically the Mining Track XDE440 and Mining Excavator XE800D, as shown in Figure 5-8.


图 5-8 徐州工程机械集团矿机机械

Figure 5-8 Xuzhou Construction Machinery Group Mining Machinery

Due to the operation of construction machinery in harsh environments and the need to manage a large number of data acquisition devices, an intelligent data acquisition terminal was designed to integrate these devices for installation and use on the vehicle. The intelligent data acquisition terminals are depicted in Figure 5-9, with the technical requirements and parameter specifications detailed in Table 5-2 and Table 5-3.

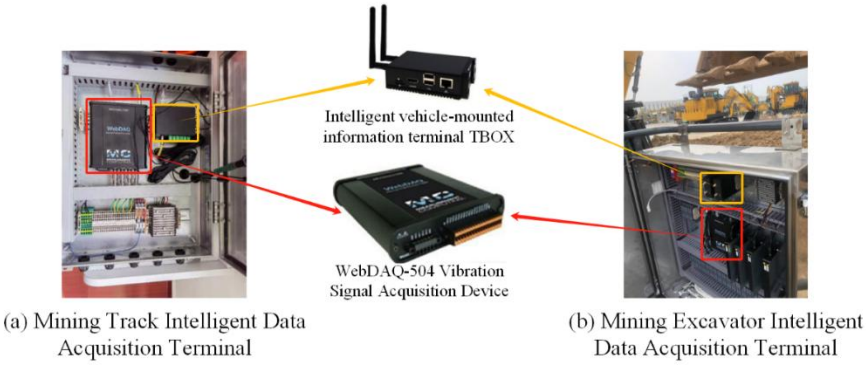



图 5-9 建筑机械智能数据采集终端

Figure 5-9 Construction machinery intelligent data acquisition terminals

表 5-2 矿卡智能数据采集终端参数

Table 5-2 Mining track intelligent data acquisition terminal parameters

| Function | Parameter |
|---|---|
| Shell Material | 304 Stainless Steel |
| Installation Hole Distance | 400*500 |
| Surface Treatment | Polished, maintaining the original color of stainless steel |
| Waterproof Rating | IP65 |
| Power Supply Requirement | DC24V |
| Wiring Outlet Method | Waterproof connector outlet |
| Connector Type/Brand | Weipu |

表 5-3 矿挖智能数据采集终端参数

Table 5-3 Mining excavator intelligent data acquisition terminal parameters

| Function | Parameter |
|---|---|
| Shell Material | 304 Stainless Steel |
| Installation Hole Distance | 613*655 |
| Surface Treatment | Polished, maintaining the original color of stainless steel |
| Waterproof Rating | IP65 |
| Power Supply Requirement | DC24V |
| Wiring Outlet Method | Waterproof connector outlet |
| Connector Type/Brand | Weipu |

The Intelligent Data Acquisition Terminals are strategically installed in environmentally protected areas of the construction machinery shown in the Figure 5-

10, ensuring resilience against harsh operational conditions, while maintaining proximity to the components monitored by the vibration sensors to optimize signal fidelity and data integrity.

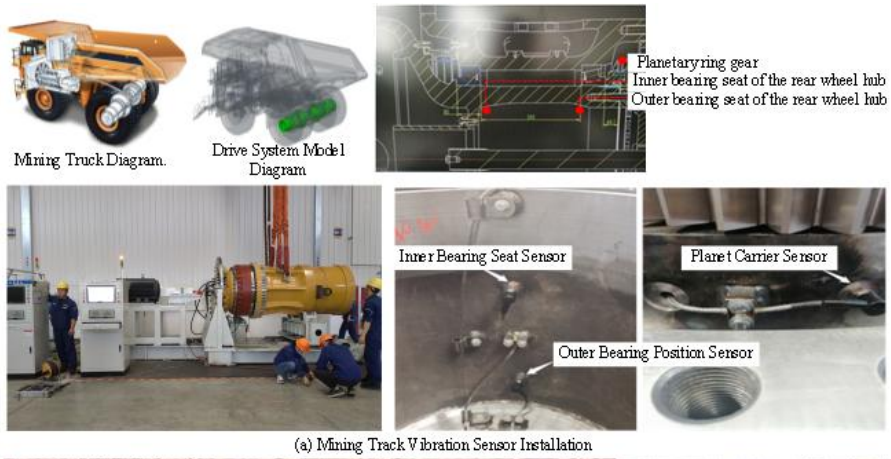
(a) Mining Track Intelligent Data Acquisition Terminal Installation

(b) Mining Excavator Intelligent Data Acquisition Terminal Installation

图 5-10 建筑机械智能数据采集终端

Figure 5-10 Construction machinery intelligent data acquisition terminals

Vibration sensors are mounted on critical components of the construction machinery to enable accurate acquisition of vibration signals originating from the bearings shown in the Figure 5-11.




(a) Mining Track Vibration Sensor Installation


(b) Mining Excavator Vibration Sensor Installation

图 5-11 建筑机械振动传感器安装情况

Figure 5-11 Construction machinery vibration sensor installation

## 5.6 总结（Summary）

In this chapter, the implementation of the cloud-edge collaborative fault diagnosis system for construction machinery was discussed. The chapter begins with the design of the system, highlighting its three main components: the equipment plane, the edge plane, and the cloud plane. Each of these components plays a critical role in ensuring the efficient and adaptive diagnosis of machinery faults. The equipment plane is responsible for data acquisition through various sensors, while the edge plane processes and analyzes data locally, ensuring real-time fault detection. The cloud plane handles the more computationally intensive tasks, such as model training, large-scale data processing, and storage. The system was implemented with a focus on integrating cloud and edge computing technologies, allowing for optimized data processing and reduced reliance on cloud resources. This collaboration addresses the challenges of high latency, network dependency, and bandwidth limitations often encountered in purely cloud-based systems. The edge plane processes data through intelligent algorithms, making local decisions on what data to send to the cloud for further analysis or storage, based on pre-set thresholds. The chapter further explores the interaction between the components, detailing the processes of data collection, preprocessing, fault diagnosis, decision-making, and model updates. The proposed fault diagnosis model was tested and implemented using real-world construction machinery, showing promising results in terms of efficiency, adaptability, and real-time performance. Through this cloud-edge collaboration, the system demonstrates the potential for advanced fault diagnosis and prediction in harsh operational environments, offering significant improvements over traditional methods. The architecture presented in this chapter provides a robust foundation for intelligent fault diagnosis systems in construction machinery, utilizing the strengths of both edge and cloud computing to improve accuracy, reduce computational overhead, and ensure real-time performance.

# 6 结论与未来展望
# 6 Conclusions and Future Perspectives

## 6.1 总结（Conclusions）

This research presented a novel cloud-edge collaborative fault diagnosis system for construction machinery, integrating both edge and cloud computing technologies to enable efficient, real-time fault detection and diagnosis. The proposed system architecture leverages the strengths of edge computing for local data processing, ensuring low-latency decision-making, while the cloud handles more computationally intensive tasks such as model training, storage, and large-scale data processing. This collaboration allows for scalable, robust, and adaptive fault diagnosis in industrial applications, improving the performance and reliability of construction machinery. The experimental results demonstrate that the cloud-edge collaborative approach effectively addresses key challenges, such as network latency, bandwidth constraints, and the need for high-performance fault diagnosis. By partitioning the fault diagnosis model between the edge and cloud, the system ensures optimal resource utilization, with real-time diagnostic capabilities on the edge and enhanced model accuracy on the cloud. Additionally, the system's ability to update models via Over-the-Air (OTA) ensures that the diagnostic models remain up-to-date, improving overall system performance over time. Furthermore, the research explored the use of deep learning models, specifically convolutional neural networks (CNN), in fault diagnosis and the application of multi-sensor data aggregation to enhance classification accuracy. The results highlight the potential for using deep learning models in real-world industrial settings, offering a robust solution for fault detection in harsh environments. This system design offers a promising approach for future advancements in intelligent maintenance and fault diagnosis in the construction machinery sector.

## 6.2 研究创新（Research Innovations）

This research introduces several novel contributions to the field of fault diagnosis for construction machinery, leveraging the integration of cloud and edge computing frameworks. The key innovations of this study include:

(1) Cloud-Edge Collaborative Fault Diagnosis Framework: The study proposes a hybrid fault diagnosis system that effectively combines edge and cloud computing resources. This collaboration enhances the efficiency and scalability of fault detection and diagnosis. By partitioning the model between the edge and cloud, the system

ensures optimized use of resources, with edge devices handling real-time data processing and the cloud providing more computationally intensive model training and data storage.

(2) Adaptive Model Splitting Strategy: A unique model splitting strategy is introduced, where the convolutional neural network (CNN) model is divided between the edge and cloud partitions. This strategy optimizes the computational load, with the edge performing local data processing through the first two filter layers, while the cloud handles the more computationally expensive remaining layers. This approach ensures that real-time diagnostics are achieved on the edge, and model accuracy is maintained in the cloud.

(3) Deep Learning Integration with Multi-Sensor Data: The research also explores the use of deep learning, specifically convolutional neural networks (CNNs), in fault diagnosis tasks. The system aggregates data from multiple sensors, enhancing the classification accuracy and robustness of the diagnosis. The ability to process sensor data in real-time, with a streamlined model on the edge, provides a significant advancement in intelligent fault diagnosis.

(4) OTA Model Updates: The introduction of Over-The-Air (OTA) model updates for both cloud and edge components allow the system to maintain up-to-date models without requiring physical intervention. This facilitates continuous improvement of fault diagnosis capabilities, ensuring the system remains adaptive and capable of handling new fault scenarios.

(5) Efficient Data Management and Decision-Making: The integration of decision-making rules on both the edge and cloud layers ensures that data is handled effectively, with pre-set thresholds guiding the decision-making process for fault classification. This results in more accurate fault detection and faster response times, reducing the time spent in human intervention and increasing overall system reliability.

## 6.3 未来展望（Future Perspectives）

The cloud-edge collaborative fault diagnosis system introduced in this study represents significant progress in enabling real-time fault detection and prediction for construction machinery. However, several promising avenues for future development exist to further enhance its performance and applicability. One potential direction for improvement is the exploration of even more lightweight computing platforms for executing edge models. Although the system currently utilizes lightweight deep learning models, future research could focus on utilizing less powerful computing

devices, such as the ESP32, to run edge models. This would facilitate the deployment of fault diagnosis capabilities on devices with minimal computational resources, thereby improving cost-effectiveness and broadening the system's applicability across various industrial settings. Additionally, further investigation into advanced fault prediction methods, such as reinforcement learning or hybrid models that combine multiple machine learning techniques, could enhance the system's ability to predict faults proactively. These approaches would improve maintenance strategies by forecasting potential failures before they occur. Moreover, optimizing data fusion from multiple sensor types could enhance the robustness and accuracy of fault diagnosis by integrating complementary information from diverse sources. Scalability remains a crucial consideration, particularly as the number of connected devices and sensors expands. To ensure optimal system performance in large-scale industrial environments, further advancements in data storage, transmission techniques, and network protocols will be essential. Incorporating dynamic decision-making processes, as opposed to relying solely on predefined thresholds, could also improve fault detection accuracy. By adjusting system behavior based on real-time data and operational conditions, the system could become more adaptive and responsive to changing circumstances. Finally, integrating domain-specific knowledge or expert systems with machine learning models could enhance the interpretability of diagnostic results, providing valuable insights into the underlying causes of faults and guiding more effective maintenance strategies. In conclusion, while the current system provides a solid foundation for cloud-edge collaborative fault diagnosis in construction machinery, there is substantial potential for further advancements. Key areas for improvement include the application of more lightweight computing solutions, enhanced fault prediction techniques, optimized data fusion, and better scalability to address the growing demands of industrial applications.

# 参考文献


[1] Hoang D T, Kang H J. A survey on deep learning based bearing fault diagnosis[J]. Neurocomputing, 2019, 335: 327-335.

[2] Rai A, Upadhyay S H. A review on signal processing techniques utilized in the fault diagnosis of rolling element bearings[J]. Tribology International, 2016, 96: 289-306.

[3] 杨永灿, 刘韬, 柳小勤, 等. 基于注意力机制的一维卷积神经网络行星齿轮箱故障诊断[J]. 机械与电子, 2021, 39(10): 3-8.

[4] Goyal D, Vanraj, Pabla B S, et al. Condition monitoring parameters for fault diagnosis of fixed axis gearbox: a review[J]. Archives of Computational Methods in Engineering, 2017, 24: 543-556.

[5] Lei Y, Lin J, Zuo M J, et al. Condition monitoring and fault diagnosis of planetary gearboxes: A review[J]. Measurement, 2014, 48: 292-305.

[6] Pandya D H, Upadhyay S H, Harsha S P. Fault diagnosis of rolling element bearing with intrinsic mode function of acoustic emission data using APF-KNN[J]. Expert Systems with Applications, 2013, 40(10): 4137-4145.

[7] Li C, Sanchez R V, Zurita G, et al. Gearbox fault diagnosis based on deep random forest fusion of acoustic and vibratory signals[J]. Mechanical systems and signal processing, 2016, 76: 283-293.

[8] Qian G, Lu S, Pan D, et al. Edge computing: A promising framework for real-time fault diagnosis and dynamic control of rotating machines using multi-sensor data[J]. IEEE Sensors Journal, 2019, 19(11): 4211-4220.

[9] Yang T, Pen H, Wang Z, et al. Feature knowledge based fault detection of induction motors through the analysis of stator current data[J]. IEEE Transactions on instrumentation and Measurement, 2016, 65(3): 549-558.

[10] Zhao Z, Wu S, Qiao B, et al. Enhanced sparse period-group lasso for bearing fault diagnosis[J]. IEEE Transactions on Industrial Electronics, 2018, 66(3): 2143-2153.

[11] Wang S, Chen X, Tong C, et al. Matching synchrosqueezing wavelet transform and application to aeroengine vibration monitoring[J]. IEEE Transactions on Instrumentation and Measurement, 2016, 66(2): 360-372.

[12] 李舜酩, 郭海东, 李殿荣. 振动信号处理方法综述[J]. 仪器仪表学报, 2013, 34(8): 1907-1915.

[13] Inoue T, Sueoka A, Kanemoto H, et al. Detection of minute signs of a small fault in a periodic or a quasi-periodic signal by the harmonic wavelet transform[J]. Mechanical systems and signal processing, 2007, 21(5): 2041-2055.

[14] Lei Y, Lin J, He Z, et al. A review on empirical mode decomposition in fault diagnosis of rotating machinery[J]. Mechanical systems and signal processing, 2013, 35(1-2): 108-126.

[15] Lei, Y.; Yang, B.; Jiang, X.; Jia, F.; Li, N.; Nandi, A.K. Applications of machine learning to machine fault diagnosis: A review and roadmap. Mech. Syst. Signal Process. 2020, 138, 106587.

[16] Lei Y. Intelligent fault diagnosis and remaining useful life prediction of rotating machinery[M]. Butterworth-Heinemann, 2016.

[17] Niu G. Data-driven technology for engineering systems health management[J]. Springer Singapore, 2017, 10: 978-981.

[18] Lu L, Han X, Li J, et al. A review on the key issues for lithium-ion battery management in electric vehicles[J]. Journal of power sources, 2013, 226: 272-288.

[19] Frank P M. Fault diagnosis in dynamic systems using analytical and knowledge-based redundancy: A survey and some new results[J]. automatica, 1990, 26(3): 459-474.

[20] Wu C, Zhu C, Ge Y, et al. A review on fault mechanism and diagnosis approach for li‐ion batteries[J]. Journal of Nanomaterials, 2015, 2015(1): 631263.

[21] Rezvanizaniani S M, Liu Z, Chen Y, et al. Review and recent advances in battery health monitoring and prognostics technologies for electric vehicle (EV) safety and mobility[J]. Journal of power sources, 2014, 256: 110-124.

[22] Zhou D H, Hu Y Y. Fault diagnosis techniques for dynamic systems[J]. Acta Automatica Sinica, 2009, 35(6): 748-758.

[23] Yang F, Xiao D. Model and fault inference with the framework of probabilistic SDG[C]//2006 9th International Conference on Control, Automation, Robotics and Vision. IEEE, 2006: 1-6.

[24] Paté‐Cornell M E. Fault trees vs. event trees in reliability analysis[J]. Risk Analysis, 1984, 4(3): 177-186.

[25] Held M, Brönnimann R. Safe cell, safe battery? Battery fire investigation using FMEA, FTA and practical experiments[J]. Microelectronics Reliability, 2016, 64: 705-710.

[26] Filippetti F, Martelli M, Franceschini G, et al. Development of expert system knowledge base to on-line diagnosis of rotor electrical faults of induction motors[C]//Conference Record of the 1992 IEEE Industry Applications Society Annual Meeting. IEEE, 1992: 92-99.

[27] Isermann R. Model-based fault-detection and diagnosis–status and applications[J]. Annual Reviews in control, 2005, 29(1): 71-85.

[28] Hwang I, Kim S, Kim Y, et al. A survey of fault detection, isolation, and reconfiguration methods[J]. IEEE transactions on control systems technology, 2009, 18(3): 636-653.

[29] Liu Z, Ahmed Q, Zhang J, et al. Structural analysis based sensors fault detection and isolation of cylindrical lithium-ion batteries in automotive applications[J]. Control engineering practice,

2016, 52: 46-58.

[30] Krysander M, Frisk E. Sensor placement for fault diagnosis[J]. IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans, 2008, 38(6): 1398-1410.

[31] Svärd C, Nyberg M. Automated design of an FDI system for the wind turbine benchmark[J]. Journal of Control Science and Engineering, 2012, 2012(1): 989873.

[32] Svärd C, Nyberg M, Frisk E, et al. Automotive engine FDI by application of an automated model-based and data-driven design methodology[J]. Control Engineering Practice, 2013, 21(4): 455-472.

[33] Krysander M, Åslund J, Nyberg M. An efficient algorithm for finding minimal overconstrained subsystems for model-based diagnosis[J]. IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans, 2007, 38(1): 197-206.

[34] Svärd C, Nyberg M, Frisk E. A greedy approach for selection of residual generators[C]//Proceedings of the 22nd International Workshop on Principles of Diagnosis (DX-11). 2011.

[35] Svard C, Nyberg M. Residual generators for fault diagnosis using computation sequences with mixed causality applied to automotive systems[J]. IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans, 2010, 40(6): 1310-1328.

[36] Bagheri F, Khaloozaded H, Abbaszadeh K. Stator fault detection in induction machines by parameter estimation, using adaptive kalman filter[C]//2007 Mediterranean Conference on Control & Automation. IEEE, 2007: 1-6.

[37] Liu X Q, Zhang H Y, Liu J, et al. Fault detection and diagnosis of permanent-magnet DC motor based on parameter estimation and neural network[J]. IEEE transactions on industrial electronics, 2000, 47(5): 1021-1030.

[38] Odendaal H M, Jones T. Actuator fault detection and isolation: An optimised parity space approach[J]. Control Engineering Practice, 2014, 26: 222-232.

[39] Staroswiecki M, Comtet-Varga G. Analytical redundancy relations for fault detection and isolation in algebraic dynamic systems[J]. Automatica, 2001, 37(5): 687-699.

[40] Singh A, Izadian A, Anwar S. Fault diagnosis of Li-Ion batteries using multiple-model adaptive estimation[C]//IECON 2013-39th Annual Conference of the IEEE Industrial Electronics Society. IEEE, 2013: 3524-3529.

[41] Sidhu A, Izadian A, Anwar S. Adaptive nonlinear model-based fault diagnosis of Li-ion batteries[J]. IEEE Transactions on Industrial Electronics, 2014, 62(2): 1002-1011.

[42] Das M, Sadhu S, Ghoshal T K. Fault detection and isolation using an adaptive unscented Kalman filter[J]. IFAC Proceedings Volumes, 2014, 47(1): 326-332.

[43] Alavi S M M, Samadi M F, Saif M. Plating Mechanism Detection in Lithium-ion batteries, by using a particle-filtering based estimation technique[C]//2013 American Control Conference. IEEE, 2013: 4356-4361.

[44] Dey S, Biron Z A, Tatipamula S, et al. On-board thermal fault diagnosis of lithium-ion batteries for hybrid electric vehicle application[J]. IFAC-PapersOnLine, 2015, 48(15): 389-394.

[45] Lin X, Perez H E, Siegel J B, et al. Online parameterization of lumped thermal dynamics in cylindrical lithium ion batteries for core temperature estimation and health monitoring[J]. IEEE Transactions on Control Systems Technology, 2012, 21(5): 1745-1755.

[46] Park S, Himmelblau D M. Fault detection and diagnosis via parameter estimation in lumped dynamic systems[J]. Industrial & Engineering Chemistry Process Design and Development, 1983, 22(3): 482-487.

[47] Feng X, Weng C, Ouyang M, et al. Online internal short circuit detection for a large format lithium ion battery[J]. Applied energy, 2016, 161: 168-180.

[48] Isermann R, Balle P. Trends in the application of model-based fault detection and diagnosis of technical processes[J]. Control engineering practice, 1997, 5(5): 709-719.

[49] Isermann R. Process fault detection based on modeling and estimation methods—A survey[J]. automatica, 1984, 20(4): 387-404.

[50] Kinnaert M. Design of redundancy relations for failure detection and isolation by constrained optimization[J]. International journal of control, 1996, 63(3): 609-622.

[51] Venkatasubramanian V, Rengaswamy R, Kavuri S N. A review of process fault detection and diagnosis: Part II: Qualitative models and search strategies[J]. Computers & chemical engineering, 2003, 27(3): 313-326.

[52] Jack L B, Nandi A K. Support vector machines for detection and characterization of rolling element bearing faults[J]. Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science, 2001, 215(9): 1065-1074.

[53] Xie J, Du G, Shen C, et al. An end-to-end model based on improved adaptive deep belief network and its application to bearing fault diagnosis[J]. IEEE Access, 2018, 6: 63584-63596.

[54] Mao W, Feng W, Liu Y, et al. A new deep auto-encoder method with fusing discriminant information for bearing fault diagnosis[J]. Mechanical Systems and Signal Processing, 2021, 150: 107233.

[55] Huang Y C, Wang P J. Infrared Air Turbine Dental Handpiece Rotor Fault Diagnosis with Convolutional Neural Network[J]. Sensors & Materials, 2020, 32(11): 3545–3558.

[56] Wu Q, Ding K, Huang B. Approach for fault prognosis using recurrent neural network[J]. Journal of Intelligent Manufacturing, 2020, 31(7): 1621-1633.

[57] Goodfellow I J, Pouget-Abadie J, Mirza M, et al. Generative adversarial nets[J]. Advances in neural information processing systems, 2014, 27: 2672–2680.

[58] Tang X, Xu Z, Wang Z. A novel fault diagnosis method of rolling bearing based on integrated vision transformer model[J]. Sensors, 2022, 22(10): 3878.

[59] Li T, Zhao Z, Sun C, et al. Multireceptive field graph convolutional networks for machine fault diagnosis[J]. IEEE Transactions on Industrial Electronics, 2020, 68(12): 12739-12749.

[60] Gou L, Li H, Zheng H, et al. Aeroengine control system sensor fault diagnosis based on CWT and CNN[J]. Mathematical Problems in Engineering, 2020, 2020(1): 5357146.

[61] Jiang J, Bie Y, Li J, et al. Fault diagnosis of the bushing infrared images based on mask R‐CNN and improved PCNN joint algorithm[J]. High voltage, 2021, 6(1): 116-124.

[62] Tran M Q, Liu M K, Tran Q V, et al. Effective fault diagnosis based on wavelet and convolutional attention neural network for induction motors[J]. IEEE Transactions on Instrumentation and Measurement, 2021, 71: 1-13.

[63] Meng S, Kang J, Chi K, et al. Intelligent fault diagnosis of gearbox based on multiple synchrosqueezing S-Transform and convolutional neural networks[J]. International Journal of Performability Engineering, 2020, 16(4): 528.

[64] Chen Y L, Chiang Y, Chiu P H, et al. High-dimensional phase space reconstruction with a convolutional neural network for structural health monitoring[J]. Sensors, 2021, 21(10): 3514.

[65] Guo Q, Li Y, Song Y, et al. Intelligent fault diagnosis method based on full 1-D convolutional generative adversarial network[J]. IEEE Transactions on Industrial Informatics, 2019, 16(3): 2044-2053.

[66] Huang D, Li S, Qin N, et al. Fault diagnosis of high-speed train bogie based on the improved-CEEMDAN and 1-D CNN algorithms[J]. IEEE Transactions on Instrumentation and Measurement, 2021, 70: 1-11.

[67] Ince T, Kiranyaz S, Eren L, et al. Real-time motor fault detection by 1-D convolutional neural networks[J]. IEEE Transactions on Industrial Electronics, 2016, 63(11): 7067-7075.

[68] Zhang W, Peng G, Li C, et al. A new deep learning model for fault diagnosis with good anti-noise and domain adaptation ability on raw vibration signals[J]. Sensors, 2017, 17(2): 425.

[69] Wu X, Peng Z, Ren J, et al. Rub-impact fault diagnosis of rotating machinery based on 1-D convolutional neural networks[J]. IEEE Sensors Journal, 2019, 20(15): 8349-8363.

[70] Du C, Zhang X, Zhong R, et al. Unmanned aerial vehicle rotor fault diagnosis based on interval sampling reconstruction of vibration signals and a one-dimensional convolutional neural network deep learning method[J]. Measurement Science and Technology, 2022, 33(6): 065003.

[71] Wang H, Liu Z, Peng D, et al. Understanding and learning discriminant features based on

multiattention 1DCNN for wheelset bearing fault diagnosis[J]. IEEE Transactions on Industrial Informatics, 2019, 16(9): 5735-5745.

[72] Jiménez-Guarneros M, Morales-Perez C, de Jesus Rangel-Magdaleno J. Diagnostic of combined mechanical and electrical faults in ASD-powered induction motor using MODWT and a lightweight 1-D CNN[J]. IEEE Transactions on Industrial Informatics, 2021, 18(7): 4688-4697.

[73] Khan M A, Kim Y H, Choo J. Intelligent fault detection using raw vibration signals via dilated convolutional neural networks[J]. The Journal of Supercomputing, 2020, 76(10): 8086-8100.

[74] Yongbo L I, Xiaoqiang D U, Fangyi W A N, et al. Rotating machinery fault diagnosis based on convolutional neural network and infrared thermal imaging[J]. Chinese Journal of Aeronautics, 2020, 33(2): 427-438.

[75] Gong B, Du X. Research on analog circuit fault diagnosis based on CBAM-CNN[C]//2021 IEEE International Conference on Electronic Technology, Communication and Information (ICETCI). IEEE, 2021: 258-261.

[76] Ran R, Xu X, Qiu S, et al. Crack-SegNet: surface crack detection in complex background using encoder-decoder architecture[C]//Proceedings of the 2021 4th International Conference on Sensors, Signal and Image Processing. 2021: 15-22.

[77] Zollanvari A, Kunanbayev K, Bitaghsir S A, et al. Transformer fault prognosis using deep recurrent neural network over vibration signals[J]. IEEE Transactions on Instrumentation and Measurement, 2020, 70: 1-11.

[78] Encalada-Dávila Á, Moyón L, Tutivén C, et al. Early fault detection in the main bearing of wind turbines based on gated recurrent unit (GRU) neural networks and SCADA data[J]. IEEE/ASME Transactions On Mechatronics, 2022, 27(6): 5583-5593.

[79] Hao S, Ge F X, Li Y, et al. Multisensor bearing fault diagnosis based on one-dimensional convolutional long short-term memory networks[J]. Measurement, 2020, 159: 107802.

[80] Shi Z, Chehade A. A dual-LSTM framework combining change point detection and remaining useful life prediction[J]. Reliability Engineering & System Safety, 2021, 205: 107257.

[81] Ma M, Mao Z. Deep-convolution-based LSTM network for remaining useful life prediction[J]. IEEE Transactions on Industrial Informatics, 2020, 17(3): 1658-1667.

[82] Yuan M, Wu Y, Lin L. Fault diagnosis and remaining useful life estimation of aero engine using LSTM neural network[C]//2016 IEEE international conference on aircraft utility systems (AUS). IEEE, 2016: 135-140.

[83] Ling J, Liu G J, Li J L, et al. Fault prediction method for nuclear power machinery based on Bayesian PPCA recurrent neural network model[J]. Nuclear Science and Techniques, 2020,

31(8): 75.

[84] Peng K, Jiao R, Dong J, et al. A deep belief network based health indicator construction and remaining useful life prediction using improved particle filter[J]. Neurocomputing, 2019, 361: 19-28.

[85] Zhang Y, Ji J, Ma B. Fault diagnosis of reciprocating compressor using a novel ensemble empirical mode decomposition-convolutional deep belief network[J]. Measurement, 2020, 156: 107619.

[86] Zhang Y, Ji J, Ma B. Reciprocating compressor fault diagnosis using an optimized convolutional deep belief network[J]. Journal of Vibration and Control, 2020, 26(17-18): 1538-1548.

[87] Yu J, Liu G. Knowledge extraction and insertion to deep belief network for gearbox fault diagnosis[J]. Knowledge-Based Systems, 2020, 197: 105883.

[88] Chen Z, Chen X, Li C, et al. Vibration-based gearbox fault diagnosis using deep neural networks[J]. Journal of Vibroengineering, 2017, 19(4): 2475-2496.

[89] Jiang G, Zhao J, Jia C, et al. Intelligent fault diagnosis of gearbox based on vibration and current signals: a multimodal deep learning approach[C]//2019 prognostics and system health management conference (phm-qingdao). IEEE, 2019: 1-6.

[90] Chen Z, Li C, Sánchez R V. Multi-layer neural network with deep belief network for gearbox fault diagnosis[J]. Journal of Vibroengineering, 2015, 17(5): 2379-2392.

[91] He X, Ma J. Weak fault diagnosis of rolling bearing based on FRFT and DBN[J]. Systems Science & Control Engineering, 2020, 8(1): 57-66.

[92] Zhao X, Jia M. A new local-global deep neural network and its application in rotating machinery fault diagnosis[J]. Neurocomputing, 2019, 366: 215-233.

[93] Gao S, Xu L, Zhang Y, et al. Rolling bearing fault diagnosis based on intelligent optimized self-adaptive deep belief network[J]. Measurement Science and Technology, 2020, 31(5): 055009.

[94] Gao S, Xu L, Zhang Y, et al. Rolling bearing fault diagnosis based on SSA optimized self-adaptive DBN[J]. ISA transactions, 2022, 128: 485-502.

[95] Zhang C, He Y, Jiang S, et al. Transformer fault diagnosis method based on self-powered RFID sensor tag, DBN, and MKSVM[J]. IEEE Sensors Journal, 2019, 19(18): 8202-8214.

[96] Lin J, Su L, Yan Y, et al. Prediction method for power transformer running state based on LSTM_DBN network[J]. Energies, 2018, 11(7): 1880.

[97] Li T, Zhou Z, Li S, et al. The emerging graph neural networks for intelligent fault diagnostics and prognostics: A guideline and a benchmark study[J]. Mechanical Systems and Signal

Processing, 2022, 168: 108653.

[98] Sperduti A, Starita A. Supervised neural networks for the classification of structures[J]. IEEE transactions on neural networks, 1997, 8(3): 714-735.

[99] Chen Z, Xu J, Alippi C, et al. Graph neural network-based fault diagnosis: A review[J]. arXiv preprint arXiv:2111.08185, 2021.

[100] Wu Z, Pan S, Chen F, et al. A comprehensive survey on graph neural networks[J]. IEEE transactions on neural networks and learning systems, 2020, 32(1): 4-24.

[101] Chen Z, Xu J, Peng T, et al. Graph convolutional network-based method for fault diagnosis using a hybrid of measurement and prior knowledge[J]. IEEE transactions on cybernetics, 2021, 52(9): 9157-9169.

[102] Tang Y, Zhang X, Zhai Y, et al. Rotating machine systems fault diagnosis using semisupervised conditional random field-based graph attention network[J]. IEEE Transactions on Instrumentation and Measurement, 2021, 70: 1-10.

[103] Liu L, Zhao H, Hu Z. Graph dynamic autoencoder for fault detection[J]. Chemical Engineering Science, 2022, 254: 117637.

[104] Chen D, Liu R, Hu Q, et al. Interaction-aware graph neural networks for fault diagnosis of complex industrial processes[J]. IEEE Transactions on neural networks and learning systems, 2021, 34(9): 6015-6028.

[105] Hudson D A, Manning C D. Compositional attention networks for machine reasoning[J]. arXiv preprint arXiv:1803.03067, 2018.

[106] Hernández A, Amigó J M. Attention mechanisms and their applications to complex systems[J]. Entropy, 2021, 23(3): 283.

[107] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017, 30 : 5998–6008.

[108] Jin Y, Hou L, Chen Y. A time series transformer based method for the rotating machinery fault diagnosis[J]. Neurocomputing, 2022, 494: 379-395.

[109] Ding Y, Jia M, Miao Q, et al. A novel time–frequency Transformer based on self–attention mechanism and its application in fault diagnosis of rolling bearings[J]. Mechanical Systems and Signal Processing, 2022, 168: 108616.

[110] Zhang Z, Song W, Li Q. Dual-aspect self-attention based on transformer for remaining useful life prediction[J]. IEEE Transactions on Instrumentation and Measurement, 2022, 71: 1-11.

[111] Zheng S, Farahat A, Gupta C. Generative adversarial networks for failure prediction[C]//Machine Learning and Knowledge Discovery in Databases: European

Conference, ECML PKDD 2019, Würzburg, Germany, September 16–20, 2019, Proceedings, Part III. Springer International Publishing, 2020: 621-637.

[112] Shao S, Wang P, Yan R. Generative adversarial networks for data augmentation in machine fault diagnosis[J]. Computers in Industry, 2019, 106: 85-93.

[113] Chawla N V, Bowyer K W, Hall L O, et al. SMOTE: synthetic minority over-sampling technique[J]. Journal of artificial intelligence research, 2002, 16: 321-357.

[114] Zhao Z, Li B, Dong R, et al. A surface defect detection method based on positive samples[C]//PRICAI 2018: Trends in Artificial Intelligence: 15th Pacific Rim International Conference on Artificial Intelligence, Nanjing, China, August 28–31, 2018, Proceedings, Part II 15. Springer International Publishing, 2018: 473-481.

[115] Pan T, Chen J, Zhang T, et al. Generative adversarial network in mechanical fault diagnosis under small sample: A systematic review on applications and future perspectives[J]. ISA transactions, 2022, 128: 1-10.

[116] Liu S, Jiang H, Wu Z, et al. Data synthesis using deep feature enhanced generative adversarial networks for rolling bearing imbalanced fault diagnosis[J]. Mechanical Systems and Signal Processing, 2022, 163: 108139.

[117] Li F, Tang T, Tang B, et al. Deep convolution domain-adversarial transfer learning for fault diagnosis of rolling bearings[J]. Measurement, 2021, 169: 108339.

[118] Shao H, Xia M, Wan J, et al. Modified stacked autoencoder using adaptive Morlet wavelet for intelligent fault diagnosis of rotating machinery[J]. IEEE/ASME Transactions on Mechatronics, 2021, 27(1): 24-33.

[119] Ma S, Chen M, Wu J, et al. High-voltage circuit breaker fault diagnosis using a hybrid feature transformation approach based on random forest and stacked autoencoder[J]. IEEE Transactions on Industrial Electronics, 2018, 66(12): 9777-9788.

[120] He Z, Shao H, Ding Z, et al. Modified deep autoencoder driven by multisource parameters for fault transfer prognosis of aeroengine[J]. IEEE Transactions on Industrial Electronics, 2021, 69(1): 845-855.

[121] Xu F, Tse Y L. Roller bearing fault diagnosis using stacked denoising autoencoder in deep learning and Gath–Geva clustering algorithm without principal component analysis and data label[J]. Applied Soft Computing, 2018, 73: 898-913.

[122] Majumdar A. Blind denoising autoencoder[J]. IEEE transactions on neural networks and learning systems, 2018, 30(1): 312-317.

[123] Miao M, Sun Y, Yu J. Sparse representation convolutional autoencoder for feature learning of vibration signals and its applications in machinery fault diagnosis[J]. IEEE Transactions on

Industrial Electronics, 2021, 69(12): 13565-13575.

[124] Remadna I, Terrissa L S, Al Masry Z, et al. RUL prediction using a fusion of attention-based convolutional variational autoencoder and ensemble learning classifier[J]. IEEE Transactions on Reliability, 2022, 72(1): 106-124.

[125] Shen C, Qi Y, Wang J, et al. An automatic and robust features learning method for rotating machinery fault diagnosis based on contractive autoencoder[J]. Engineering Applications of Artificial Intelligence, 2018, 76: 170-184.

[126] Yang Z, Baraldi P, Zio E. A method for fault detection in multi-component systems based on sparse autoencoder-based deep neural networks[J]. Reliability Engineering & System Safety, 2022, 220: 108278.

[127] Chen Z, Li W. Multisensor feature fusion for bearing fault diagnosis using sparse autoencoder and deep belief network[J]. IEEE Transactions on instrumentation and measurement, 2017, 66(7): 1693-1702.

[128] Mell P, Grance T. The NIST Definition of Cloud Computing [R]. NIST Special Publication 800-145, 2011, September: 1–7.

[129] 李林哲, 周佩雷, 程鹏, 等. 边缘计算的架构, 挑战与应用[J]. 大数据, 2019, 5(2): 3-16.

[130] Ioffe S, Szegedy C. Batch normalization: Accelerating deep network training by reducing internal covariate shift[C]//International conference on machine learning. pmlr, 2015: 448-456.

[131] Teerapittayanon S, McDanel B, Kung H T. Branchynet: Fast inference via early exiting from deep neural networks[C]//2016 23rd international conference on pattern recognition (ICPR). IEEE, 2016: 2464-2469.

# 作者简历

## 一、基本情况

姓名：伊斯罗贡 性别：男 民族：乌兹别克 出生年月：1994-09-01 籍贯：乌兹别克斯坦塔什干市

2017-09—2021-06 中国矿业大计算机学院学士

2021-09—2025-06 中国矿业大计算机学院硕士

## 二、获奖情况

1. 2021 年 09 月，中国政府奖学金 CSC

## 三、研究项目

1. 工程机械大扭矩轮毂驱动关键技术及应用示范，国家重点研发计划，项目编号：2019YFB20066400，参加人员。
2. 面向变工况滚动轴承故障诊断的深度学习与域适应研究, 中央高校基本科研业务费专项目资金资助，项目编号：2019ZDPY08，参加人员。
3. 中国矿大徐工矿业智能装备技术研究院，大数据驱动的液压铲智能运维关键技术研究，参与人员。

# 学位论文原创性声明

本人郑重声明：所呈交的学位论文《起重机卷扬系统健康度评价研究》，是本人在导师指导下，在中国矿业大学攻读学位期间进行的研究工作所取得的成果。据我所知，除文中已经标明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

年 月 日

# 学位论文数据集

| 关键词* | 密级* | 中图分类号* | UDC | 论文资助 |
|---|---|---|---|---|
| 云边协同；工程机械；深度学习；智能系统；实时故障诊断 | 公开 | TP311 | 004 | |

| 学位授予单位名称* | 学位授予单位代码* | 学位类别* | 学位级别* |
|---|---|---|---|
| 中国矿业大学 | 10290 | 工学 | 硕士 |

| 论文题名* | 并列题名 | 论文语种* |
|---|---|---|
| 基于云边协同的工程机械故障诊断系统研究 | Research on Fault Diagnosis System for Construction Machinery Based on Cloud Edge Collaboration | 英文 |

| 作者姓名* | 伊斯罗贡 | 学号* | FS21170001 |
|---|---|---|---|
| 培养单位名称* | 培养单位代码* | 培养单位地址 | 邮编 |
| 中国矿业大学 | 10290 | 江苏省徐州市 | 221116 |
| 学科专业* | 研究方向* | 学制* | 学位授予年* |
| 计算机科学与技术 | 智能故障诊断 | 3 | 2025 |
| 论文提交日期* | 2025 年 6 月 | | |
| 导师姓名* | 张博 | 职称* | 副教授 |

| 评阅人 | 答辩委员会主席* | 答辩委员会成员 |
|---|---|---|
| 盲评 | 牛强 | 姚睿，刘兵，徐晓，李仲年 |

电子版论文提交格式　文本（ ✓ ） 图像（　） 视频（　） 音频（　） 多媒体（　） 其他（　）

推荐格式：application/msword; application/pdf

| 电子版论文出版（发布）者 | 电子版论文出版（发布）地 | 权限声明 |
|---|---|---|
| | | |

| 论文总页数* | 117 |
|---|---|

注：共 33 项，其中带*为必填数据，共 22 项。